Reqify

Project Team

Abdul Ahad 22I-2500
Samra Saleem 22I-2727

Session 2022-2026

Supervised by

Dr. Ahmad Raza Shahid

Co-Supervised by

Dr. Shahela Saif

Department of Software Engineering

National University of Computer and Emerging Sciences
Islamabad, Pakistan

June, 2026

Contents

1 Introduction **1**

1.1 Existing Solutions 1

1.1.1 Overview of Existing Tools 2

1.2 Problem Statement 2

1.3 Scope 3

1.4 Modules 4

1.4.1 Pattern-Based Requirement Extractor 4

1.4.2 Requirement Categorization 4

1.4.3 Ambiguity and Completeness Detector 5

1.4.4 Conflict and Consistency Analyzer 5

1.4.5 Requirement Prioritization 5

1.4.6 Change Impact Analyzer 6

1.4.7 Requirement Risk Estimator 6

1.5 Work Division 6

2 Project Requirements **9**

2.1 Use-case 10

2.1.1 Use-case Diagram 10

2.2 Detailed Use Cases 10

2.2.1 Use Case UC01 Import Requirements 10

2.2.2 Use Case UC02 Extract Requirements 12

2.2.3 Use Case UC03 Classify Requirements 14

2.2.4 Use Case UC04 Map Non-Functional Requirements to Quality Attributes 16

2.2.5 Use Case UC05 Analyze Ambiguity in Requirements 17

2.2.6 Use Case UC06 Check Completeness of Requirements 19

2.2.7 Use Case UC07 Detect Conflicts & Duplicates 20

2.2.8 Use Case UC08 Generate Conflict Matrix 22

2.2.9 Use Case UC09 Generate Dependency Graph 24

2.2.10 Use Case UC10 Simulate Requirement Change Impact 25

2.2.11 Use Case UC11 Initiate Prioritization 27

2.2.12 Use Case UC12 Generate Final Priority List 29

2.2.13 Use Case UC13 Assess Requirement Risk 30
2.2.14 Use Case UC14 Generate Risk Report 32
2.3 Functional Requirements 34
2.3.1 Module 1: Pattern-Based Requirement Extractor 34
2.3.2 Module 2: Requirement Categorization 34
2.3.3 Module 3: Ambiguity and Completeness Detector 35
2.3.4 Module 4: Conflict and Consistency Analyzer 35
2.3.5 Module 5: Requirement Prioritization 35
2.3.6 Module 6: Change Impact Analyzer 36
2.3.7 Module 7: Requirement Risk Estimator 36
2.4 Non-Functional Requirements 37
2.4.1 Reliability 37
2.4.2 Usability 37
2.4.3 Performance 38
2.4.4 Security 38

3 System Overview **41**
3.1 Architectural Design 41
3.2 Design Models 41
3.3 Data Design 42
3.4 Domain Model 42

4 Implementation and Testing **45**
4.1 Algorithm Design 45
4.1.1 Algorithm 1: Functional vs Non-Functional Classification 45
4.1.2 Algorithm 2: Non-Functional Requirement Quality Attribute Mapping 46
4.1.3 Algorithm 3: Ambiguity Detection and Scoring 47
4.1.4 Algorithm 4: Conflict and Consistency Analysis 48
4.1.5 Algorithm 5: Requirements Completeness Analyzer 49
4.2 External APIs/SDKs 50
4.3 Testing Details 50
4.3.1 Unit Testing 50
4.3.1.1 Test screenshots 50
4.3.1.2 Test cases 50

5 Conclusions and Future Work **115**
5.1 Conclusion 115
5.2 Future Work 115

References **117**

A Appendices **119**
A.1 Appendix A 119
A.1.1 Use Case Diagram example (Online Shopping System) 119
A.1.2 Detail Use Case Example 120
A.1.3 Event-Response Table for a Highway Intersection 121
A.1.4 Story Board Example For Android App 122
A.2 Appendix B 123
A.2.1 Domain Model Example For Online Shopping 123
A.3 Appendix C 124
A.3.1 Box And Line Example For Online Shopping 124
A.3.2 Architecture Pattern Example For Online Shopping 125
A.4 Appendix D 126
A.4.1 Activity Diagram 126
A.4.2 Class Diagram 127
A.4.3 Sequence Diagram 128
A.4.4 State Transition Diagram 128
A.4.5 Data Flow Diagram 130

List of Figures

2.1 Use Case diagram 10

A.1 Use Case Diagram for the Online Shopping System 119
A.2 Detail Use Case Example 120
A.3 Example of Event Response Table 121
A.4 Example of Story Board 122
A.5 Domain Model Example For Online Shopping Application 123
A.6 Box and Line Diagram For Online Shopping Application 124
A.7 Architecture Pattern For Online Shopping Application 125
A.8 Activity Diagram For Online Shopping Application 126
A.9 Class Diagram For Online Shopping Application 127
A.10 Sequence Diagram For Online Shopping Application 128
A.11 State Transition Diagram For Online Shopping Application 128

List of Tables

1.1 Comparison of Existing Solutions 2
1.2 Work Division 7

2.1 Use Case UC01 Import Requirements 11
2.2 Use Case UC02 Extract Requirements 12
2.3 Use Case UC03 Classify Requirements 14
2.4 Use Case UC04 Map Non-Functional Requirements to Quality Attributes 16
2.5 Use Case UC05 Analyze Ambiguity in Requirements 17
2.6 Use Case UC06 Check Completeness of Requirements 19
2.7 Use Case UC07 Detect Conflicts & Duplicates 21
2.8 Use Case UC08 Generate Conflict Matrix 22
2.9 Use Case UC09 Generate Dependency Graph 24
2.10 Use Case UC10 Simulate Requirement Change Impact 26
2.11 Use Case UC11 Initiate Prioritization 27
2.12 Use Case UC12 Generate Final Priority List 29
2.13 Use Case UC13 Assess Requirement Risk 31
2.14 Use Case UC14 Generate Risk Report 32

4.1 External Libraries and Frameworks Used in Reqify 50
4.2 Unit Testing Results for User Authentication 51
4.3 Unit Testing Results for Requirement Classification (FR/NFR) 51
4.4 Unit Testing Results for NFR Quality Attribute Classification 63
4.5 Unit Testing Results for Ambiguity Detection 67
4.6 Unit Testing Results for Conflict and Consistency Detection 80
4.7 Unit Testing Results for Completeness Analysis 88
4.8 Unit Testing Results for Requirement Prioritization 92
4.9 Unit Testing Results for Change Impact Analyzer 102
4.10 Unit Testing Results for Risk Estimator 111

Chapter 1

Introduction

The requirements engineering phase is a critical part of the Software Development Life Cycle (SDLC), as all design, development, and testing activities depend on well-defined requirements. Poorly written or incomplete requirements often lead to misunderstandings, rework, increased costs, and even project failure. Research from [2] indicates that 60 to 80 percent of project failures are caused by inadequate specifications.

Most requirement analysis is still performed manually using document reviews or keyword-based checks. These traditional methods lack contextual understanding and frequently fail to detect ambiguity, conflicts, missing details, and hidden dependencies. As software systems grow more complex, fully manual verification becomes increasingly inefficient and unreliable.

Reqify, the Intelligent Requirements Assurance and Impact Analysis Platform, aims to strengthen the requirements engineering stage by incorporating automation and intelligence into requirement analysis. The system ensures that requirements are clear, unambiguous, consistent, conflict-free, and aligned. It also provides organized requirement categorization and prioritization, enabling stakeholders to focus on the most critical demands first. As a result, Reqify establishes a stronger foundation for the SDLC, reducing requirement-driven risks and improving overall project success.

1.1 Existing Solutions

Several tools and methodologies have been developed to assist in requirements engineering, aiming to improve the quality, clarity, and consistency of software requirements. While these tools offer valuable features, they often fall short in providing comprehensive, context-aware, and scalable solutions. Below, we review some of the prominent existing systems and their limitations.

1.1.1 Overview of Existing Tools

ScopeMaster detects ambiguities, conflicts, and supports ISO sizing and test generation using rule-based analysis, but lacks contextual understanding.

QVScribe performs linguistic checks and ensures INCOSE/EARS compliance via templates, but is highly template-dependent and lacks correlation analysis.

ReqInOne uses LLMs to structure SRS documents and offers partial IEEE/ISO compliance, but often conflates context with requirements and lacks dependency tracking.

IBM DOORS Next (formerly IBM Rational DOORS) is an enterprise requirements management platform offering traceability, baseline management, and collaboration features, but relies on manual reviews for quality assurance and lacks AI-driven semantic analysis.

Table 1.1: Comparison of Existing Solutions

System Name	System Overview	System Limitations
ScopeMaster	Ambiguity detection, ISO sizing, test generation, quality analysis	Rule-based only, no contextual understanding
QVScribe	Linguistic checks, INCOSE/EARS compliance, template analysis	Template-dependent, no correlation analysis
ReqInOne	LLM-based structuring, partial IEEE/ISO compliance, NLP	Context-requirement mixing, partial compliance
IBM DOORS Next	Enterprise traceability, baseline management, collaboration, change tracking	Manual quality checks, no AI/semantic analysis, high cost

1.2 Problem Statement

Software projects frequently experience delays, cost overruns, and quality issues due to poorly defined requirements. Ambiguities, inconsistencies, and missing details in requirement documents often create misunderstandings between developers and stakeholders, increasing project risks and leading to costly rework. Traditional requirement analysis methods primarily rely on manual reviews and checklists, which are effective at identifying obvious issues but often fail to capture contextual or semantic meaning. As a result, contradictions, redundancies, and hidden dependencies within requirement documents may remain unnoticed until later stages of development, when correcting them becomes significantly more expensive and time-consuming. Requirement changes further

complicate development because even minor modifications can affect multiple dependent features if their impact is not properly analyzed. [1]

Existing automated requirement analysis tools largely depend on rule-based techniques, which lack the ability to interpret domain context or understand natural language semantics effectively. Consequently, many ambiguities and potential risks remain unresolved during early development phases. To address these limitations, this project proposes the development of an AI-powered requirements analysis platform. By leveraging Natural Language Processing (NLP) techniques, the system will automatically analyze requirement documents to detect ambiguities, conflicts, contradictions, and hidden dependencies. The platform will also evaluate the potential impact of requirement changes, enabling stakeholders to make informed decisions early in the development process. Additionally, hierarchical categorization and prioritization of requirements will help development teams focus on the most critical functionalities first. By improving the accuracy and efficiency of requirement analysis, the proposed system aims to reduce requirement-related risks, enhance communication between stakeholders, and improve the overall success rate of software projects.

1.3 Scope

The scope of this project defines what will be included and excluded, helping to set clear expectations for the reader. This project focuses on developing an intelligent platform that enhances the requirements engineering phase through AI and NLP-based automation. The system analyzes structured requirement statements provided by the user, which must follow a standardized format to ensure reliable results in downstream modules. Extracted requirements are categorized into functional and non-functional groups, with non-functional requirements further organized according to quality attributes.

A context-aware ambiguity and quality detector identifies vague terms and ensures clarity. Completeness checks verify that all essential requirements are present, flagging missing elements for stakeholder review. A conflict and consistency analyzer detects contradictions, duplications, or inconsistencies across the requirement set. The platform provides a prioritization advisor, ranking requirements by importance and refining rankings using ambiguity and conflict scores, allowing teams to focus on high-value, well-defined features.

A change impact analyzer traces how modifications to one requirement affect related requirements, enabling informed decisions before implementation. A requirement risk estimator assesses ambiguity, dependency, and volatility risks, highlighting uncertainties that could affect project stability. The system is delivered as a secure, web-based platform using modern technologies, ensuring accessibility and ease of use.

The scope explicitly excludes refinement or rewriting of requirements, as well as later stages of the software development lifecycle, such as coding, testing, or deployment. Advanced features, such as automated generation of design or code artifacts, are not included. Post-deployment support, maintenance, and integration with external project management tools are outside the scope. The platform focuses solely on requirement quality assurance, classification, prioritization, change impact analysis, and risk estimation.

1.4 Modules

Here the modules of our proposed system:

1.4.1 Pattern-Based Requirement Extractor

This module extracts structural elements from requirements using the format [Localization] [Actor|Owner] [Action] [Target|Owned] [Constraint]. To ensure reliable analysis in subsequent modules, users are restricted to input the single actionable statements. Multiple-action statements are automatically flagged and excluded, as they distort semantic embeddings and weaken contextual traceability, resulting in reduced accuracy in downstream analyses. .

1. Collect and parse inputs from Word, PDF, and CSV files
2. Extract the structural elements from each requirement
3. Normalize them through stopword removal and lemmatization
4. Generate structured outputs for further analysis

1.4.2 Requirement Categorization

This module organizes requirements to provide a structured overview for analysis and implementation.

1. Classify requirements into functional and non-functional categories
2. Map non-functional requirements to quality attributes such as performance, security, usability, reliability, and maintainability etc.

1.4.3 Ambiguity and Completeness Detector

This module evaluates the clarity, context, and completeness of requirements by identifying vague expressions, hidden assumptions, and missing elements using NLP-based analysis. For completeness checking, the system is restricted to three predefined application domains: "E-Commerce, Food Delivery, and Learning Management Systems (LMS)".

The module performs the following functions:

1. Analyzes requirements using embeddings to capture semantic meaning and context.
2. Detects ambiguous, vague, or subjective expressions and assigns an ambiguity score.
3. Maps each requirement to domain coverage categories to verify inclusion of all necessary system aspects.
4. Performs binary classification of the particular system requirements as **Complete** or **Incomplete** based on coverage evaluation.

1.4.4 Conflict and Consistency Analyzer

This module ensures that requirements are free from contradictions, duplications, and inconsistencies by analyzing their semantic meaning. The conflict detection system is domain-aware and currently supports 12 specialized domains: e-commerce, healthcare, finance, education, HRMS, CRM, logistics, hotel management, telecommunications, insurance, banking, and restaurant management.

1. Compare requirement pairs to detect similarities and potential duplicates
2. Identify requirements that contradict or are logically incompatible
3. Normalize constraints such as units, dates, and ranges for consistent comparison
4. Summarize them in a structured conflict matrix

1.4.5 Requirement Prioritization

This module provides AI-driven prioritization of requirements by combining stakeholder input with quality metrics.

1. Initially prioritize requirements based on importance (e.g., Must Have, Should Have, Could Have)

2. Refine prioritization using AI-based ambiguity and conflict scores. This ensures that vague or conflicting requirements are given lower priority

3. Generate a final hybrid priority list

1.4.6 Change Impact Analyzer

This module visualizes the ripple effects of requirement changes, enabling informed decision-making.

1. Represent requirements and their dependencies as a graph.

2. Trace both direct and indirect effects of changes

3. Simulate adding, modifying, or removing requirements for stakeholder testing

4. Visualize impacts and classify severity levels as low, medium, or high

1.4.7 Requirement Risk Estimator

This module evaluates risks associated with each requirement to highlight areas of potential concern.

1. Define risk categories such as ambiguity, volatility, and dependency

2. Score each requirement according to these risk dimensions

3. Aggregate individual scores into an overall Risk Index for each requirement

4. Assign risk levels (Low, Medium, High) to make results easy to interpret

5. Generate a comprehensive risk report or matrix, highlighting the most critical requirements that need extra attention.

1.5 Work Division

Here is the table of work division:

Table 1.2: Work Division

Name	Registration	Responsibility/ Module / Feature
Ahad	22i-2500	(Module 1- Feat 1-2) Pattern based Requirement extractor
Samra	22i-2727	(Module 1- Feat 3-4) Pattern based Requirement extractor
Samra	22i-2727	(Module 2- Feat 1) Requirement Categorization
Ahad	22i-2500	(Module 2- Feat 2) Requirement Categorization
Samra	22i-2727	(Module 3- Feat 1-4) Ambiguity and completeness Detector
Ahad	22i-2500	(Module 4- Feat 1-4) Conflict and Consistency Analyzer
Samra	22i-2727	(Module 5- Feat 1-3) Prioritization Advisor
Ahad	22i-2500	(Module 6- Feat 1-4) Change Impact Analyzer
Ahad	22i-2500	(Module 7- Feat 1-3) Requirement Risk Estimator
Samra	22i-2727	(Module 7- Feat 4-5) Requirement Risk Estimator

Chapter 2

Project Requirements

This chapter describes the functional and non-functional requirements of the project.

2.1 Use-case

2.1.1 Use-case Diagram

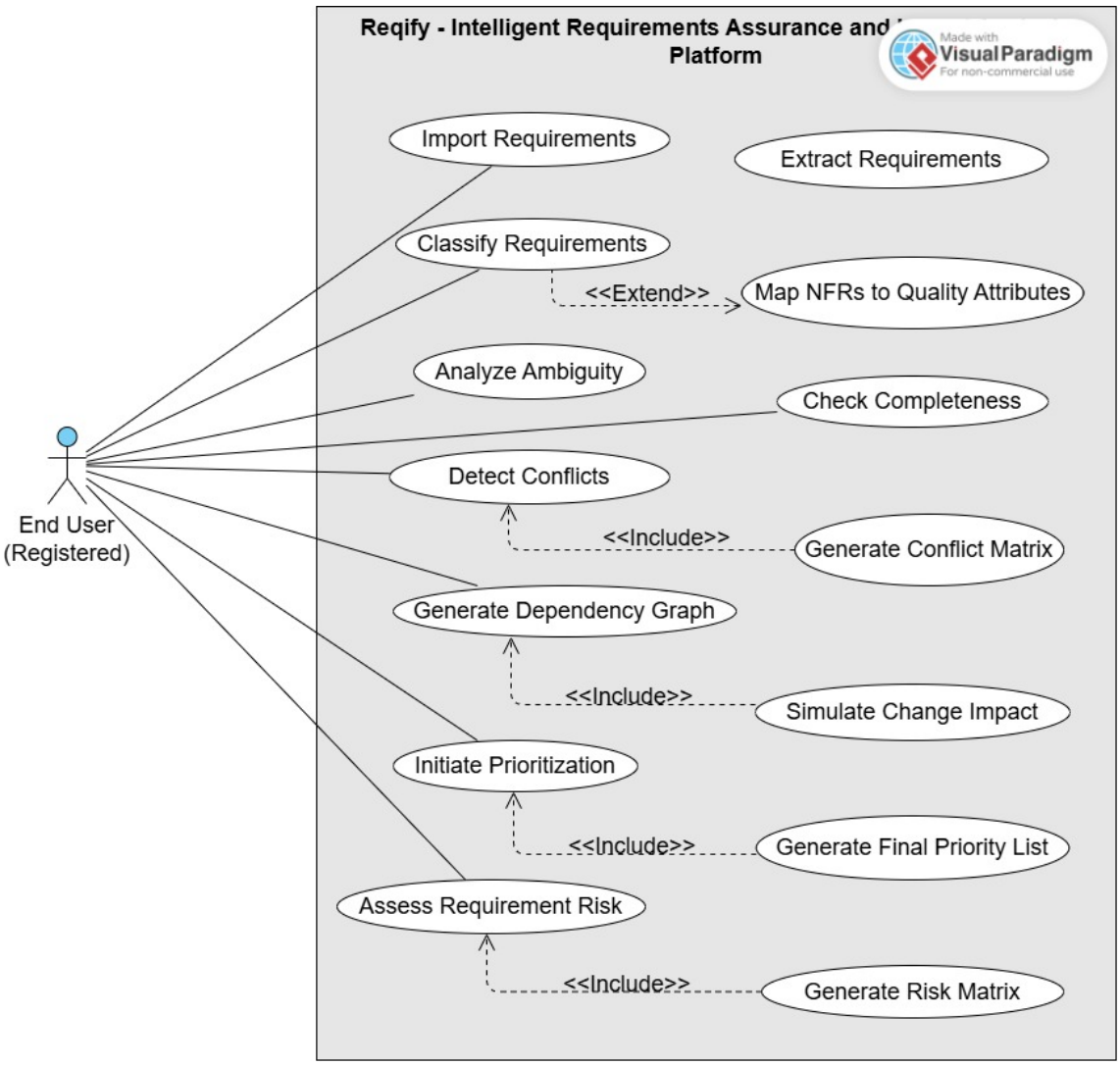

Figure 2.1: Use Case diagram

2.2 Detailed Use Cases

This section provides detailed use case descriptions for the Intelligent Requirements Assurance and Impact Analysis Platform.

2.2.1 Use Case UC01 Import Requirements

Table 2.1: Use Case UC01 Import Requirements

Use Case ID	UC01
Use Case Name	Import Requirements
Scope	Intelligent Requirements Assurance and Impact Analysis Platform
Level	User Goal
Primary Actor	End User
Stakeholders and Interests	• **Project Team (Samra, Ahad):** Need smooth document import to begin requirement analysis. • **Supervisor and Co-Supervisor:** Want reliable input handling that aligns with research objectives. • **University Evaluation Panel:** Requires traceable source documents for assessment. • **University Administration:** Ensures compliance with academic/project standards. • **End Users (Software Development Teams):** Depend on accurate inputs for downstream modules. • **Industry Experts:** Expect robust, industry-valid import processes.
Pre-conditions	• Product Owner is logged in. • Source requirement file is available in Word, PDF, or CSV format.
Success Guarantee	The system successfully imports the requirement file and stores it securely for further processing.

<table>
<tr><td rowspan="4">Main Success Scenario</td><td><table>
<tr><th>User Action</th><th>System Response</th></tr>
<tr><td>1. Product Owner selects Import Requirements.</td><td>2. The system opens a file picker allowing only one file (Word/PDF/CSV).</td></tr>
<tr><td>3. Product Owner uploads the file.</td><td>4. The system validates the file type and size.</td></tr>
<tr><td>5. The system stores the file in the repository with metadata (filename, timestamp, uploader).</td><td>6. The system displays a confirmation message: Requirements Imported Successfully.</td></tr>
</table></td></tr>
<tr><td>Extensions</td><td>• 3a. If the uploaded file is corrupted or unsupported, the system rejects it and shows an error message.
• 4a. If file size exceeds the limit, the system alerts the user and cancels upload.</td></tr>
<tr><td>Special Requirements</td><td>• Only one file can be imported at a time.
• Upload must use a secure connection (TLS) with integrity checks.</td></tr>
<tr><td>Technology and Data Variation List</td><td>• Supported formats: .docx, .pdf, .csv.
• Metadata stored alongside file for traceability.</td></tr>
<tr><td>Frequency of Occurrence</td><td>High at the start of projects; medium when new requirement documents are provided.</td></tr>
<tr><td>Open Issues</td><td>• Handling multi-language files.
• OCR for scanned PDFs not included in MVP.</td></tr>
</table>

2.2.2 Use Case UC02 Extract Requirements

Table 2.2: Use Case UC02 Extract Requirements

Use Case ID	UC02
Use Case Name	Extract Requirements

<table>
<tr><td>Scope</td><td>Intelligent Requirements Assurance and Impact Analysis Platform</td></tr>
<tr><td>Level</td><td>User Goal</td></tr>
<tr><td>Primary Actor</td><td>End User</td></tr>
<tr><td>Stakeholders and Interests</td><td><ul><li>Project Team (Samra, Ahad): Need structured, atomic requirements for analysis.</li><li>Supervisor and Co-Supervisor: Ensure clarity and correctness of extraction process.</li><li>University Evaluation Panel: Review structured requirements for evaluation.</li><li>University Administration: Ensure project follows academic standards.</li><li>End Users (Software Development Teams): Require actionable requirements for implementation.</li><li>Industry Experts: Provide external validation on quality of extracted requirements.</li></ul></td></tr>
<tr><td>Pre-conditions</td><td><ul><li>Exactly one requirement document has been successfully imported into the system (mandatory).</li></ul></td></tr>
<tr><td>Success Guarantee</td><td>The system generates a complete and structured list of requirement statements from the document.</td></tr>
<tr><td>Main Success Scenario</td><td><table>
<tr><th>User Action</th><th>System Response</th></tr>
<tr><td>1. Product Owner selects Extract Requirements.</td><td>2. The system loads the previously imported document.</td></tr>
<tr><td>3. Product Owner clicks Run Extraction.</td><td>4. The system scans the document and identifies structural elements of each requirement.</td></tr>
<tr><td>5. Product Owner waits while the process completes.</td><td>6. The system finalizes extraction, stores the structured output, and displays a confirmation message: Requirements Extraction Completed.</td></tr>
</table></td></tr>
</table>

Extensions	• 3a. If the document is empty or unreadable, the system shows: No extractable requirements found.
Special Requirements	• Each extracted requirement must represent a single intent.
Technology and Data Variation List	• System accepts .docx, .pdf, .csv as input formats.
Frequency of Occurrence	Medium (used each time new documents are imported).
Open Issues	• Support for multi-language documents. • OCR for scanned PDFs (future work).

2.2.3 Use Case UC03 Classify Requirements

Table 2.3: Use Case UC03 Classify Requirements

Use Case ID	UC03
Use Case Name	Classify Requirements
Scope	Intelligent Requirements Assurance and Impact Analysis Platform
Level	User Goal
Primary Actor	End User

<table>
<tr><td>Stakeholders and Interests</td><td>

- **Project Team (Samra, Ahad):** Need requirements structured into functional vs. non-functional categories for clarity.
- **Supervisor and Co-Supervisor:** Want to ensure correct categorization in line with research objectives.
- **University Evaluation Panel:** Require structured classification for evaluation.
- **University Administration:** Ensure adherence to standards.
- **End Users (Software Development Teams):** Depend on categorized requirements for design and development.
- **Industry Experts:** Validate the classification scheme against industry practice.

</td></tr>
<tr><td>Pre-conditions</td><td>

- A set of extracted requirements exists in the system.

</td></tr>
<tr><td>Success Guarantee</td><td>All requirements are categorized as either Functional or Non-Functional.</td></tr>
<tr><td>Main Success Scenario</td><td>
<table>
<tr><th>User Action</th><th>System Response</th></tr>
<tr><td>1. Product Owner selects Classify Requirements.</td><td>2. The system loads the available extracted requirements.</td></tr>
<tr><td>3. Product Owner initiates classification.</td><td>4. The system analyzes each requirement.</td></tr>
<tr><td>5. The system labels requirements as Functional or Non-Functional.</td><td>6. The system saves the classification and shows confirmation: Classification Completed.</td></tr>
</table>
</td></tr>
<tr><td>Extensions</td><td>

- 4a. If a requirement cannot be clearly classified, the system flags it for manual review.

</td></tr>
<tr><td>Special Requirements</td><td>

- Each requirement must be assigned to exactly one category (Functional or Non-Functional).

</td></tr>
<tr><td>Technology and Data Variation List</td><td>

- Categories stored in system database linked with requirement IDs.

</td></tr>
</table>

Frequency of Occurrence	Medium (performed once per requirement set, repeated after updates).
Open Issues	• Handling ambiguous requirements that partially overlap categories.

2.2.4 Use Case UC04 Map Non-Functional Requirements to Quality Attributes

Table 2.4: Use Case UC04 Map Non-Functional Requirements to Quality Attributes

Use Case ID	UC04
Use Case Name	Map Non-Functional Requirements to Quality Attributes
Scope	Intelligent Requirements Assurance and Impact Analysis Platform
Level	User Goal
Primary Actor	End User
Stakeholders and Interests	• **Project Team (Samra, Ahad):** Need detailed NFR attributes for structured analysis. • **Supervisor and Co-Supervisor:** Want mapping aligned with project objectives. • **University Evaluation Panel:** Require structured mapping for assessment. • **University Administration:** Ensure consistency with academic project standards. • **End Users (Software Development Teams):** Depend on mapped NFRs for design and implementation. • **Industry Experts:** Validate mapping quality against industry practices.
Pre-conditions	• Requirements have been classified as Non-Functional.
Success Guarantee	Each Non-Functional Requirement is linked to an appropriate quality attribute.

<table>
<tr><td rowspan="4">Main Success Scenario</td><td><table>
<tr><th>User Action</th><th>System Response</th></tr>
<tr><td>1. Product Owner selects Map NFRs to Quality Attributes.</td><td>2. The system loads all Non-Functional Requirements.</td></tr>
<tr><td>3. Product Owner initiates mapping.</td><td>4. The system analyzes each NFR.</td></tr>
<tr><td>5. The system assigns each NFR to exactly one quality attribute (e.g., performance, security, usability, reliability, maintainability).</td><td>6. The system saves mappings and shows confirmation: NFR Mapping Completed.</td></tr>
</table></td></tr>
<tr><td>Extensions</td><td>• 4a. If an NFR cannot be matched automatically, the system flags it for manual assignment.</td></tr>
<tr><td>Special Requirements</td><td>• Each NFR must be linked to exactly one quality attribute.</td></tr>
<tr><td>Technology and Data Variation List</td><td>• Attributes based on general IEEE software engineering standards (no specific model required).</td></tr>
<tr><td>Frequency of Occurrence</td><td>Medium (after classification or when new NFRs are added).</td></tr>
<tr><td>Open Issues</td><td>• Some NFRs may not fit neatly into predefined attributes, requiring human judgment for proper mapping.</td></tr>
</table>

2.2.5 Use Case UC05 Analyze Ambiguity in Requirements

Table 2.5: Use Case UC05 Analyze Ambiguity in Requirements

Use Case ID	UC05
Use Case Name	Analyze Ambiguity in Requirements
Scope	Intelligent Requirements Assurance and Impact Analysis Platform

<table>
<tr><td>Level</td><td>User Goal</td></tr>
<tr><td>Primary Actor</td><td>End User</td></tr>
<tr><td>Stakeholders and Interests</td><td>• Project Team (Samra, Ahad): Need ambiguous requirements flagged for correction.
• Supervisor and Co-Supervisor: Ensure analysis is research-aligned.
• University Evaluation Panel: Require clarity in requirements for assessment.
• End Users (Software Development Teams): Depend on unambiguous requirements for implementation.
• Industry Experts: Validate clarity for industry use.</td></tr>
<tr><td>Pre-conditions</td><td>• A set of extracted requirements exists in the system.</td></tr>
<tr><td>Success Guarantee</td><td>Each requirement is analyzed for semantic meaning and context, and an ambiguity score is generated.</td></tr>
<tr><td>Main Success Scenario</td><td><table>
<tr><th>User Action</th><th>System Response</th></tr>
<tr><td>1. Product Owner selects Analyze Ambiguity.</td><td>2. The system loads all available requirements.</td></tr>
<tr><td>3. Product Owner starts the analysis.</td><td>4. The system analyzes the ambiguity of each requirement.</td></tr>
<tr><td>5. The system assigns an ambiguity score to each requirement.</td><td>6. The system stores the results and displays a summary: Ambiguity Analysis Completed.</td></tr>
</table></td></tr>
<tr><td>Extensions</td><td>• 4a. If the requirement set is too small or incomplete, the system notifies: Insufficient data to perform ambiguity analysis.</td></tr>
<tr><td>Technology and Data Variation List</td><td>• Scores stored with requirement IDs for later reference.</td></tr>
<tr><td>Frequency of Occurrence</td><td>Medium (after each extraction or major update).</td></tr>
</table>

Open Issues	• Semantic models may misinterpret domain-specific terminology, requiring human oversight.

2.2.6 Use Case UC06 Check Completeness of Requirements

Table 2.6: Use Case UC06 Check Completeness of Requirements

Use Case ID	UC06
Use Case Name	Check Completeness of Requirements
Scope	Intelligent Requirements Assurance and Impact Analysis Platform
Level	User Goal
Primary Actor	End User
Stakeholders and Interests	• **Project Team (Samra, Ahad):** Need coverage check to ensure no gaps. • **Supervisor and Co-Supervisor:** Want systematic completeness verification. • **University Evaluation Panel:** Assess completeness as part of evaluation. • **University Administration:** Ensure compliance with academic standards. • **End Users (Software Development Teams):** Depend on complete requirements for full implementation. • **Industry Experts:** Validate coverage against industry expectations.
Pre-conditions	• Requirements are already present in the system and available for analysis.
Success Guarantee	The system produces a Completeness Report showing mapped categories and highlighting missing or incomplete requirements.

Main Success Scenario	**User Action**	**System Response**
	1. Product Owner selects Check Completeness of Requirements.	2. The system loads all available requirements.
	3. Product Owner initiates the completeness check.	4. The system maps each requirement to predefined domain coverage categories to verify inclusion of all necessary system aspects.
	–	5. The system assign the binary label as Complete or Incomplete.
	–	6. The system generates the Completeness Report, displays it, and provides an option to export.
Extensions	• 4a. If a requirement cannot be mapped to any category, the system flags it for manual assignment.	
Special Requirements	• The Completeness Report must clearly indicate coverage gaps and highlight all requirements marked as incomplete for review.	
Technology and Data Variation List	• Report exportable in PDF and CSV formats.	
Frequency of Occurrence	Medium (after imports or major requirement updates).	
Open Issues	• Coverage categories may need to be updated for different domains or evolving project requirements.	

2.2.7 Use Case UC07 Detect Conflicts & Duplicates

Table 2.7: Use Case UC07 Detect Conflicts & Duplicates

Use Case ID	UC07
Use Case Name	Detect Conflicts & Duplicates
Scope	Intelligent Requirements Assurance and Impact Analysis Platform
Level	User Goal
Primary Actor	End User
Stakeholders and Interests	• **Project Team (Samra, Ahad):** Avoid rework by catching contradictions and duplicates early. • **Supervisor and Co-Supervisor:** Ensure internal consistency for evaluation. • **University Evaluation Panel:** Review consistency as part of project quality. • **End Users (Software Development Teams):** Need a consistent set to design/implement reliably. • **Industry Experts:** Expect conflict-free requirements for real-world feasibility.
Pre-conditions	• Requirements are already present in the system and available for analysis.
Success Guarantee	The system identifies potential conflicts and duplicates, and marks affected requirements for review.

<table>
<tr><td rowspan="5">Main Success Scenario</td><td><table>
<tr><th>User Action</th><th>System Response</th></tr>
<tr><td>1. Product Owner selects Detect Conflicts & Duplicates.</td><td>2. The system loads all available requirements.</td></tr>
<tr><td>3. Product Owner initiates the detection process.</td><td>4. The system compares requirements to find potential duplicates.</td></tr>
<tr><td>–</td><td>5. The system flags logically incompatible or contradictory requirements.</td></tr>
<tr><td>–</td><td>6. The system summarizes findings and shows a review list of detected duplicates and conflicts.</td></tr>
</table></td></tr>
<tr><td>Extensions</td><td>• 5a. If a potential issue is low-confidence, the system marks it Needs Manual Review.</td></tr>
<tr><td>Special Requirements</td><td>• Each detected issue must include a clear rationale (brief explanation) to support reviewer decisions.</td></tr>
<tr><td>Technology and Data Variation List</td><td>• Findings stored with requirement IDs and issue type (duplicate/conflict).</td></tr>
<tr><td>Frequency of Occurrence</td><td>Medium (after major edits or imports).</td></tr>
<tr><td>Open Issues</td><td>• Edge cases where similar wording has different intent may cause false positives.</td></tr>
</table>

2.2.8 Use Case UC08 Generate Conflict Matrix

Table 2.8: Use Case UC08 Generate Conflict Matrix

Use Case ID	UC08

<table>
<tr><td>Use Case Name</td><td>Generate Conflict Matrix</td></tr>
<tr><td>Scope</td><td>Intelligent Requirements Assurance and Impact Analysis Platform</td></tr>
<tr><td>Level</td><td>User Goal</td></tr>
<tr><td>Primary Actor</td><td>End User</td></tr>
<tr><td>Stakeholders and Interests</td><td><ul><li>Project Team (Samra, Ahad): Need a structured view to plan resolutions.</li><li>Supervisor and Co-Supervisor: Assess breadth and severity of issues.</li><li>University Evaluation Panel: Consider matrix evidence in grading.</li><li>End Users (Software Development Teams): Prioritize fixes based on impact.</li><li>Industry Experts: Expect a concise artifact for review and validation.</li></ul></td></tr>
<tr><td>Pre-conditions</td><td><ul><li>Results from Detect Conflicts & Duplicates are already available in the system (mandatory).</li></ul></td></tr>
<tr><td>Success Guarantee</td><td>The system produces a Conflict Matrix summarizing duplicates and conflicts, with status and severity clearly indicated.</td></tr>
<tr><td>Main Success Scenario</td><td><table>
<tr><th>User Action</th><th>System Response</th></tr>
<tr><td>1. Product Owner selects Generate Conflict Matrix.</td><td>2. The system loads the existing detection results.</td></tr>
<tr><td>3. Product Owner initiates matrix generation.</td><td>4. The system organizes identified issues into a matrix.</td></tr>
<tr><td>–</td><td>5. The system highlights high-severity issues for immediate attention.</td></tr>
<tr><td>–</td><td>6. The system displays the Conflict Matrix and provides options to export it.</td></tr>
</table></td></tr>
<tr><td>Extensions</td><td><ul><li>4a. If some detected issues lack severity labels, the system marks them as Unclassified and prompts for manual review.</li></ul></td></tr>
</table>

Special Requirements	• The Conflict Matrix must present conflicts and duplicates in a clear tabular structure with unique references for traceability.
Technology and Data Variation List	• Export formats supported: PDF, CSV.
Frequency of Occurrence	Medium (after each detection run or major requirement changes).
Open Issues	• Deciding how to handle overlapping conflicts (e.g., when one requirement conflicts with multiple others in different ways).

2.2.9 Use Case UC09 Generate Dependency Graph

Table 2.9: Use Case UC09 Generate Dependency Graph

Use Case ID	UC09
Use Case Name	Generate Dependency Graph
Scope	Intelligent Requirements Assurance and Impact Analysis Platform
Level	User Goal
Primary Actor	Product Owner
Stakeholders and Interests	• **Project Team (Samra, Ahad):** Need a dependency graph for clarity and analysis. • **Supervisor and Co-Supervisor:** Ensure correct mapping of requirement relationships. • **University Evaluation Panel:** Assess visualization as part of project evaluation. • **End Users (Software Development Teams):** Use dependency view for design and risk planning. • **Industry Experts:** Validate dependency mapping approach.

<table>
<tr><td>Pre-conditions</td><td>• Requirements are already present in the system and available for mapping.</td></tr>
<tr><td>Success Guarantee</td><td>The system generates a dependency graph that shows how requirements are related to one another.</td></tr>
<tr><td>Main Success Scenario</td><td>
<table>
<tr><th>User Action</th><th>System Response</th></tr>
<tr><td>1. Product Owner selects Generate Dependency Graph.</td><td>2. The system loads all requirements in the project.</td></tr>
<tr><td>3. Product Owner starts the graph generation.</td><td>4. The system analyzes relationships and dependencies among requirements.</td></tr>
<tr><td>–</td><td>5. The system generates the dependency graph structure.</td></tr>
<tr><td>–</td><td>6. The system displays the graph visually with options to zoom, filter, and export.</td></tr>
</table>
</td></tr>
<tr><td>Extensions</td><td>• 4a. If some requirements have no dependencies, the system still includes them as isolated nodes in the graph.</td></tr>
<tr><td>Special Requirements</td><td>• The dependency graph must represent all requirements and their relationships clearly, ensuring each node and link is uniquely identifiable.</td></tr>
<tr><td>Technology and Data Variation List</td><td>• Export formats supported: PDF, PNG.</td></tr>
<tr><td>Frequency of Occurrence</td><td>Medium (used at project start and after major updates).</td></tr>
<tr><td>Open Issues</td><td>• Ensuring the graph remains readable and uncluttered as the number of requirements grows.</td></tr>
</table>

2.2.10 Use Case UC10 Simulate Requirement Change Impact

Table 2.10: Use Case UC10 Simulate Requirement Change Impact

Use Case ID	UC10
Use Case Name	Simulate Requirement Change Impact
Scope	Intelligent Requirements Assurance and Impact Analysis Platform
Level	User Goal
Primary Actor	End User
Stakeholders and Interests	• **Project Team (Samra, Ahad):** Need to anticipate ripple effects of changes. • **Supervisor and Co-Supervisor:** Require structured simulations for evaluation. • **University Evaluation Panel:** Check capability to simulate impact for grading. • **End Users (Software Development Teams):** Understand how a change affects dependencies. • **Industry Experts:** Validate impact analysis for practical use.
Pre-condition	• A dependency graph has already been generated for the project.
Success Guarantee	The system simulates the impact of a selected change and shows affected requirements with severity levels.

<table>
<tr><td rowspan="5">Main Success Scenario</td><td><table>
<tr><th>User Action</th><th>System Response</th></tr>
<tr><td>1. Product Owner selects Simulate Change Impact.</td><td>2. The system loads the existing dependency graph.</td></tr>
<tr><td>3. Product Owner chooses a requirement to add, modify, or remove.</td><td>4. The system traces direct and indirect dependencies linked to that change.</td></tr>
<tr><td>–</td><td>5. The system identifies all affected requirements and classifies the severity (Low, Medium, High).</td></tr>
<tr><td>–</td><td>6. The system displays the simulation results and allows export for reporting.</td></tr>
</table></td></tr>
<tr><td>Extension</td><td>• 3a. If the chosen requirement is isolated, the system reports No impact detected.</td></tr>
<tr><td>Special Requirement</td><td>• Each simulation must produce a clear list of affected requirements with severity levels.</td></tr>
<tr><td>Technology and Data Variation List</td><td>• Simulation results exportable in PDF and CSV formats.</td></tr>
<tr><td>Frequency of Occurrence</td><td>Medium (performed whenever a change is being evaluated).</td></tr>
<tr><td>Open Issues</td><td>• Defining severity consistently across different types of requirement changes.</td></tr>
</table>

2.2.11 Use Case UC11 Initiate Prioritization

Table 2.11: Use Case UC11 Initiate Prioritization

Use Case ID	UC11
Use Case Name	Initiate Prioritization

<table>
<tr><td>Scope</td><td>Intelligent Requirements Assurance and Impact Analysis Platform</td></tr>
<tr><td>Level</td><td>User Goal</td></tr>
<tr><td>Primary Actor</td><td>End User</td></tr>
<tr><td>Stakeholders and Interests</td><td><ul><li>Project Team (Samra, Ahad): Need a clear baseline of requirement importance.</li><li>Supervisor and Co-Supervisor: Want transparent prioritization process.</li><li>University Evaluation Panel: Evaluate how requirements are prioritized for feasibility.</li><li>End Users (Software Development Teams): Depend on priority levels for planning implementation.</li><li>Industry Experts: Review prioritization against industry practices.</li></ul></td></tr>
<tr><td>Pre-condition</td><td><ul><li>Requirements are already present in the system.</li></ul></td></tr>
<tr><td>Success Guarantee</td><td>Each requirement is assigned an initial priority: Must Have, Should Have, or Could Have.</td></tr>
<tr><td>Main Success Scenario</td><td><table>
<tr><th>User Action</th><th>System Response</th></tr>
<tr><td>1. Product Owner selects Assign Requirement Priorities.</td><td>2. The system loads all requirements.</td></tr>
<tr><td>3. Product Owner initiates priority assignment.</td><td>4. The system assigns each requirement to Must Have, Should Have, or Could Have based on stakeholder input or predefined rules.</td></tr>
<tr><td>–</td><td>5. The system saves the assigned priorities.</td></tr>
<tr><td>–</td><td>6. The system confirms: Initial Prioritization Completed.</td></tr>
</table></td></tr>
<tr><td>Extension</td><td><ul><li>4a. If a requirement does not fit clearly into any category, the system flags it as Unassigned Needs Review.</li></ul></td></tr>
</table>

Special Requirement	• Each requirement must belong to exactly one priority category.
Technology and Data Variation List	• Export to CSV or PDF for stakeholder review.
Frequency of Occurrence	Medium (at the start of project planning).
Open Issues	• Different stakeholders may disagree on initial priority consensus rules may be needed.

2.2.12 Use Case UC12 Generate Final Priority List

Table 2.12: Use Case UC12 Generate Final Priority List

Use Case ID	UC12
Use Case Name	Generate Final Priority List
Scope	Intelligent Requirements Assurance and Impact Analysis Platform
Level	User Goal
Primary Actor	End User
Stakeholders and Interests	• **Project Team (Samra, Ahad):** Need an accurate, conflict-aware priority list. • **Supervisor and Co-Supervisor:** Want refinement based on system analysis. • **University Evaluation Panel:** Evaluate quality of final priority results. • **End Users (Software Development Teams):** Use refined list for phased implementation. • **Industry Experts:** Validate priority scoring logic.

<table>
<tr><td>Pre-conditions</td><td>• Initial priorities are already assigned (UC11 completed).
• Ambiguity and conflict analysis results are available in the system.</td></tr>
<tr><td>Success Guarantee</td><td>The system generates a final hybrid priority list, refined by ambiguity and conflict scores.</td></tr>
<tr><td>Main Success Scenario</td><td><table>
<tr><th>User Action</th><th>System Response</th></tr>
<tr><td>1. Product Owner selects Generate Final Priority List.</td><td>2. The system loads requirements with their initial priority categories.</td></tr>
<tr><td>3. Product Owner initiates refinement.</td><td>4. The system adjusts priorities using ambiguity and conflict scores.</td></tr>
<tr><td>–</td><td>5. The system compiles the hybrid priority list.</td></tr>
<tr><td>–</td><td>6. The system displays the final list and provides an export option.</td></tr>
</table></td></tr>
<tr><td>Extensions</td><td>• 4a. If no ambiguity/conflict results are available, the system notifies: Run Ambiguity & Conflict Analysis first.</td></tr>
<tr><td>Special Requirements</td><td>• Final list must show both the original priority and the refined score adjustment for transparency.</td></tr>
<tr><td>Technology and Data Variation List</td><td>• Exportable in CSV and PDF formats.</td></tr>
<tr><td>Frequency of Occurrence</td><td>Medium (done after initial prioritization and analyses).</td></tr>
<tr><td>Open Issues</td><td>• Balancing automated scoring with stakeholder judgment in refinement.</td></tr>
</table>

2.2.13 Use Case UC13 Assess Requirement Risk

Table 2.13: Use Case UC13 Assess Requirement Risk

<table>
<tr><td>Use Case ID</td><td>UC13</td></tr>
<tr><td>Use Case Name</td><td>Assess Requirement Risk</td></tr>
<tr><td>Scope</td><td>Intelligent Requirements Assurance and Impact Analysis Platform</td></tr>
<tr><td>Level</td><td>User Goal</td></tr>
<tr><td>Primary Actor</td><td>End User</td></tr>
<tr><td>Stakeholders and Interests</td><td>• Project Team (Samra, Ahad): Identify risky requirements early.
• Supervisor and Co-Supervisor: Assess if risks are being addressed systematically.
• University Evaluation Panel: Evaluate rigor of projects risk assessment.
• End Users (Software Development Teams): Focus attention on requirements with higher risk.
• Industry Experts: Validate risk scoring approach for practical adoption.</td></tr>
<tr><td>Pre-conditions</td><td>• Requirements are already present in the system.</td></tr>
<tr><td>Success Guarantee</td><td>Each requirement is assigned risk scores and a corresponding risk level (Low, Medium, High).</td></tr>
<tr><td>Main Success Scenario</td><td>
<table>
<tr><th>User Action</th><th>System Response</th></tr>
<tr><td>1. Product Owner selects Assess Requirement Risk.</td><td>2. The system loads all available requirements.</td></tr>
<tr><td>3. Product Owner initiates risk assessment.</td><td>4. The system scores each requirement across risk categories (ambiguity, volatility, dependency).</td></tr>
<tr><td>–</td><td>5. The system calculates an overall Risk Index per requirement.</td></tr>
<tr><td>–</td><td>6. The system assigns a risk level (Low, Medium, High) and stores the results.</td></tr>
</table>
</td></tr>
</table>

Extensions	• 4a. If a requirement lacks sufficient data for scoring, the system marks it Unscored Needs Review.
Special Requirements	• The risk assessment must provide both a numerical score and a categorical level for each requirement to ensure clarity in evaluation.
Technology and Data Variation List	• Risk scoring stored with requirement IDs.
Frequency of Occurrence	Medium (after prioritization or major updates).
Open Issues	• Different projects may define risk categories differently, requiring customization.

2.2.14 Use Case UC14 Generate Risk Report

Table 2.14: Use Case UC14 Generate Risk Report

Use Case ID	UC14
Use Case Name	Generate Risk Report
Scope	Intelligent Requirements Assurance and Impact Analysis Platform
Level	User Goal
Primary Actor	End User

<table>
<tr><td>Stakeholders and Interests</td><td>

- **Project Team (Samra, Ahad):** Need a structured report to guide decisions.
- **Supervisor and Co-Supervisor:** Evaluate comprehensiveness of risk findings.
- **University Evaluation Panel:** Use report evidence in grading.
- **End Users (Software Development Teams):** Focus testing and mitigation on high-risk items.
- **Industry Experts:** Expect industry-standard reporting for validation.

</td></tr>
<tr><td>Pre-conditions</td><td>

- Risk assessment results (UC13) are available in the system.

</td></tr>
<tr><td>Success Guarantee</td><td>The system generates a Risk Report/Matrix summarizing all requirements with their risk levels and scores.</td></tr>
<tr><td>Main Success Scenario</td><td>
<table>
<tr><th>User Action</th><th>System Response</th></tr>
<tr><td>1. Product Owner selects Generate Risk Report.</td><td>2. The system loads the latest risk assessment results.</td></tr>
<tr><td>3. Product Owner initiates report generation.</td><td>4. The system compiles requirement IDs, risk categories, scores, and assigned levels.</td></tr>
<tr><td>–</td><td>5. The system organizes results into a Risk Matrix highlighting critical items.</td></tr>
<tr><td>–</td><td>6. The system displays the Risk Report and provides export options.</td></tr>
</table>
</td></tr>
<tr><td>Extensions</td><td>

- 4a. If some requirements are marked Unscored Needs Review, the system lists them separately in the report.

</td></tr>
<tr><td>Special Requirements</td><td>

- The Risk Report must clearly highlight High-Risk requirements to guide mitigation efforts.

</td></tr>
</table>

Technology and Data Variation List	• Export formats: PDF, CSV.
Frequency of Occurrence	Medium (after risk assessment or before project reviews).
Open Issues	• Deciding how to balance quantitative scores vs. expert judgment in the final report.

2.3 Functional Requirements

This section describes the functional requirements of the proposed system in natural language form. The requirements are organized by individual modules, each representing a major functional component of the Reqify platform. These requirements define what the system must do to achieve its objectives effectively and accurately.

2.3.1 Module 1: Pattern-Based Requirement Extractor

1. FR1: The system shall allow users to upload requirement documents in multiple formats such as Word, PDF, and CSV.
2. FR2: The system shall extract structural elements of requirements from uploaded documents.
3. FR3: The system shall normalize the extracted elements through stopword removal and lemmatization.
4. FR4: The system shall generate structured output to be utilized by subsequent modules.

2.3.2 Module 2: Requirement Categorization

Following are the functional requirements for Module 2:

1. FR1: The system shall classify extracted requirements into two main types: functional and non-functional.
2. FR2: The system shall further organize non-functional requirements according to quality attributes such as performance, security, usability, reliability, and maintainability.

2.3.3 Module 3: Ambiguity and Completeness Detector

Following are the functional requirements for Module 3:

1. FR1: The system shall analyze requirement statements using contextual embeddings to understand their semantic meaning.
2. FR2: The system shall identify vague, ambiguous, or subjective expressions within the requirements.
3. FR3: The system shall assign an ambiguity score to each requirement to measure its clarity level.
4. FR4: The system shall map each requirement to predefined domain coverage categories to verify inclusion of all necessary system aspects.
5. FR5: The system shall perform binary classification of the system requirementa as Complete or Incomplete based on coverage evaluation.

2.3.4 Module 4: Conflict and Consistency Analyzer

Following are the functional requirements for Module 4:

1. FR1: The system shall compare requirement pairs to detect potential similarities or duplications.
2. FR2: The system shall identify conflicting or logically incompatible requirements through semantic comparison.
3. FR3: The system shall normalize constraints such as numeric ranges, units, and dates to ensure consistent analysis.
4. FR4: The system shall summarize all conflicts, duplications, and inconsistencies in a structured conflict matrix.

2.3.5 Module 5: Requirement Prioritization

Following are the functional requirements for Module 5:

1. FR1: The system shall allow initial prioritization of requirements based on predefined categories (e.g., Must Have, Should Have, Could Have).

2. FR2: The system shall utilize ambiguity and conflict scores to refine the prioritization process.

3. FR3: The system shall generate a final, AI-driven hybrid priority list for stakeholder decision-making.

2.3.6 Module 6: Change Impact Analyzer

Following are the functional requirements for Module 6:

1. FR1: The system shall represent requirements and their dependencies in the form of a graph structure.

2. FR2: The system shall identify both direct and indirect impacts resulting from any change to a requirement.

3. FR3: The system shall simulate scenarios for adding, modifying, or removing requirements to assess potential effects.

4. FR4: The system shall visualize the impact of requirement changes and classify their severity as low, medium, or high.

2.3.7 Module 7: Requirement Risk Estimator

Following are the functional requirements for Module 7:

1. FR1: The system shall define risk categories such as ambiguity, volatility, and dependency.

2. FR2: The system shall calculate a risk score for each requirement according to these defined categories.

3. FR3: The system shall combine all risk factors into a single overall risk index for each requirement.

4. FR4: The system shall assign risk levels (Low, Medium, High) for easier interpretation and analysis.

5. FR5: The system shall generate a comprehensive risk report highlighting the most critical and unstable requirements.

2.4 Non-Functional Requirements

This section specifies the non-functional or quality requirements of the proposed system. These requirements define the systems overall qualities such as reliability, usability, performance, and security. They ensure that Reqify not only meets its functional goals but also performs efficiently, remains secure, and provides a positive user experience.

2.4.1 Reliability

The system must provide reliable performance and minimize failures during execution. It should maintain consistent operation under normal and peak loads.

1. REL1: The system shall have a Mean Time Between Failures (MTBF) of at least 500 hours of continuous operation.
2. REL2: The system shall be capable of automatically detecting internal processing errors and logging them for further review.
3. REL3: In the event of a failure, the system shall preserve the latest processed data to prevent loss of extracted or analyzed requirements.
4. REL4: The system shall recover automatically from minor faults without requiring a restart.
5. REL5: The system shall ensure a success rate of at least 95% in executing extraction and analysis tasks without interruption.

2.4.2 Usability

The system shall provide an intuitive, user-friendly, and accessible interface suitable for technical and non-technical users.

1. USE1: The system shall allow new users to perform requirement extraction and analysis tasks within 10 minutes of first use without external assistance.
2. USE2: The system shall provide a clean and organized web interface for easy navigation between modules.
3. USE3: The system shall display error messages and guidance prompts in simple, understandable language to assist users in resolving issues.

4. USE4: The system shall maintain a consistent layout and color scheme across all interfaces for a cohesive user experience.

5. USE5: The system shall support accessibility standards to accommodate users with visual or cognitive impairments.

2.4.3 Performance

The system shall deliver efficient and responsive processing for all requirement analysis operations.

1. PER1: The system shall process a document of up to 2000 requirements within 10 seconds under normal conditions.

2. PER2: The system shall support at least 50 concurrent users without a noticeable drop in response time.

3. PER3: 90% of operations, including extraction and categorization, shall complete within 3 seconds of user initiation.

4. PER4: The system shall maintain response latency below 1 second for data retrieval and report generation.

5. PER5: The system shall ensure resource utilization remains within 80% of available system capacity during peak load.

2.4.4 Security

The system shall ensure data confidentiality, integrity, and controlled access to all user and project information.

1. SEC1: The system shall protect all uploaded documents and generated reports from unauthorized access.

2. SEC2: The system shall implement secure communication using HTTPS for all client-server interactions.

3. SEC3: The system shall authenticate users before allowing access to any project data or reports.

4. SEC4: The system shall log all user activities, including document uploads and analysis actions, for auditing purposes.

5. SEC5: The system shall automatically terminate inactive user sessions after 15 minutes of inactivity to prevent unauthorized access.

Chapter 3

System Overview

Give a general description of the functionality, context, and design of your project. Provide any background information if necessary.

3.1 Architectural Design

Develop a modular program structure and explain the relationships between the modules to achieve the complete functionality of the system. This is a high-level overview of how the systems modules collaborate with each other to achieve the desired functionality.

Dont go into too much detail about the individual subsystems. The main purpose is to gain a general understanding of how and why the system was decomposed, and how the individual parts work together.

Provide a diagram showing the major subsystems and their connections.

- In the initial design stage, create a Box and Line Diagram for a simpler representation of the systems.
- After finalizing the architecture style/pattern diagram (MVC, Client-Server, Layered, Multi-tiered), create a detailed mapping of modules/components to each part of the architecture.

To view examples of box and line diagrams and architecture styles, see Appendix C.

3.2 Design Models

Create design models as are applicable to your system. Provide detailed descriptions with each of the models that you add. Also ensure visibility of all diagrams.

Design Models for Object Oriented Development Approach

The applicable models for the project using object oriented development approach may include:

Activity Diagram
Class Diagram
Class-level Sequence Diagram
State Transition Diagram (for the projects which include event handling and backend processes)

Design Models for Procedural Approach

The applicable models for the project using procedural approach may include:

Activity Diagram
Data Flow Diagram (data flow diagram should be extended to 2-3 levels. It should clearly list all processes, their sources/sinks and data stores.)
System-level Sequence Diagram
State Transition Diagram (for the projects which include event handling and backend processes)

Examples of above diagrams are given in Appendix D.

3.3 Data Design

Explain how the information domain of your system is transformed into data structures. Describe how the major data or system entities are stored, processed, and organized.

List any databases or data storage items.

3.4 Domain Model

The domain model is a conceptual representation of the various entities, their attributes, and the relationships between them within the system. It serves as a bridge between the requirements and the design of the software, providing a clear understanding of the business domain and how it interacts with the system being developed.

A domain model typically includes key components such as:

- **Entities**: These are the primary objects within the domain that hold data and repre-

sent real-world concepts. Each entity is defined by its attributes and behaviors.

- **Attributes**: Characteristics or properties of the entities that define their state. For example, a "Customer" entity might have attributes such as `customerID`, `name`, `email`, and `address`.
- **Relationships**: These illustrate how different entities interact with each other. Relationships can be one-to-one, one-to-many, or many-to-many, depending on how entities are associated.
- **Associations**: Connections between entities that help clarify their interdependencies. For instance, a "Product" may be associated with a "Category," indicating the type of product it belongs to.

By constructing a domain model, developers can better understand the system's requirements, facilitate communication among stakeholders, and guide the subsequent design and implementation phases. It also aids in identifying potential issues early in the development process, ensuring a more robust architecture.

An example of a domain model for an online shopping system is provided in Appendix B.

Chapter 4

Implementation and Testing

4.1 Algorithm Design

This section describes the algorithmic approaches used to implement the major functionalities of Reqify.

4.1.1 Algorithm 1: Functional vs Non-Functional Classification

Algorithm 1 Binary Requirement Classification using RoBERTa

Require: Requirement text r, Pre-trained RoBERTa model M

Ensure: Classification label $\{Functional, Non\text{-}Functional\}$

1: $tokens \leftarrow$ tokenize(r) using RoBERTa tokenizer
2: $embeddings \leftarrow M.encode(tokens)$
3: $logits \leftarrow$ classifier_head($embeddings$)
4: $probabilities \leftarrow$ softmax($logits$)
5: $label \leftarrow \arg\max(probabilities)$
6: **return** $label$

This algorithm uses a fine-tuned RoBERTa transformer model to classify requirements into functional or non-functional categories based on contextual embeddings and semantic understanding of requirement text.

4.1.2 Algorithm 2: Non-Functional Requirement Quality Attribute Mapping

Algorithm 2 Multi-class NFR Quality Attribute Classification

Require: Non-functional requirement r_{nfr}, Trained classifier C

Ensure: Quality attribute $\{Performance, Security, Usability, Reliability, Maintainability, ...\}$

1: $tokens \leftarrow$ tokenize(r_{nfr})
2: $embeddings \leftarrow$ generate_embeddings($tokens$)
3: $features \leftarrow$ extract_features($embeddings$)
4: $logits \leftarrow C.predict(features)$
5: $probabilities \leftarrow$ softmax($logits$)
6: $quality_attribute \leftarrow \arg\max(probabilities)$
7: **return** $quality_attribute$

This algorithm maps non-functional requirements to specific quality attributes (performance, security, usability, reliability, maintainability, etc.) using a multi-class classification model trained on labeled NFR datasets.

4.1.3 Algorithm 3: Ambiguity Detection and Scoring

Algorithm 3 Binary Classification and Ambiguity Type Analysis

Require: Set of requirements $R = \{r_1, r_2, ..., r_n\}$, RoBERTa model M_{binary}, SetFit model M_{types}

Ensure: Comprehensive ambiguity report with labels, scores, and types

```
report ← ∅
for each requirement r_i in R do
    // Stage 1: Binary Classification
    tokens ← tokenize(r_i) using RoBERTa tokenizer
    embeddings ← M_binary.encode(tokens)
    logits ← classification_head(embeddings)
    label, confidence ← classify(logits)                    ▷ Ambiguous or Clear
    if label = "Ambiguous" then
        // Stage 2: Ambiguity Type Detection
        type_embeddings ← M_types.encode(r_i)
        type_scores ← detect_types(type_embeddings)
        detected_types ← {t : type_scores[t] > 0.5}
                        ▷ Types: Semantic, Lexical, Pragmatic, Syntactic, Referential
        // Stage 3: Aggregate Scoring
        ambiguity_score ← compute_aggregate_score(type_scores)
        severity ← classify_severity(ambiguity_score)
                                                            ▷ High, Medium, Low
        report[r_i] ← {label, confidence, detected_types, type_scores, ambiguity_score, severity}
    else
        report[r_i] ← {label : "Clear", confidence}
    end if
end for
return report
```

This algorithm employs a two-model approach for comprehensive ambiguity analysis. First, a fine-tuned RoBERTa model performs binary classification to identify ambiguous requirements with high confidence. For detected ambiguous requirements, a SetFit model analyzes five ambiguity types (Semantic, Lexical, Pragmatic, Syntactic, Referential) and assigns individual scores. The system aggregates these scores to produce an overall ambiguity score (0.0-1.0) and classifies severity levels (High, Medium, Low) for prioritized review.

4.1.4 Algorithm 4: Conflict and Consistency Analysis

Algorithm 4 Semantic Conflict Detection using Sentence Transformers

Require: Set of requirements $R = \{r_1, r_2, ..., r_n\}$, Sentence Transformer model M, Similarity threshold θ

Ensure: Conflict report with similarity scores and identified conflicts

```
embeddings ← []
// Generate embeddings for all requirements
for each r_i in R do
    emb_i ← M.encode(r_i)
    embeddings.append(emb_i)
end for
conflict_matrix ← ∅
// Pairwise similarity comparison
for i ← 0 to n − 1 do
    for j ← i + 1 to n do
        similarity ← cosine_similarity(embeddings[i], embeddings[j])
        if similarity > θ then                    ▷ Potential conflict or duplicate
            conflict_type ← classify_conflict(r_i, r_j, similarity)
            conflict_matrix[(r_i, r_j)] ← {similarity, conflict_type}
        end if
    end for
end for
// Generate comprehensive report
report ← format_conflict_report(conflict_matrix)
return report
```

This algorithm utilizes Sentence Transformers to generate dense vector embeddings for each requirement. It performs pairwise comparison using cosine similarity to detect semantically similar requirements that may represent duplicates or contradictions. Requirements exceeding the similarity threshold are flagged and classified, with results compiled into a structured conflict matrix for stakeholder review.

4.1.5 Algorithm 5: Requirements Completeness Analyzer

Algorithm 5 Multi-Strategy NLP-based Completeness Analysis

Require: Requirements text R, Domain D, NLP model M, Knowledge base KB

Ensure: Coverage percentage and missing modules

```
doc ← M.PROCESS(R)
lemmas ← EXTRACTLEMMAS(doc)
modules ← KB.GETMODULES(D)
covered ← ∅, missing ← ∅
for each module in modules do
    if HASNEGATIVECONTEXT(doc, module) then
        missing.ADD(module)                          ▷ Exclude if not required
        continue
    end if
                                  ▷ Calculate weighted score from 5 strategies
    s1 ← MATCHRATIO(module.concepts, lemmas)
    s2 ← MATCHRATIO(module.actions, lemmas)
    s3 ← MATCHRATIO(module.tech_terms, R)
    s4 ← COSINESIM(doc.vector, module.vector)
    s5 ← PHRASEMATCH(doc, module)
    score ← 0.30s1 + 0.20s2 + 0.15s3 + 0.20s4 + 0.15s5
    if score ≥ 0.25 or (s1 > 0 and s2 > 0) then
        covered.ADD(module)
    else
        missing.ADD(module)
    end if
end for
coverage ← (|covered|/|modules|) × 100
return {coverage, missing}
```

This algorithm detects module completeness using multi-strategy NLP: concept matching, action verbs, technical terms, semantic similarity (word embeddings), and phrase patterns. Negative context filtering prevents false positives. Adaptive thresholds ensure robustness across IEEE, technical, and business formats.

4.2 External APIs/SDKs

The following table describes third-party libraries and frameworks used in the Reqify implementation:

Table 4.1: External Libraries and Frameworks Used in Reqify

Library and Version	Description	Purpose of Usage	Functions/Classes Used
Hugging Face Transformers (v4.35.0)	Pre-trained NLP models library	Requirement classification and ambiguity detection	AutoModel.from_pretrained(), AutoTokenizer
SetFit	Few-shot learning framework	Ambiguity type classification	SetFitModel.from_pretrained()
spaCy (v3.7.2)	NLP processing library	Requirement extraction and preprocessing	spacy.load('en_core_web_sm')
Sentence Transformers	Semantic embedding library	Conflict detection and similarity analysis	SentenceTransformer.encode()

4.3 Testing Details

Once the system has been successfully developed, Testing is performed to verify system functionality, and ensure compliance with functional requirements.

4.3.1 Unit Testing

Unit tests validate individual modules and API endpoints in isolation, ensuring that each component functions correctly before integration into the complete system pipeline.

Each module of the system was tested independently using black-box unit testing, where test cases were designed based solely on inputs and expected outputs without any knowledge of the internal implementation. For every module, test cases were executed by sending POST requests to each module's dedicated API endpoint with requirement text as input and verifying that the returned JSON response matched the expected result. This approach ensures that each module behaves correctly from the user's perspective and meets its functional specification in isolation before system-level integration.

4.3.1.1 Test screenshots

4.3.1.2 Test cases

Table 4.2: Unit Testing Results for User Authentication

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/ Fail
TC001	Verify user registration with valid data	Backend server running	Send POST to /api/ register	Username: user_236fmh Email: test_236fmh@ example.com Password: password123	201 Created	User added to DB	201 Created	Pass
TC002	Verify login with valid credentials	User registered	Send POST to /api/ login	Email: test_236fmh@ example.com Password: password123	200 OK + Token	JWT token received	200 OK	Pass
TC003	Verify login with invalid password	User registered	Send POST to /api/ login	Email: test_236fmh@ example.com Password: wrongpassword	400 Bad Request	Error message	400 Bad Request	Pass
TC004	Verify registration with duplicate email	User registered	Send POST to /api/ register	Username: user_236fmh Email: test_236fmh@ example.com Password: password123	400 Bad Request	Error message	400 Bad Request	Pass

Table 4.3: Unit Testing Results for Requirement Classification (FR/NFR)

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/ Fail
TC001	Verify FR classification of REQ-1	API server running	Send POST to /predict/fr-nfr	"The system shall allow registered users to browse product catalog by category."	Prediction: FR	JSON response received	As Expected	Pass
TC002	Verify FR classification of REQ-2	API server running	Send POST to /predict/fr-nfr	"The system shall allow registered users to add products to shopping cart with quantity selection."	Prediction: FR	JSON response received	As Expected	Pass
TC003	Verify FR classification of REQ-3	API server running	Send POST to /predict/fr-nfr	"The system shall allow registered users to remove items from shopping cart at any time."	Prediction: FR	JSON response received	As Expected	Pass
TC004	Verify FR classification of REQ-4	API server running	Send POST to /predict/fr-nfr	"The system shall allow registered users to place orders using saved payment methods with order confirmation."	Prediction: FR	JSON response received	As Expected	Pass
TC005	Verify FR classification of REQ-5	API server running	Send POST to /predict/fr-nfr	"The system shall allow registered users to track order status in real time after purchase."	Prediction: FR	JSON response received	As Expected	Pass
TC006	Verify FR classification of REQ-6	API server running	Send POST to /predict/fr-nfr	"The system shall allow registered users to submit product reviews after order delivery."	Prediction: FR	JSON response received	As Expected	Pass

Continued on next page

Table 4.3 Continued from previous page

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/ Fail
TC007	Verify FR classification of REQ-7	API server running	Send POST to /predict/fr-nfr	"The system shall allow registered users to add products to wishlist for later purchase."	Prediction: FR	JSON response received	As Expected	Pass
TC008	Verify FR classification of REQ-8	API server running	Send POST to /predict/fr-nfr	"The system shall allow registered users to apply discount coupons during checkout with validity check."	Prediction: FR	JSON response received	As Expected	Pass
TC009	Verify FR classification of REQ-9	API server running	Send POST to /predict/fr-nfr	"The system shall allow admin users to manage product inventory by updating stock levels."	Prediction: FR	JSON response received	As Expected	Pass
TC010	Verify FR classification of REQ-10	API server running	Send POST to /predict/fr-nfr	"The system shall allow admin users to process refund requests within 3 business days."	Prediction: FR	JSON response received	As Expected	Pass
TC011	Verify FR classification of REQ-11	API server running	Send POST to /predict/fr-nfr	"The system shall allow registered users to search products by keyword with filter support."	Prediction: FR	JSON response received	As Expected	Pass
TC012	Verify FR classification of REQ-12	API server running	Send POST to /predict/fr-nfr	"The system shall allow registered users to view order history from their account dashboard."	Prediction: FR	JSON response received	As Expected	Pass
TC013	Verify FR classification of REQ-13	API server running	Send POST to /predict/fr-nfr	"The system shall allow admin users to generate sales reports by date range."	Prediction: FR	JSON response received	As Expected	Pass
TC014	Verify FR classification of REQ-14	API server running	Send POST to /predict/fr-nfr	"The system shall allow registered users to manage their shipping addresses in account settings."	Prediction: FR	JSON response received	As Expected	Pass
TC015	Verify FR classification of REQ-15	API server running	Send POST to /predict/fr-nfr	"The system shall allow registered users to complete payment using credit card, debit card, or digital wallet."	Prediction: FR	JSON response received	As Expected	Pass
TC016	Verify NFR classification of REQ-16	API server running	Send POST to /predict/fr-nfr	"The system shall respond to product search queries within 2 seconds under normal load."	Prediction: NFR	JSON response received	As Expected	Pass
TC017	Verify NFR classification of REQ-17	API server running	Send POST to /predict/fr-nfr	"The system shall support at least 500 concurrent users without performance degradation."	Prediction: NFR	JSON response received	As Expected	Pass
TC018	Verify NFR classification of REQ-18	API server running	Send POST to /predict/fr-nfr	"The system shall encrypt all payment and personal data using AES-256 encryption."	Prediction: NFR	JSON response received	As Expected	Pass

Continued on next page

Table 4.3 Continued from previous page

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/ Fail
TC019	Verify FR classification of REQ-19	API server running	Send POST to /predict/fr-nfr	"The system shall authenticate all users using JWT tokens with 24-hour expiration."	Prediction: FR	JSON response received	As Expected	Pass
TC020	Verify NFR classification of REQ-20	API server running	Send POST to /predict/fr-nfr	"The system shall be available 99.9% of the time on a monthly basis."	Prediction: NFR	JSON response received	As Expected	Pass
TC021	Verify NFR classification of REQ-21	API server running	Send POST to /predict/fr-nfr	"The system shall recover from any server failure within 5 minutes without data loss."	Prediction: NFR	JSON response received	As Expected	Pass
TC022	Verify NFR classification of REQ-22	API server running	Send POST to /predict/fr-nfr	"The system shall provide a responsive web interface accessible on desktop and mobile browsers."	Prediction: NFR	JSON response received	As Expected	Pass
TC023	Verify NFR classification of REQ-23	API server running	Send POST to /predict/fr-nfr	"The system shall allow new users to complete their first purchase within 10 minutes without external help."	Prediction: NFR	JSON response received	As Expected	Pass
TC024	Verify NFR classification of REQ-24	API server running	Send POST to /predict/fr-nfr	"The system shall run on Chrome, Firefox, Safari, and Edge browsers without functional differences."	Prediction: NFR	JSON response received	As Expected	Pass
TC025	Verify NFR classification of REQ-25	API server running	Send POST to /predict/fr-nfr	"The system shall integrate with third-party payment gateways including Stripe and PayPal."	Prediction: NFR	JSON response received	As Expected	Pass
TC026	Verify NFR classification of REQ-26	API server running	Send POST to /predict/fr-nfr	"The system shall be deployable on AWS, Azure, and Google Cloud without code changes."	Prediction: NFR	JSON response received	As Expected	Pass
TC027	Verify NFR classification of REQ-27	API server running	Send POST to /predict/fr-nfr	"The system shall follow modular architecture to allow addition of new modules without affecting existing ones."	Prediction: NFR	JSON response received	As Expected	Pass
TC028	Verify NFR classification of REQ-28	API server running	Send POST to /predict/fr-nfr	"The system shall log all user actions including login, purchase, and profile updates for audit purposes."	Prediction: NFR	JSON response received	As Expected	Pass
TC029	Verify FR classification of REQ-29	API server running	Send POST to /predict/fr-nfr	"The system shall process orders quickly and efficiently."	Prediction: FR	JSON response received	As Expected	Pass
TC030	Verify NFR classification of REQ-30	API server running	Send POST to /predict/fr-nfr	"The system should handle many users simultaneously without issues."	Prediction: NFR	JSON response received	As Expected	Pass

Continued on next page

Table 4.3 Continued from previous page

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/ Fail
TC031	Verify NFR classification of REQ-31	API server running	Send POST to /predict/fr-nfr	"The system shall provide a user-friendly and intuitive shopping experience."	Prediction: NFR	JSON response received	As Expected	Pass
TC032	Verify NFR classification of REQ-32	API server running	Send POST to /predict/fr-nfr	"The system shall ensure products are delivered in a reasonable time."	Prediction: NFR	JSON response received	As Expected	Pass
TC033	Verify NFR classification of REQ-33	API server running	Send POST to /predict/fr-nfr	"The system shall bank all transaction data securely in the backend."	Prediction: NFR	JSON response received	As Expected	Pass
TC034	Verify FR classification of REQ-34	API server running	Send POST to /predict/fr-nfr	"The system shall allow users register using email and password."	Prediction: FR	JSON response received	As Expected	Pass
TC035	Verify NFR classification of REQ-35	API server running	Send POST to /predict/fr-nfr	"The system should support registration by using email and password."	Prediction: NFR	JSON response received	As Expected	Pass
TC036	Verify FR classification of REQ-36	API server running	Send POST to /predict/fr-nfr	"The registered user shall be able to view past orders from their profile page."	Prediction: FR	JSON response received	As Expected	Pass
TC037	Verify FR classification of REQ-37	API server running	Send POST to /predict/fr-nfr	"The system shall allow guest users to checkout without creating an account."	Prediction: FR	JSON response received	As Expected	Pass
TC038	Verify NFR classification of REQ-38	API server running	Send POST to /predict/fr-nfr	"The system shall require all users to register and log in before accessing checkout."	Prediction: NFR	JSON response received	As Expected	Pass
TC039	Verify FR classification of REQ-39	API server running	Send POST to /predict/fr-nfr	"The system shall process all refund requests within 24 hours of submission."	Prediction: FR	JSON response received	As Expected	Pass
TC040	Verify NFR classification of REQ-40	API server running	Send POST to /predict/fr-nfr	"The system shall process refund requests within 7 to 10 business days after approval."	Prediction: NFR	JSON response received	As Expected	Pass
TC041	Verify NFR classification of REQ-41	API server running	Send POST to /predict/fr-nfr	"The system shall support a maximum of 200 concurrent users at peak load."	Prediction: NFR	JSON response received	As Expected	Pass
TC042	Verify FR classification of REQ-42	API server running	Send POST to /predict/fr-nfr	"The system shall display product prices in USD only across all pages."	Prediction: FR	JSON response received	As Expected	Pass
TC043	Verify FR classification of REQ-43	API server running	Send POST to /predict/fr-nfr	"The system shall allow users to view product prices in their local currency based on location."	Prediction: FR	JSON response received	As Expected	Pass
TC044	Verify FR classification of REQ-44	API server running	Send POST to /predict/fr-nfr	“The system shall allow registered users to browse nearby restaurants by location and cuisine type.”	Prediction: FR	JSON response received	As Expected	Pass

Continued on next page

Table 4.3 Continued from previous page

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/ Fail
TC045	Verify FR classification of REQ-45	API server running	Send POST to /predict/fr-nfr	"The system shall allow registered users to view restaurant menus with item descriptions and prices."	Prediction: FR	JSON response received	As Expected	Pass
TC046	Verify FR classification of REQ-46	API server running	Send POST to /predict/fr-nfr	"The system shall allow registered users to add food items to cart with quantity and customization options."	Prediction: FR	JSON response received	As Expected	Pass
TC047	Verify FR classification of REQ-47	API server running	Send POST to /predict/fr-nfr	"The system shall allow registered users to remove items from cart before checkout."	Prediction: FR	JSON response received	As Expected	Pass
TC048	Verify FR classification of REQ-48	API server running	Send POST to /predict/fr-nfr	"The system shall allow registered users to place food orders using saved payment methods with order confirmation."	Prediction: FR	JSON response received	As Expected	Pass
TC049	Verify FR classification of REQ-49	API server running	Send POST to /predict/fr-nfr	"The system shall allow registered users to track delivery status in real time after order placement."	Prediction: FR	JSON response received	As Expected	Pass
TC050	Verify FR classification of REQ-50	API server running	Send POST to /predict/fr-nfr	"The system shall allow registered users to rate and review restaurants after order delivery."	Prediction: FR	JSON response received	As Expected	Pass
TC051	Verify FR classification of REQ-51	API server running	Send POST to /predict/fr-nfr	"The system shall allow registered users to apply promo codes during checkout with validity check."	Prediction: FR	JSON response received	As Expected	Pass
TC052	Verify FR classification of REQ-52	API server running	Send POST to /predict/fr-nfr	"The system shall allow registered users to schedule orders for a future delivery time with confirmation."	Prediction: FR	JSON response received	As Expected	Pass
TC053	Verify FR classification of REQ-53	API server running	Send POST to /predict/fr-nfr	"The system shall allow registered users to view order history from their account dashboard."	Prediction: FR	JSON response received	As Expected	Pass
TC054	Verify FR classification of REQ-54	API server running	Send POST to /predict/fr-nfr	"The system shall allow registered users to manage saved delivery addresses in account settings."	Prediction: FR	JSON response received	As Expected	Pass
TC055	Verify FR classification of REQ-55	API server running	Send POST to /predict/fr-nfr	"The system shall allow registered users to reorder from previous orders with a single action."	Prediction: FR	JSON response received	As Expected	Pass

Continued on next page

Table 4.3 Continued from previous page

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/ Fail
TC056	Verify FR classification of REQ-56	API server running	Send POST to /predict/fr-nfr	"The system shall allow restaurant owners to manage their menu by adding, updating, or removing items."	Prediction: FR	JSON response received	As Expected	Pass
TC057	Verify FR classification of REQ-57	API server running	Send POST to /predict/fr-nfr	"The system shall allow restaurant owners to accept or reject incoming orders within 2 minutes."	Prediction: FR	JSON response received	As Expected	Pass
TC058	Verify FR classification of REQ-58	API server running	Send POST to /predict/fr-nfr	"The system shall allow delivery agents to update delivery status at each checkpoint in real time."	Prediction: FR	JSON response received	As Expected	Pass
TC059	Verify FR classification of REQ-59	API server running	Send POST to /predict/fr-nfr	"The system shall allow admin users to manage restaurant listings and verify partner restaurants."	Prediction: FR	JSON response received	As Expected	Pass
TC060	Verify FR classification of REQ-60	API server running	Send POST to /predict/fr-nfr	"The system shall allow registered users to complete payment using credit card, debit card, cash on delivery, or digital wallet."	Prediction: FR	JSON response received	As Expected	Pass
TC061	Verify FR classification of REQ-61	API server running	Send POST to /predict/fr-nfr	"The system shall allow registered users to contact customer support through in-app chat after order placement."	Prediction: FR	JSON response received	As Expected	Pass
TC062	Verify NFR classification of REQ-62	API server running	Send POST to /predict/fr-nfr	"The system shall load restaurant listings within 3 seconds under normal network conditions."	Prediction: NFR	JSON response received	As Expected	Pass
TC063	Verify NFR classification of REQ-63	API server running	Send POST to /predict/fr-nfr	"The system shall support at least 1000 concurrent users without performance degradation."	Prediction: NFR	JSON response received	As Expected	Pass
TC064	Verify NFR classification of REQ-64	API server running	Send POST to /predict/fr-nfr	"The system shall encrypt all payment and personal data using AES-256 encryption."	Prediction: NFR	JSON response received	As Expected	Pass
TC065	Verify FR classification of REQ-65	API server running	Send POST to /predict/fr-nfr	"The system shall authenticate all users using JWT tokens with 24-hour expiration."	Prediction: FR	JSON response received	As Expected	Pass
TC066	Verify NFR classification of REQ-66	API server running	Send POST to /predict/fr-nfr	"The system shall log all user actions including login, order placement, and payment for audit purposes."	Prediction: NFR	JSON response received	As Expected	Pass
TC067	Verify NFR classification of REQ-67	API server running	Send POST to /predict/fr-nfr	"The system shall be available 99.9% of the time on a monthly basis."	Prediction: NFR	JSON response received	As Expected	Pass

Continued on next page

Table 4.3 Continued from previous page

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/ Fail
TC068	Verify NFR classification of REQ-68	API server running	Send POST to /predict/fr-nfr	"The system shall recover from any server failure within 5 minutes without data loss."	Prediction: NFR	JSON response received	As Expected	Pass
TC069	Verify NFR classification of REQ-69	API server running	Send POST to /predict/fr-nfr	"The system shall provide a responsive mobile-first interface accessible on Android and iOS."	Prediction: NFR	JSON response received	As Expected	Pass
TC070	Verify NFR classification of REQ-70	API server running	Send POST to /predict/fr-nfr	"The system shall allow new users to place their first order within 5 minutes without external help."	Prediction: NFR	JSON response received	As Expected	Pass
TC071	Verify NFR classification of REQ-71	API server running	Send POST to /predict/fr-nfr	"The system shall run on Chrome, Firefox, Safari, and Edge browsers without functional differences."	Prediction: NFR	JSON response received	As Expected	Pass
TC072	Verify FR classification of REQ-72	API server running	Send POST to /predict/fr-nfr	"The system shall integrate with third-party mapping services including Google Maps for delivery tracking."	Prediction: FR	JSON response received	As Expected	Pass
TC073	Verify NFR classification of REQ-73	API server running	Send POST to /predict/fr-nfr	"The system shall be deployable on AWS, Azure, and Google Cloud without code changes."	Prediction: NFR	JSON response received	As Expected	Pass
TC074	Verify NFR classification of REQ-74	API server running	Send POST to /predict/fr-nfr	"The system shall follow modular architecture to allow addition of new features without affecting existing modules."	Prediction: NFR	JSON response received	As Expected	Pass
TC075	Verify NFR classification of REQ-75	API server running	Send POST to /predict/fr-nfr	"The system shall provide API documentation for all endpoints to support future integrations."	Prediction: NFR	JSON response received	As Expected	Pass
TC076	Verify FR classification of REQ-76	API server running	Send POST to /predict/fr-nfr	"The system shall deliver food orders as fast as possible to nearby customers."	Prediction: FR	JSON response received	As Expected	Pass
TC077	Verify NFR classification of REQ-77	API server running	Send POST to /predict/fr-nfr	"The system should handle a large number of simultaneous orders without slowing down."	Prediction: NFR	JSON response received	As Expected	Pass
TC078	Verify NFR classification of REQ-78	API server running	Send POST to /predict/fr-nfr	"The system shall provide a smooth and pleasant ordering experience for all users."	Prediction: NFR	JSON response received	As Expected	Pass
TC079	Verify NFR classification of REQ-79	API server running	Send POST to /predict/fr-nfr	"The system shall ensure delivery agents reach customers in a reasonable amount of time."	Prediction: NFR	JSON response received	As Expected	Pass

Continued on next page

Table 4.3 Continued from previous page

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/ Fail
TC080	Verify NFR classification of REQ-80	API server running	Send POST to /predict/fr-nfr	"The system shall process payments in a secure and efficient manner at all times."	Prediction: NFR	JSON response received	As Expected	Pass
TC081	Verify FR classification of REQ-81	API server running	Send POST to /predict/fr-nfr	"The system shall allow customers to add meals to their basket before placing an order."	Prediction: FR	JSON response received	As Expected	Pass
TC082	Verify FR classification of REQ-82	API server running	Send POST to /predict/fr-nfr	"The system shall allow users to put food items into cart prior to checkout."	Prediction: FR	JSON response received	As Expected	Pass
TC083	Verify FR classification of REQ-83	API server running	Send POST to /predict/fr-nfr	"The system shall allow registered users to view their past food orders from their profile."	Prediction: FR	JSON response received	As Expected	Pass
TC084	Verify FR classification of REQ-84	API server running	Send POST to /predict/fr-nfr	"The system shall allow users to access previous order history from the account section."	Prediction: FR	JSON response received	As Expected	Pass
TC085	Verify FR classification of REQ-85	API server running	Send POST to /predict/fr-nfr	"The system shall allow guest users to place food orders without creating an account."	Prediction: FR	JSON response received	As Expected	Pass
TC086	Verify FR classification of REQ-86	API server running	Send POST to /predict/fr-nfr	"The system shall require all users to register and log in before accessing the ordering feature."	Prediction: FR	JSON response received	As Expected	Pass
TC087	Verify FR classification of REQ-87	API server running	Send POST to /predict/fr-nfr	"The system shall automatically assign the nearest available delivery agent to every new order."	Prediction: FR	JSON response received	As Expected	Pass
TC088	Verify FR classification of REQ-88	API server running	Send POST to /predict/fr-nfr	"The system shall allow restaurant owners to choose and assign their own preferred delivery agents."	Prediction: FR	JSON response received	As Expected	Pass
TC089	Verify NFR classification of REQ-89	API server running	Send POST to /predict/fr-nfr	"The system shall support a maximum of 300 concurrent users at peak load."	Prediction: NFR	JSON response received	As Expected	Pass
TC090	Verify FR classification of REQ-90	API server running	Send POST to /predict/fr-nfr	"The system shall display all menu prices in USD only across all restaurant pages."	Prediction: FR	JSON response received	As Expected	Pass
TC091	Verify FR classification of REQ-91	API server running	Send POST to /predict/fr-nfr	"The system shall display menu prices in the local currency of the user based on their detected location."	Prediction: FR	JSON response received	As Expected	Pass
TC092	Verify FR classification of REQ-92	API server running	Send POST to /predict/fr-nfr	"The system shall cancel unaccepted orders automatically after 5 minutes of no restaurant response."	Prediction: FR	JSON response received	As Expected	Pass

Continued on next page

Table 4.3 Continued from previous page

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/ Fail
TC093	Verify FR classification of REQ-93	API server running	Send POST to /predict/fr-nfr	"The system shall wait up to 15 minutes before cancelling orders that have not been accepted by the restaurant."	Prediction: FR	JSON response received	As Expected	Pass
TC094	Verify FR classification of REQ-94	API server running	Send POST to /predict/fr-nfr	"The system shall allow admin users to create and manage courses with title, description, and category."	Prediction: FR	JSON response received	As Expected	Pass
TC095	Verify FR classification of REQ-95	API server running	Send POST to /predict/fr-nfr	"The system shall allow instructors to upload course content including videos, PDFs, and presentations."	Prediction: FR	JSON response received	As Expected	Pass
TC096	Verify FR classification of REQ-96	API server running	Send POST to /predict/fr-nfr	"The system shall allow registered students to browse and search available courses by category and keyword."	Prediction: FR	JSON response received	As Expected	Pass
TC097	Verify FR classification of REQ-97	API server running	Send POST to /predict/fr-nfr	"The system shall allow registered students to enroll in courses after completing payment or free registration."	Prediction: FR	JSON response received	As Expected	Pass
TC098	Verify FR classification of REQ-98	API server running	Send POST to /predict/fr-nfr	"The system shall allow registered students to access course content after successful enrollment."	Prediction: FR	JSON response received	As Expected	Pass
TC099	Verify FR classification of REQ-99	API server running	Send POST to /predict/fr-nfr	"The system shall allow registered students to track their course progress with completion percentage."	Prediction: FR	JSON response received	As Expected	Pass
TC100	Verify FR classification of REQ-100	API server running	Send POST to /predict/fr-nfr	"The system shall allow instructors to create quizzes and assignments with configurable deadlines."	Prediction: FR	JSON response received	As Expected	Pass
TC101	Verify FR classification of REQ-101	API server running	Send POST to /predict/fr-nfr	"The system shall allow registered students to attempt quizzes within the allowed time limit and submission window."	Prediction: FR	JSON response received	As Expected	Pass
TC102	Verify FR classification of REQ-102	API server running	Send POST to /predict/fr-nfr	"The system shall allow instructors to grade student assignments and provide written feedback."	Prediction: FR	JSON response received	As Expected	Pass
TC103	Verify FR classification of REQ-103	API server running	Send POST to /predict/fr-nfr	"The system shall allow registered students to view their grades and feedback from the course dashboard."	Prediction: FR	JSON response received	As Expected	Pass

Continued on next page

Table 4.3 Continued from previous page

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/ Fail
TC104	Verify FR classification of REQ-104	API server running	Send POST to /predict/fr-nfr	"The system shall allow registered students to participate in course discussion forums by posting and replying."	Prediction: FR	JSON response received	As Expected	Pass
TC105	Verify FR classification of REQ-105	API server running	Send POST to /predict/fr-nfr	"The system shall allow instructors to schedule and host live classes with registered enrolled students."	Prediction: FR	JSON response received	As Expected	Pass
TC106	Verify FR classification of REQ-106	API server running	Send POST to /predict/fr-nfr	"The system shall allow registered students to download course completion certificates after finishing all requirements."	Prediction: FR	JSON response received	As Expected	Pass
TC107	Verify FR classification of REQ-107	API server running	Send POST to /predict/fr-nfr	"The system shall allow admin users to manage user accounts including students and instructors."	Prediction: FR	JSON response received	As Expected	Pass
TC108	Verify FR classification of REQ-108	API server running	Send POST to /predict/fr-nfr	"The system shall allow registered students to submit assignments before the deadline with file upload support."	Prediction: FR	JSON response received	As Expected	Pass
TC109	Verify FR classification of REQ-109	API server running	Send POST to /predict/fr-nfr	"The system shall allow instructors to send announcements to all enrolled students of a course."	Prediction: FR	JSON response received	As Expected	Pass
TC110	Verify FR classification of REQ-110	API server running	Send POST to /predict/fr-nfr	"The system shall allow registered students to save courses to a wishlist for later enrollment."	Prediction: FR	JSON response received	As Expected	Pass
TC111	Verify FR classification of REQ-111	API server running	Send POST to /predict/fr-nfr	"The system shall allow admin users to generate reports on course enrollment and student performance."	Prediction: FR	JSON response received	As Expected	Pass
TC112	Verify NFR classification of REQ-112	API server running	Send POST to /predict/fr-nfr	"The system shall load course content pages within 3 seconds under normal network conditions."	Prediction: NFR	JSON response received	As Expected	Pass
TC113	Verify NFR classification of REQ-113	API server running	Send POST to /predict/fr-nfr	"The system shall support at least 2000 concurrent users without performance degradation."	Prediction: NFR	JSON response received	As Expected	Pass
TC114	Verify NFR classification of REQ-114	API server running	Send POST to /predict/fr-nfr	"The system shall encrypt all user data and payment information using AES-256 encryption."	Prediction: NFR	JSON response received	As Expected	Pass

Continued on next page

Table 4.3 Continued from previous page

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/ Fail
TC115	Verify FR classification of REQ-115	API server running	Send POST to /predict/fr-nfr	"The system shall authenticate all users using JWT tokens with 24-hour expiration."	Prediction: FR	JSON response received	As Expected	Pass
TC116	Verify NFR classification of REQ-116	API server running	Send POST to /predict/fr-nfr	"The system shall log all user actions including login, enrollment, and submission for audit purposes."	Prediction: NFR	JSON response received	As Expected	Pass
TC117	Verify NFR classification of REQ-117	API server running	Send POST to /predict/fr-nfr	"The system shall be available 99.9% of the time on a monthly basis."	Prediction: NFR	JSON response received	As Expected	Pass
TC118	Verify NFR classification of REQ-118	API server running	Send POST to /predict/fr-nfr	"The system shall recover from any server failure within 5 minutes without loss of student progress data."	Prediction: NFR	JSON response received	As Expected	Pass
TC119	Verify NFR classification of REQ-119	API server running	Send POST to /predict/fr-nfr	"The system shall provide a responsive interface accessible on desktop, tablet, and mobile browsers."	Prediction: NFR	JSON response received	As Expected	Pass
TC120	Verify NFR classification of REQ-120	API server running	Send POST to /predict/fr-nfr	"The system shall allow new students to enroll in their first course within 5 minutes without external help."	Prediction: NFR	JSON response received	As Expected	Pass
TC121	Verify NFR classification of REQ-121	API server running	Send POST to /predict/fr-nfr	"The system shall run on Chrome, Firefox, Safari, and Edge browsers without functional differences."	Prediction: NFR	JSON response received	As Expected	Pass
TC122	Verify NFR classification of REQ-122	API server running	Send POST to /predict/fr-nfr	"The system shall integrate with third-party video streaming services including YouTube and Vimeo."	Prediction: NFR	JSON response received	As Expected	Pass
TC123	Verify NFR classification of REQ-123	API server running	Send POST to /predict/fr-nfr	"The system shall be deployable on AWS, Azure, and Google Cloud without code changes."	Prediction: NFR	JSON response received	As Expected	Pass
TC124	Verify NFR classification of REQ-124	API server running	Send POST to /predict/fr-nfr	"The system shall follow modular architecture to allow addition of new modules without affecting existing features."	Prediction: NFR	JSON response received	As Expected	Pass
TC125	Verify NFR classification of REQ-125	API server running	Send POST to /predict/fr-nfr	"The system shall provide API documentation for all endpoints to support future third-party integrations."	Prediction: NFR	JSON response received	As Expected	Pass

Continued on next page

Table 4.3 Continued from previous page

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/ Fail
TC126	Verify NFR classification of REQ-126	API server running	Send POST to /predict/fr-nfr	"The system shall deliver course content quickly and smoothly to all students."	Prediction: NFR	JSON response received	As Expected	Pass
TC127	Verify NFR classification of REQ-127	API server running	Send POST to /predict/fr-nfr	"The system should handle a large number of students accessing courses at the same time without issues."	Prediction: NFR	JSON response received	As Expected	Pass
TC128	Verify NFR classification of REQ-128	API server running	Send POST to /predict/fr-nfr	"The system shall provide an engaging and interactive learning experience for all users."	Prediction: NFR	JSON response received	As Expected	Pass
TC129	Verify FR classification of REQ-129	API server running	Send POST to /predict/fr-nfr	"The system shall ensure instructors can easily manage their course materials without difficulty."	Prediction: FR	JSON response received	As Expected	Pass
TC130	Verify FR classification of REQ-130	API server running	Send POST to /predict/fr-nfr	"The system shall process student payments in a secure and timely manner during enrollment."	Prediction: FR	JSON response received	As Expected	Pass
TC131	Verify FR classification of REQ-131	API server running	Send POST to /predict/fr-nfr	"The system shall allow learners to sign up for available courses after making payment."	Prediction: FR	JSON response received	As Expected	Pass
TC132	Verify FR classification of REQ-132	API server running	Send POST to /predict/fr-nfr	"The system shall allow students to register for a course upon successful fee submission."	Prediction: FR	JSON response received	As Expected	Pass
TC133	Verify FR classification of REQ-133	API server running	Send POST to /predict/fr-nfr	"The system shall allow registered students to view their assessment scores from the learning portal."	Prediction: FR	JSON response received	As Expected	Pass
TC134	Verify FR classification of REQ-134	API server running	Send POST to /predict/fr-nfr	"The system shall allow students to access their quiz and assignment results from the course page."	Prediction: FR	JSON response received	As Expected	Pass
TC135	Verify FR classification of REQ-135	API server running	Send POST to /predict/fr-nfr	"The system shall allow guest users to access and view full course content without registration."	Prediction: FR	JSON response received	As Expected	Pass
TC136	Verify NFR classification of REQ-136	API server running	Send POST to /predict/fr-nfr	"The system shall require all users to register and log in before accessing any course content."	Prediction: NFR	JSON response received	As Expected	Pass
TC137	Verify FR classification of REQ-137	API server running	Send POST to /predict/fr-nfr	"The system shall automatically pass all students who complete 100 percent of course videos."	Prediction: FR	JSON response received	As Expected	Pass

Continued on next page

Table 4.3 Continued from previous page

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/ Fail
TC138	Verify FR classification of REQ-138	API server running	Send POST to /predict/fr-nfr	“The system shall require students to pass all quizzes and assignments before receiving course completion certification.”	Prediction: FR	JSON response received	As Expected	Pass
TC139	Verify NFR classification of REQ-139	API server running	Send POST to /predict/fr-nfr	“The system shall support a maximum of 500 concurrent users at peak load.”	Prediction: NFR	JSON response received	As Expected	Pass
TC140	Verify FR classification of REQ-140	API server running	Send POST to /predict/fr-nfr	“The system shall display all course prices in USD only across all pages.”	Prediction: FR	JSON response received	As Expected	Pass
TC141	Verify FR classification of REQ-141	API server running	Send POST to /predict/fr-nfr	“The system shall display course prices in the local currency of the student based on their detected location.”	Prediction: FR	JSON response received	As Expected	Pass
TC142	Verify FR classification of REQ-142	API server running	Send POST to /predict/fr-nfr	“The system shall automatically issue course completion certificates immediately after the final quiz submission.”	Prediction: FR	JSON response received	As Expected	Pass
TC143	Verify FR classification of REQ-143	API server running	Send POST to /predict/fr-nfr	“The system shall issue course completion certificates only after instructor manual approval of all submitted assignments.”	Prediction: FR	JSON response received	As Expected	Pass

Table 4.4: Unit Testing Results for NFR Quality Attribute Classification

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/ Fail
TC001	Verify Performance classification of REQ-16	API server running	Send POST to /predict/nfr-type	"The system shall respond to product search queries within 2 seconds under normal load."	Prediction: Performance	JSON response received	As Expected	Pass
TC002	Verify Performance classification of REQ-17	API server running	Send POST to /predict/nfr-type	"The system shall support at least 500 concurrent users without performance degradation."	Prediction: Performance	JSON response received	As Expected	Pass
TC003	Verify Security classification of REQ-18	API server running	Send POST to /predict/nfr-type	"The system shall encrypt all payment and personal data using AES-256 encryption."	Prediction: Security	JSON response received	As Expected	Pass
TC004	Verify Reliability classification of REQ-20	API server running	Send POST to /predict/nfr-type	"The system shall be available 99.9% of the time on a monthly basis."	Prediction: Reliability	JSON response received	As Expected	Pass

Continued on next page

Table 4.4 Continued from previous page

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/ Fail
TC005	Verify Reliability classification of REQ-21	API server running	Send POST to /predict/nfr-type	"The system shall recover from any server failure within 5 minutes without data loss."	Prediction: Reliability	JSON response received	As Expected	Pass
TC006	Verify Usability classification of REQ-22	API server running	Send POST to /predict/nfr-type	"The system shall provide a responsive web interface accessible on desktop and mobile browsers."	Prediction: Usability	JSON response received	As Expected	Pass
TC007	Verify Usability classification of REQ-23	API server running	Send POST to /predict/nfr-type	"The system shall allow new users to complete their first purchase within 10 minutes without external help."	Prediction: Usability	JSON response received	As Expected	Pass
TC008	Verify Portability classification of REQ-24	API server running	Send POST to /predict/nfr-type	"The system shall run on Chrome, Firefox, Safari, and Edge browsers without functional differences."	Prediction: Portability	JSON response received	As Expected	Pass
TC009	Verify Compatibility classification of REQ-25	API server running	Send POST to /predict/nfr-type	"The system shall integrate with third-party payment gateways including Stripe and PayPal."	Prediction: Compatibility	JSON response received	As Expected	Pass
TC010	Verify Portability classification of REQ-26	API server running	Send POST to /predict/nfr-type	"The system shall be deployable on AWS, Azure, and Google Cloud without code changes."	Prediction: Portability	JSON response received	As Expected	Pass
TC011	Verify Maintainability classification of REQ-27	API server running	Send POST to /predict/nfr-type	"The system shall follow modular architecture to allow addition of new modules without affecting existing ones."	Prediction: Maintainability	JSON response received	As Expected	Pass
TC012	Verify Security classification of REQ-28	API server running	Send POST to /predict/nfr-type	"The system shall log all user actions including login, purchase, and profile updates for audit purposes."	Prediction: Security	JSON response received	As Expected	Pass
TC013	Verify Usability classification of REQ-30	API server running	Send POST to /predict/nfr-type	"The system should handle many users simultaneously without issues."	Prediction: Usability	JSON response received	As Expected	Pass
TC014	Verify Usability classification of REQ-31	API server running	Send POST to /predict/nfr-type	"The system shall provide a user-friendly and intuitive shopping experience."	Prediction: Usability	JSON response received	As Expected	Pass
TC015	Verify Maintainability classification of REQ-32	API server running	Send POST to /predict/nfr-type	"The system shall ensure products are delivered in a reasonable time."	Prediction: Maintainability	JSON response received	As Expected	Pass

Continued on next page

Table 4.4 Continued from previous page

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/ Fail
TC016	Verify Reliability classification of REQ-33	API server running	Send POST to /predict/nfr-type	"The system shall bank all transaction data securely in the backend."	Prediction: Reliability	JSON response received	As Expected	Pass
TC017	Verify Security classification of REQ-35	API server running	Send POST to /predict/nfr-type	"The system should support registration by using email and password."	Prediction: Security	JSON response received	As Expected	Pass
TC018	Verify Usability classification of REQ-38	API server running	Send POST to /predict/nfr-type	"The system shall require all users to register and log in before accessing checkout."	Prediction: Usability	JSON response received	As Expected	Pass
TC019	Verify Reliability classification of REQ-40	API server running	Send POST to /predict/nfr-type	"The system shall process refund requests within 7 to 10 business days after approval."	Prediction: Reliability	JSON response received	As Expected	Pass
TC020	Verify Performance classification of REQ-41	API server running	Send POST to /predict/nfr-type	"The system shall support a maximum of 200 concurrent users at peak load."	Prediction: Performance	JSON response received	As Expected	Pass
TC021	Verify Performance classification	API server running	Send POST to /predict/nfr-type	"The system shall respond to search queries within 200ms."	Prediction: Performance	JSON response received	As Expected	Pass
TC022	Verify Performance classification	API server running	Send POST to /predict/nfr-type	"The application shall handle 1000 concurrent users without degradation."	Prediction: Performance	JSON response received	As Expected	Pass
TC023	Verify Security classification	API server running	Send POST to /predict/nfr-type	"All user passwords must be hashed using bcrypt."	Prediction: Security	JSON response received	As Expected	Pass
TC024	Verify Security classification	API server running	Send POST to /predict/nfr-type	"The system shall use SSL/TLS for all communications."	Prediction: Security	JSON response received	As Expected	Pass
TC025	Verify Usability classification	API server running	Send POST to /predict/nfr-type	"The interface shall be intuitive enough for new users to learn within 10 minutes."	Prediction: Usability	JSON response received	As Expected	Pass
TC026	Verify Usability classification	API server running	Send POST to /predict/nfr-type	"The system shall provide clear error messages for invalid inputs."	Prediction: Usability	JSON response received	As Expected	Pass
TC027	Verify Reliability classification	API server running	Send POST to /predict/nfr-type	"The system shall have an availability of 99.99 percent during business hours."	Prediction: Reliability	JSON response received	As Expected	Pass

Continued on next page

Table 4.4 Continued from previous page

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/ Fail
TC028	Verify Reliability classification	API server running	Send POST to /predict/nfr-type	"The system shall recover from a crash within 2 minutes."	Prediction: Reliability	JSON response received	As Expected	Pass
TC029	Verify Maintainability classification	API server running	Send POST to /predict/nfr-type	"The code shall follow the PEP 8 style guide for Python."	Prediction: Maintainability	JSON response received	As Expected	Pass
TC030	Verify Portability classification	API server running	Send POST to /predict/nfr-type	"The application shall run on both Windows and macOS."	Prediction: Portability	JSON response received	As Expected	Pass
TC031	Verify Portability classification	API server running	Send POST to /predict/nfr-type	"The database shall be migratable to a different cloud provider without code changes."	Prediction: Portability	JSON response received	As Expected	Pass
TC032	Verify Performance classification of REQ-32	API server running	Send POST to /predict/nfr-type	“The system shall load restaurant listings within 3 seconds under normal network conditions.”	Prediction: Performance	JSON response received	As Expected	Pass
TC033	Verify Performance classification of REQ-33	API server running	Send POST to /predict/nfr-type	“The system shall support at least 1000 concurrent users without performance degradation.”	Prediction: Performance	JSON response received	As Expected	Pass
TC034	Verify Security classification of REQ-34	API server running	Send POST to /predict/nfr-type	“The system shall encrypt all payment and personal data using AES-256 encryption.”	Prediction: Security	JSON response received	As Expected	Pass
TC035	Verify Security classification of REQ-35	API server running	Send POST to /predict/nfr-type	“The system shall log all user actions including login, order placement, and payment for audit purposes.”	Prediction: Security	JSON response received	As Expected	Pass
TC036	Verify Reliability classification of REQ-36	API server running	Send POST to /predict/nfr-type	“The system shall be available 99.9% of the time on a monthly basis.”	Prediction: Reliability	JSON response received	As Expected	Pass
TC037	Verify Reliability classification of REQ-37	API server running	Send POST to /predict/nfr-type	“The system shall recover from any server failure within 5 minutes without data loss.”	Prediction: Reliability	JSON response received	As Expected	Pass
TC038	Verify Usability classification of REQ-38	API server running	Send POST to /predict/nfr-type	“The system shall provide a responsive mobile-first interface accessible on Android and iOS.”	Prediction: Usability	JSON response received	As Expected	Pass
TC039	Verify Usability classification of REQ-39	API server running	Send POST to /predict/nfr-type	“The system shall allow new users to place their first order within 5 minutes without external help.”	Prediction: Usability	JSON response received	As Expected	Pass

Continued on next page

Table 4.4 Continued from previous page

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/ Fail
TC040	Verify Portability classification of REQ-40	API server running	Send POST to /predict/nfr-type	"The system shall run on Chrome, Firefox, Safari, and Edge browsers without functional differences."	Prediction: Portability	JSON response received	As Expected	Pass
TC041	Verify Portability classification of REQ-41	API server running	Send POST to /predict/nfr-type	"The system shall be deployable on AWS, Azure, and Google Cloud without code changes."	Prediction: Portability	JSON response received	As Expected	Pass
TC042	Verify Maintainability classification of REQ-42	API server running	Send POST to /predict/nfr-type	"The system shall follow modular architecture to allow addition of new features without affecting existing modules."	Prediction: Maintainability	JSON response received	As Expected	Pass
TC043	Verify Maintainability classification of REQ-43	API server running	Send POST to /predict/nfr-type	"The system shall provide API documentation for all endpoints to support future integrations."	Prediction: Maintainability	JSON response received	As Expected	Pass
TC044	Verify Performance classification of REQ-44	API server running	Send POST to /predict/nfr-type	"The system should handle a large number of simultaneous orders without slowing down."	Prediction: Performance	JSON response received	As Expected	Pass
TC045	Verify Usability classification of REQ-45	API server running	Send POST to /predict/nfr-type	"The system shall provide a smooth and pleasant ordering experience for all users."	Prediction: Usability	JSON response received	As Expected	Pass
TC046	Verify Reliability classification of REQ-46	API server running	Send POST to /predict/nfr-type	"The system shall ensure delivery agents reach customers in a reasonable amount of time."	Prediction: Reliability	JSON response received	As Expected	Pass
TC047	Verify Reliability classification of REQ-47	API server running	Send POST to /predict/nfr-type	"The system shall process payments in a secure and efficient manner at all times."	Prediction: Reliability	JSON response received	As Expected	Pass
TC048	Verify Performance classification of REQ-48	API server running	Send POST to /predict/nfr-type	"The system shall support a maximum of 300 concurrent users at peak load."	Prediction: Performance	JSON response received	As Expected	Pass

Table 4.5: Unit Testing Results for Ambiguity Detection

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/ Fail
TC001	Verify clear detection of REQ-1	API server running	Send POST to /predict/ambiguity	"The system shall allow registered users to browse product catalog by category."	Prediction: Clear	JSON response received	As Expected	Pass

Continued on next page

Table 4.5 Continued from previous page

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/ Fail
TC002	Verify clear detection of REQ-2	API server running	Send POST to /predict/ambiguity	"The system shall allow registered users to add products to shopping cart with quantity selection."	Prediction: Clear	JSON response received	As Expected	Pass
TC003	Verify clear detection of REQ-3	API server running	Send POST to /predict/ambiguity	"The system shall allow registered users to remove items from shopping cart at any time."	Prediction: Clear	JSON response received	As Expected	Pass
TC004	Verify clear detection of REQ-4	API server running	Send POST to /predict/ambiguity	"The system shall allow registered users to place orders using saved payment methods with order confirmation."	Prediction: Clear	JSON response received	As Expected	Pass
TC005	Verify clear detection of REQ-5	API server running	Send POST to /predict/ambiguity	"The system shall allow registered users to track order status in real time after purchase."	Prediction: Clear	JSON response received	As Expected	Pass
TC006	Verify clear detection of REQ-6	API server running	Send POST to /predict/ambiguity	"The system shall allow registered users to submit product reviews after order delivery."	Prediction: Clear	JSON response received	As Expected	Pass
TC007	Verify clear detection of REQ-7	API server running	Send POST to /predict/ambiguity	"The system shall allow registered users to add products to wishlist for later purchase."	Prediction: Clear	JSON response received	As Expected	Pass
TC008	Verify clear detection of REQ-8	API server running	Send POST to /predict/ambiguity	"The system shall allow registered users to apply discount coupons during checkout with validity check."	Prediction: Clear	JSON response received	As Expected	Pass
TC009	Verify clear detection of REQ-9	API server running	Send POST to /predict/ambiguity	"The system shall allow admin users to manage product inventory by updating stock levels."	Prediction: Clear	JSON response received	As Expected	Pass
TC010	Verify clear detection of REQ-10	API server running	Send POST to /predict/ambiguity	"The system shall allow admin users to process refund requests within 3 business days."	Prediction: Clear	JSON response received	As Expected	Pass
TC011	Verify clear detection of REQ-11	API server running	Send POST to /predict/ambiguity	"The system shall allow registered users to search products by keyword with filter support."	Prediction: Clear	JSON response received	As Expected	Pass
TC012	Verify clear detection of REQ-12	API server running	Send POST to /predict/ambiguity	"The system shall allow registered users to view order history from their account dashboard."	Prediction: Clear	JSON response received	As Expected	Pass
TC013	Verify clear detection of REQ-13	API server running	Send POST to /predict/ambiguity	"The system shall allow admin users to generate sales reports by date range."	Prediction: Clear	JSON response received	As Expected	Pass

Continued on next page

Table 4.5 Continued from previous page

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/ Fail
TC014	Verify clear detection of REQ-14	API server running	Send POST to /predict/ambiguity	"The system shall allow registered users to manage their shipping addresses in account settings."	Prediction: Clear	JSON response received	As Expected	Pass
TC015	Verify clear detection of REQ-15	API server running	Send POST to /predict/ambiguity	"The system shall allow registered users to complete payment using credit card, debit card, or digital wallet."	Prediction: Clear	JSON response received	As Expected	Pass
TC016	Verify clear detection of REQ-16	API server running	Send POST to /predict/ambiguity	"The system shall respond to product search queries within 2 seconds under normal load."	Prediction: Clear	JSON response received	As Expected	Pass
TC017	Verify clear detection of REQ-17	API server running	Send POST to /predict/ambiguity	"The system shall support at least 500 concurrent users without performance degradation."	Prediction: Clear	JSON response received	As Expected	Pass
TC018	Verify clear detection of REQ-18	API server running	Send POST to /predict/ambiguity	"The system shall encrypt all payment and personal data using AES-256 encryption."	Prediction: Clear	JSON response received	As Expected	Pass
TC019	Verify clear detection of REQ-19	API server running	Send POST to /predict/ambiguity	"The system shall authenticate all users using JWT tokens with 24-hour expiration."	Prediction: Clear	JSON response received	As Expected	Pass
TC020	Verify clear detection of REQ-20	API server running	Send POST to /predict/ambiguity	"The system shall be available 99.9% of the time on a monthly basis."	Prediction: Clear	JSON response received	As Expected	Pass
TC021	Verify clear detection of REQ-21	API server running	Send POST to /predict/ambiguity	"The system shall recover from any server failure within 5 minutes without data loss."	Prediction: Clear	JSON response received	As Expected	Pass
TC022	Verify clear detection of REQ-22	API server running	Send POST to /predict/ambiguity	"The system shall provide a responsive web interface accessible on desktop and mobile browsers."	Prediction: Clear	JSON response received	As Expected	Pass
TC023	Verify clear detection of REQ-23	API server running	Send POST to /predict/ambiguity	"The system shall allow new users to complete their first purchase within 10 minutes without external help."	Prediction: Clear	JSON response received	As Expected	Pass
TC024	Verify clear detection of REQ-24	API server running	Send POST to /predict/ambiguity	"The system shall run on Chrome, Firefox, Safari, and Edge browsers without functional differences."	Prediction: Clear	JSON response received	As Expected	Pass
TC025	Verify clear detection of REQ-25	API server running	Send POST to /predict/ambiguity	"The system shall integrate with third-party payment gateways including Stripe and PayPal."	Prediction: Clear	JSON response received	As Expected	Pass

Continued on next page

Table 4.5 Continued from previous page

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/ Fail
TC026	Verify clear detection of REQ-26	API server running	Send POST to /predict/ambiguity	"The system shall be deployable on AWS, Azure, and Google Cloud without code changes."	Prediction: Clear	JSON response received	As Expected	Pass
TC027	Verify ambiguity detection of REQ-27	API server running	Send POST to /predict/ambiguity	"The system shall follow modular architecture to allow addition of new modules without affecting existing ones."	Prediction: Ambiguous (Medium)	JSON response received	As Expected	Pass
TC028	Verify clear detection of REQ-28	API server running	Send POST to /predict/ambiguity	"The system shall log all user actions including login, purchase, and profile updates for audit purposes."	Prediction: Clear	JSON response received	As Expected	Pass
TC029	Verify ambiguity detection of REQ-29	API server running	Send POST to /predict/ambiguity	"The system shall process orders quickly and efficiently."	Prediction: Ambiguous (High)	JSON response received	As Expected	Pass
TC030	Verify ambiguity detection of REQ-30	API server running	Send POST to /predict/ambiguity	"The system should handle many users simultaneously without issues."	Prediction: Ambiguous (High)	JSON response received	As Expected	Pass
TC031	Verify ambiguity detection of REQ-31	API server running	Send POST to /predict/ambiguity	"The system shall provide a user-friendly and intuitive shopping experience."	Prediction: Ambiguous (Medium)	JSON response received	As Expected	Pass
TC032	Verify ambiguity detection of REQ-32	API server running	Send POST to /predict/ambiguity	"The system shall ensure products are delivered in a reasonable time."	Prediction: Ambiguous (High)	JSON response received	As Expected	Pass
TC033	Verify clear detection of REQ-33	API server running	Send POST to /predict/ambiguity	"The system shall bank all transaction data securely in the backend."	Prediction: Clear	JSON response received	As Expected	Pass
TC034	Verify clear detection of REQ-34	API server running	Send POST to /predict/ambiguity	"The system shall allow users register using email and password."	Prediction: Clear	JSON response received	As Expected	Pass
TC035	Verify clear detection of REQ-35	API server running	Send POST to /predict/ambiguity	"The system should support registration by using email and password."	Prediction: Clear	JSON response received	As Expected	Pass
TC036	Verify clear detection of REQ-36	API server running	Send POST to /predict/ambiguity	"The registered user shall be able to view past orders from their profile page."	Prediction: Clear	JSON response received	As Expected	Pass
TC037	Verify clear detection of REQ-37	API server running	Send POST to /predict/ambiguity	"The system shall allow guest users to checkout without creating an account."	Prediction: Clear	JSON response received	As Expected	Pass
TC038	Verify clear detection of REQ-38	API server running	Send POST to /predict/ambiguity	"The system shall require all users to register and log in before accessing checkout."	Prediction: Clear	JSON response received	As Expected	Pass
TC039	Verify clear detection of REQ-39	API server running	Send POST to /predict/ambiguity	"The system shall process all refund requests within 24 hours of submission."	Prediction: Clear	JSON response received	As Expected	Pass

Continued on next page

Table 4.5 Continued from previous page

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/ Fail
TC040	Verify clear detection of REQ-40	API server running	Send POST to /predict/ambiguity	"The system shall process refund requests within 7 to 10 business days after approval."	Prediction: Clear	JSON response received	As Expected	Pass
TC041	Verify clear detection of REQ-41	API server running	Send POST to /predict/ambiguity	"The system shall support a maximum of 200 concurrent users at peak load."	Prediction: Clear	JSON response received	As Expected	Pass
TC042	Verify clear detection of REQ-42	API server running	Send POST to /predict/ambiguity	"The system shall display product prices in USD only across all pages."	Prediction: Clear	JSON response received	As Expected	Pass
TC043	Verify clear detection of REQ-43	API server running	Send POST to /predict/ambiguity	"The system shall allow users to view product prices in their local currency based on location."	Prediction: Clear	JSON response received	As Expected	Pass
TC044	Verify clear detection of REQ-44	API server running	Send POST to /predict/ambiguity	“The system shall allow registered users to browse nearby restaurants by location and cuisine type.”	Prediction: Clear	JSON response received	As Expected	Pass
TC045	Verify clear detection of REQ-45	API server running	Send POST to /predict/ambiguity	“The system shall allow registered users to view restaurant menus with item descriptions and prices.”	Prediction: Clear	JSON response received	As Expected	Pass
TC046	Verify clear detection of REQ-46	API server running	Send POST to /predict/ambiguity	“The system shall allow registered users to add food items to cart with quantity and customization options.”	Prediction: Clear	JSON response received	As Expected	Pass
TC047	Verify clear detection of REQ-47	API server running	Send POST to /predict/ambiguity	“The system shall allow registered users to remove items from cart before checkout.”	Prediction: Clear	JSON response received	As Expected	Pass
TC048	Verify clear detection of REQ-48	API server running	Send POST to /predict/ambiguity	“The system shall allow registered users to place food orders using saved payment methods with order confirmation.”	Prediction: Clear	JSON response received	As Expected	Pass
TC049	Verify clear detection of REQ-49	API server running	Send POST to /predict/ambiguity	“The system shall allow registered users to track delivery status in real time after order placement.”	Prediction: Clear	JSON response received	As Expected	Pass
TC050	Verify clear detection of REQ-50	API server running	Send POST to /predict/ambiguity	“The system shall allow registered users to rate and review restaurants after order delivery.”	Prediction: Clear	JSON response received	As Expected	Pass
TC051	Verify clear detection of REQ-51	API server running	Send POST to /predict/ambiguity	“The system shall allow registered users to apply promo codes during checkout with validity check.”	Prediction: Clear	JSON response received	As Expected	Pass

Continued on next page

Table 4.5 Continued from previous page

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/ Fail
TC052	Verify clear detection of REQ-52	API server running	Send POST to /predict/ambiguity	"The system shall allow registered users to schedule orders for a future delivery time with confirmation."	Prediction: Clear	JSON response received	As Expected	Pass
TC053	Verify clear detection of REQ-53	API server running	Send POST to /predict/ambiguity	"The system shall allow registered users to view order history from their account dashboard."	Prediction: Clear	JSON response received	As Expected	Pass
TC054	Verify clear detection of REQ-54	API server running	Send POST to /predict/ambiguity	"The system shall allow registered users to manage saved delivery addresses in account settings."	Prediction: Clear	JSON response received	As Expected	Pass
TC055	Verify clear detection of REQ-55	API server running	Send POST to /predict/ambiguity	"The system shall allow registered users to reorder from previous orders with a single action."	Prediction: Clear	JSON response received	As Expected	Pass
TC056	Verify clear detection of REQ-56	API server running	Send POST to /predict/ambiguity	"The system shall allow restaurant owners to manage their menu by adding, updating, or removing items."	Prediction: Clear	JSON response received	As Expected	Pass
TC057	Verify clear detection of REQ-57	API server running	Send POST to /predict/ambiguity	"The system shall allow restaurant owners to accept or reject incoming orders within 2 minutes."	Prediction: Clear	JSON response received	As Expected	Pass
TC058	Verify clear detection of REQ-58	API server running	Send POST to /predict/ambiguity	"The system shall allow delivery agents to update delivery status at each checkpoint in real time."	Prediction: Clear	JSON response received	As Expected	Pass
TC059	Verify clear detection of REQ-59	API server running	Send POST to /predict/ambiguity	"The system shall allow admin users to manage restaurant listings and verify partner restaurants."	Prediction: Clear	JSON response received	As Expected	Pass
TC060	Verify clear detection of REQ-60	API server running	Send POST to /predict/ambiguity	"The system shall allow registered users to complete payment using credit card, debit card, cash on delivery, or digital wallet."	Prediction: Clear	JSON response received	As Expected	Pass
TC061	Verify clear detection of REQ-61	API server running	Send POST to /predict/ambiguity	"The system shall allow registered users to contact customer support through in-app chat after order placement."	Prediction: Clear	JSON response received	As Expected	Pass
TC062	Verify clear detection of REQ-62	API server running	Send POST to /predict/ambiguity	"The system shall load restaurant listings within 3 seconds under normal network conditions."	Prediction: Clear	JSON response received	As Expected	Pass

Continued on next page

Table 4.5 Continued from previous page

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/ Fail
TC063	Verify clear detection of REQ-63	API server running	Send POST to /predict/ambiguity	"The system shall support at least 1000 concurrent users without performance degradation."	Prediction: Clear	JSON response received	As Expected	Pass
TC064	Verify clear detection of REQ-64	API server running	Send POST to /predict/ambiguity	"The system shall encrypt all payment and personal data using AES-256 encryption."	Prediction: Clear	JSON response received	As Expected	Pass
TC065	Verify clear detection of REQ-65	API server running	Send POST to /predict/ambiguity	"The system shall authenticate all users using JWT tokens with 24-hour expiration."	Prediction: Clear	JSON response received	As Expected	Pass
TC066	Verify clear detection of REQ-66	API server running	Send POST to /predict/ambiguity	"The system shall log all user actions including login, order placement, and payment for audit purposes."	Prediction: Clear	JSON response received	As Expected	Pass
TC067	Verify clear detection of REQ-67	API server running	Send POST to /predict/ambiguity	"The system shall be available 99.9% of the time on a monthly basis."	Prediction: Clear	JSON response received	As Expected	Pass
TC068	Verify clear detection of REQ-68	API server running	Send POST to /predict/ambiguity	"The system shall recover from any server failure within 5 minutes without data loss."	Prediction: Clear	JSON response received	As Expected	Pass
TC069	Verify clear detection of REQ-69	API server running	Send POST to /predict/ambiguity	"The system shall provide a responsive mobile-first interface accessible on Android and iOS."	Prediction: Clear	JSON response received	As Expected	Pass
TC070	Verify clear detection of REQ-70	API server running	Send POST to /predict/ambiguity	"The system shall allow new users to place their first order within 5 minutes without external help."	Prediction: Clear	JSON response received	As Expected	Pass
TC071	Verify clear detection of REQ-71	API server running	Send POST to /predict/ambiguity	"The system shall run on Chrome, Firefox, Safari, and Edge browsers without functional differences."	Prediction: Clear	JSON response received	As Expected	Pass
TC072	Verify clear detection of REQ-72	API server running	Send POST to /predict/ambiguity	"The system shall integrate with third-party mapping services including Google Maps for delivery tracking."	Prediction: Clear	JSON response received	As Expected	Pass
TC073	Verify clear detection of REQ-73	API server running	Send POST to /predict/ambiguity	"The system shall be deployable on AWS, Azure, and Google Cloud without code changes."	Prediction: Clear	JSON response received	As Expected	Pass
TC074	Verify ambiguity detection of REQ-74	API server running	Send POST to /predict/ambiguity	"The system shall follow modular architecture to allow addition of new features without affecting existing modules."	Prediction: Ambiguous	JSON response received	As Expected	Pass

Continued on next page

Table 4.5 Continued from previous page

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/ Fail
TC075	Verify clear detection of REQ-75	API server running	Send POST to /predict/ambiguity	"The system shall provide API documentation for all endpoints to support future integrations."	Prediction: Clear	JSON response received	As Expected	Pass
TC076	Verify ambiguity detection of REQ-76	API server running	Send POST to /predict/ambiguity	"The system shall deliver food orders as fast as possible to nearby customers."	Prediction: Ambiguous	JSON response received	As Expected	Pass
TC077	Verify ambiguity detection of REQ-77	API server running	Send POST to /predict/ambiguity	"The system should handle a large number of simultaneous orders without slowing down."	Prediction: Ambiguous	JSON response received	As Expected	Pass
TC078	Verify ambiguity detection of REQ-78	API server running	Send POST to /predict/ambiguity	"The system shall provide a smooth and pleasant ordering experience for all users."	Prediction: Ambiguous	JSON response received	As Expected	Pass
TC079	Verify ambiguity detection of REQ-79	API server running	Send POST to /predict/ambiguity	"The system shall ensure delivery agents reach customers in a reasonable amount of time."	Prediction: Ambiguous	JSON response received	As Expected	Pass
TC080	Verify ambiguity detection of REQ-80	API server running	Send POST to /predict/ambiguity	"The system shall process payments in a secure and efficient manner at all times."	Prediction: Ambiguous	JSON response received	As Expected	Pass
TC081	Verify clear detection of REQ-81	API server running	Send POST to /predict/ambiguity	"The system shall allow customers to add meals to their basket before placing an order."	Prediction: Clear	JSON response received	As Expected	Pass
TC082	Verify clear detection of REQ-82	API server running	Send POST to /predict/ambiguity	"The system shall allow users to put food items into cart prior to checkout."	Prediction: Clear	JSON response received	As Expected	Pass
TC083	Verify clear detection of REQ-83	API server running	Send POST to /predict/ambiguity	"The system shall allow registered users to view their past food orders from their profile."	Prediction: Clear	JSON response received	As Expected	Pass
TC084	Verify clear detection of REQ-84	API server running	Send POST to /predict/ambiguity	"The system shall allow users to access previous order history from the account section."	Prediction: Clear	JSON response received	As Expected	Pass
TC085	Verify clear detection of REQ-85	API server running	Send POST to /predict/ambiguity	"The system shall allow guest users to place food orders without creating an account."	Prediction: Clear	JSON response received	As Expected	Pass
TC086	Verify clear detection of REQ-86	API server running	Send POST to /predict/ambiguity	"The system shall require all users to register and log in before accessing the ordering feature."	Prediction: Clear	JSON response received	As Expected	Pass
TC087	Verify clear detection of REQ-87	API server running	Send POST to /predict/ambiguity	"The system shall automatically assign the nearest available delivery agent to every new order."	Prediction: Clear	JSON response received	As Expected	Pass

Continued on next page

Table 4.5 Continued from previous page

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/ Fail
TC088	Verify clear detection of REQ-88	API server running	Send POST to /predict/ambiguity	"The system shall allow restaurant owners to choose and assign their own preferred delivery agents."	Prediction: Clear	JSON response received	As Expected	Pass
TC089	Verify clear detection of REQ-89	API server running	Send POST to /predict/ambiguity	"The system shall support a maximum of 300 concurrent users at peak load."	Prediction: Clear	JSON response received	As Expected	Pass
TC090	Verify clear detection of REQ-90	API server running	Send POST to /predict/ambiguity	"The system shall display all menu prices in USD only across all restaurant pages."	Prediction: Clear	JSON response received	As Expected	Pass
TC091	Verify clear detection of REQ-91	API server running	Send POST to /predict/ambiguity	"The system shall display menu prices in the local currency of the user based on their detected location."	Prediction: Clear	JSON response received	As Expected	Pass
TC092	Verify clear detection of REQ-92	API server running	Send POST to /predict/ambiguity	"The system shall cancel unaccepted orders automatically after 5 minutes of no restaurant response."	Prediction: Clear	JSON response received	As Expected	Pass
TC093	Verify clear detection of REQ-93	API server running	Send POST to /predict/ambiguity	"The system shall wait up to 15 minutes before cancelling orders that have not been accepted by the restaurant."	Prediction: Clear	JSON response received	As Expected	Pass
TC094	Verify clear detection of REQ-94	API server running	Send POST to /predict/ambiguity	"The system shall allow admin users to create and manage courses with title, description, and category."	Prediction: Clear	JSON response received	As Expected	Pass
TC095	Verify clear detection of REQ-95	API server running	Send POST to /predict/ambiguity	"The system shall allow instructors to upload course content including videos, PDFs, and presentations."	Prediction: Clear	JSON response received	As Expected	Pass
TC096	Verify clear detection of REQ-96	API server running	Send POST to /predict/ambiguity	"The system shall allow registered students to browse and search available courses by category and keyword."	Prediction: Clear	JSON response received	As Expected	Pass
TC097	Verify clear detection of REQ-97	API server running	Send POST to /predict/ambiguity	"The system shall allow registered students to enroll in courses after completing payment or free registration."	Prediction: Clear	JSON response received	As Expected	Pass
TC098	Verify clear detection of REQ-98	API server running	Send POST to /predict/ambiguity	"The system shall allow registered students to access course content after successful enrollment."	Prediction: Clear	JSON response received	As Expected	Pass

Continued on next page

Table 4.5 Continued from previous page

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/ Fail
TC099	Verify clear detection of REQ-99	API server running	Send POST to /predict/ambiguity	"The system shall allow registered students to track their course progress with completion percentage."	Prediction: Clear	JSON response received	As Expected	Pass
TC100	Verify clear detection of REQ-100	API server running	Send POST to /predict/ambiguity	"The system shall allow instructors to create quizzes and assignments with configurable deadlines."	Prediction: Clear	JSON response received	As Expected	Pass
TC101	Verify clear detection of REQ-101	API server running	Send POST to /predict/ambiguity	"The system shall allow registered students to attempt quizzes within the allowed time limit and submission window."	Prediction: Clear	JSON response received	As Expected	Pass
TC102	Verify ambiguity detection of REQ-102	API server running	Send POST to /predict/ambiguity	"The system shall allow instructors to grade student assignments and provide written feedback."	Prediction: Ambiguous	JSON response received	As Expected	Pass
TC103	Verify clear detection of REQ-103	API server running	Send POST to /predict/ambiguity	"The system shall allow registered students to view their grades and feedback from the course dashboard."	Prediction: Clear	JSON response received	As Expected	Pass
TC104	Verify clear detection of REQ-104	API server running	Send POST to /predict/ambiguity	"The system shall allow registered students to participate in course discussion forums by posting and replying."	Prediction: Clear	JSON response received	As Expected	Pass
TC105	Verify clear detection of REQ-105	API server running	Send POST to /predict/ambiguity	"The system shall allow instructors to schedule and host live classes with registered enrolled students."	Prediction: Clear	JSON response received	As Expected	Pass
TC106	Verify clear detection of REQ-106	API server running	Send POST to /predict/ambiguity	"The system shall allow registered students to download course completion certificates after finishing all requirements."	Prediction: Clear	JSON response received	As Expected	Pass
TC107	Verify clear detection of REQ-107	API server running	Send POST to /predict/ambiguity	"The system shall allow admin users to manage user accounts including students and instructors."	Prediction: Clear	JSON response received	As Expected	Pass
TC108	Verify clear detection of REQ-108	API server running	Send POST to /predict/ambiguity	"The system shall allow registered students to submit assignments before the deadline with file upload support."	Prediction: Clear	JSON response received	As Expected	Pass
TC109	Verify clear detection of REQ-109	API server running	Send POST to /predict/ambiguity	"The system shall allow instructors to send announcements to all enrolled students of a course."	Prediction: Clear	JSON response received	As Expected	Pass

Continued on next page

Table 4.5 Continued from previous page

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/ Fail
TC110	Verify clear detection of REQ-110	API server running	Send POST to /predict/ambiguity	"The system shall allow registered students to save courses to a wishlist for later enrollment."	Prediction: Clear	JSON response received	As Expected	Pass
TC111	Verify clear detection of REQ-111	API server running	Send POST to /predict/ambiguity	"The system shall allow admin users to generate reports on course enrollment and student performance."	Prediction: Clear	JSON response received	As Expected	Pass
TC112	Verify clear detection of REQ-112	API server running	Send POST to /predict/ambiguity	"The system shall load course content pages within 3 seconds under normal network conditions."	Prediction: Clear	JSON response received	As Expected	Pass
TC113	Verify clear detection of REQ-113	API server running	Send POST to /predict/ambiguity	"The system shall support at least 2000 concurrent users without performance degradation."	Prediction: Clear	JSON response received	As Expected	Pass
TC114	Verify clear detection of REQ-114	API server running	Send POST to /predict/ambiguity	"The system shall encrypt all user data and payment information using AES-256 encryption."	Prediction: Clear	JSON response received	As Expected	Pass
TC115	Verify clear detection of REQ-115	API server running	Send POST to /predict/ambiguity	"The system shall authenticate all users using JWT tokens with 24-hour expiration."	Prediction: Clear	JSON response received	As Expected	Pass
TC116	Verify clear detection of REQ-116	API server running	Send POST to /predict/ambiguity	"The system shall log all user actions including login, enrollment, and submission for audit purposes."	Prediction: Clear	JSON response received	As Expected	Pass
TC117	Verify clear detection of REQ-117	API server running	Send POST to /predict/ambiguity	"The system shall be available 99.9% of the time on a monthly basis."	Prediction: Clear	JSON response received	As Expected	Pass
TC118	Verify clear detection of REQ-118	API server running	Send POST to /predict/ambiguity	"The system shall recover from any server failure within 5 minutes without loss of student progress data."	Prediction: Clear	JSON response received	As Expected	Pass
TC119	Verify clear detection of REQ-119	API server running	Send POST to /predict/ambiguity	"The system shall provide a responsive interface accessible on desktop, tablet, and mobile browsers."	Prediction: Clear	JSON response received	As Expected	Pass
TC120	Verify clear detection of REQ-120	API server running	Send POST to /predict/ambiguity	"The system shall allow new students to enroll in their first course within 5 minutes without external help."	Prediction: Clear	JSON response received	As Expected	Pass

Continued on next page

Table 4.5 Continued from previous page

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/ Fail
TC121	Verify clear detection of REQ-121	API server running	Send POST to /predict/ambiguity	"The system shall run on Chrome, Firefox, Safari, and Edge browsers without functional differences."	Prediction: Clear	JSON response received	As Expected	Pass
TC122	Verify clear detection of REQ-122	API server running	Send POST to /predict/ambiguity	"The system shall integrate with third-party video streaming services including YouTube and Vimeo."	Prediction: Clear	JSON response received	As Expected	Pass
TC123	Verify clear detection of REQ-123	API server running	Send POST to /predict/ambiguity	"The system shall be deployable on AWS, Azure, and Google Cloud without code changes."	Prediction: Clear	JSON response received	As Expected	Pass
TC124	Verify clear detection of REQ-124	API server running	Send POST to /predict/ambiguity	"The system shall follow modular architecture to allow addition of new modules without affecting existing features."	Prediction: Clear	JSON response received	As Expected	Pass
TC125	Verify clear detection of REQ-125	API server running	Send POST to /predict/ambiguity	"The system shall provide API documentation for all endpoints to support future third-party integrations."	Prediction: Clear	JSON response received	As Expected	Pass
TC126	Verify ambiguity detection of REQ-126	API server running	Send POST to /predict/ambiguity	"The system shall deliver course content quickly and smoothly to all students."	Prediction: Ambiguous	JSON response received	As Expected	Pass
TC127	Verify ambiguity detection of REQ-127	API server running	Send POST to /predict/ambiguity	"The system should handle a large number of students accessing courses at the same time without issues."	Prediction: Ambiguous	JSON response received	As Expected	Pass
TC128	Verify ambiguity detection of REQ-128	API server running	Send POST to /predict/ambiguity	"The system shall provide an engaging and interactive learning experience for all users."	Prediction: Ambiguous	JSON response received	As Expected	Pass
TC129	Verify ambiguity detection of REQ-129	API server running	Send POST to /predict/ambiguity	"The system shall ensure instructors can easily manage their course materials without difficulty."	Prediction: Ambiguous	JSON response received	As Expected	Pass
TC130	Verify ambiguity detection of REQ-130	API server running	Send POST to /predict/ambiguity	"The system shall process student payments in a secure and timely manner during enrollment."	Prediction: Ambiguous	JSON response received	As Expected	Pass
TC131	Verify clear detection of REQ-131	API server running	Send POST to /predict/ambiguity	"The system shall allow learners to sign up for available courses after making payment."	Prediction: Clear	JSON response received	As Expected	Pass
TC132	Verify clear detection of REQ-132	API server running	Send POST to /predict/ambiguity	"The system shall allow students to register for a course upon successful fee submission."	Prediction: Clear	JSON response received	As Expected	Pass

Continued on next page

Table 4.5 Continued from previous page

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/ Fail
TC133	Verify clear detection of REQ-133	API server running	Send POST to /predict/ambiguity	"The system shall allow registered students to view their assessment scores from the learning portal."	Prediction: Clear	JSON response received	As Expected	Pass
TC134	Verify clear detection of REQ-134	API server running	Send POST to /predict/ambiguity	"The system shall allow students to access their quiz and assignment results from the course page."	Prediction: Clear	JSON response received	As Expected	Pass
TC135	Verify clear detection of REQ-135	API server running	Send POST to /predict/ambiguity	"The system shall allow guest users to access and view full course content without registration."	Prediction: Clear	JSON response received	As Expected	Pass
TC136	Verify clear detection of REQ-136	API server running	Send POST to /predict/ambiguity	"The system shall require all users to register and log in before accessing any course content."	Prediction: Clear	JSON response received	As Expected	Pass
TC137	Verify clear detection of REQ-137	API server running	Send POST to /predict/ambiguity	"The system shall automatically pass all students who complete 100 percent of course videos."	Prediction: Clear	JSON response received	As Expected	Pass
TC138	Verify clear detection of REQ-138	API server running	Send POST to /predict/ambiguity	"The system shall require students to pass all quizzes and assignments before receiving course completion certification."	Prediction: Clear	JSON response received	As Expected	Pass
TC139	Verify clear detection of REQ-139	API server running	Send POST to /predict/ambiguity	"The system shall support a maximum of 500 concurrent users at peak load."	Prediction: Clear	JSON response received	As Expected	Pass
TC140	Verify clear detection of REQ-140	API server running	Send POST to /predict/ambiguity	"The system shall display all course prices in USD only across all pages."	Prediction: Clear	JSON response received	As Expected	Pass
TC141	Verify clear detection of REQ-141	API server running	Send POST to /predict/ambiguity	"The system shall display course prices in the local currency of the student based on their detected location."	Prediction: Clear	JSON response received	As Expected	Pass
TC142	Verify clear detection of REQ-142	API server running	Send POST to /predict/ambiguity	"The system shall automatically issue course completion certificates immediately after the final quiz submission."	Prediction: Clear	JSON response received	As Expected	Pass
TC143	Verify clear detection of REQ-143	API server running	Send POST to /predict/ambiguity	"The system shall issue course completion certificates only after instructor manual approval of all submitted assignments."	Prediction: Clear	JSON response received	As Expected	Pass

Table 4.6: Unit Testing Results for Conflict and Consistency Detection

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/ Fail
TC001	Verify inconsistency detection between REQ-5 and REQ-12	API server running	Send POST to /predict/conflict	R1: "The system shall allow registered users to track order status in real time after purchase." R2: "The system shall allow registered users to view order history from their account dashboard."	Detection: Inconsistency	JSON response received	As Expected	Pass
TC002	Verify inconsistency detection between REQ-5 and REQ-36	API server running	Send POST to /predict/conflict	R1: "The system shall allow registered users to track order status in real time after purchase." R2: "The registered user shall be able to view past orders from their profile page."	Detection: Inconsistency	JSON response received	As Expected	Pass
TC003	Verify inconsistency detection between REQ-10 and REQ-39	API server running	Send POST to /predict/conflict	R1: "The system shall allow admin users to process refund requests within 3 business days." R2: "The system shall process all refund requests within 24 hours of submission."	Detection: Inconsistency	JSON response received	As Expected	Pass
TC004	Verify inconsistency detection between REQ-10 and REQ-40	API server running	Send POST to /predict/conflict	R1: "The system shall allow admin users to process refund requests within 3 business days." R2: "The system shall process refund requests within 7 to 10 business days after approval."	Detection: Inconsistency	JSON response received	As Expected	Pass
TC005	Verify inconsistency detection between REQ-12 and REQ-36	API server running	Send POST to /predict/conflict	R1: "The system shall allow registered users to view order history from their account dashboard." R2: "The registered user shall be able to view past orders from their profile page."	Detection: Inconsistency	JSON response received	As Expected	Pass
TC006	Verify conflict detection between REQ-12 and REQ-37	API server running	Send POST to /predict/conflict	R1: "The system shall allow registered users to view order history from their account dashboard." R2: "The system shall allow guest users to checkout without creating an account."	Detection: Conflict	JSON response received	As Expected	Pass

Continued on next page

Table 4.6 Continued from previous page

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/ Fail
TC007	Verify conflict detection between REQ-13 and REQ-37	API server running	Send POST to /predict/conflict	R1: "The system shall allow admin users to generate sales reports by date range." R2: "The system shall allow guest users to checkout without creating an account."	Detection: Conflict	JSON response received	As Expected	Pass
TC008	Verify conflict detection between REQ-14 and REQ-37	API server running	Send POST to /predict/conflict	R1: "The system shall allow registered users to manage their shipping addresses in account settings." R2: "The system shall allow guest users to checkout without creating an account."	Detection: Conflict	JSON response received	As Expected	Pass
TC009	Verify duplicate detection between REQ-17 and REQ-30	API server running	Send POST to /predict/conflict	R1: "The system shall support at least 500 concurrent users without performance degradation." R2: "The system should handle many users simultaneously without issues."	Detection: Duplicate	JSON response received	As Expected	Pass
TC010	Verify inconsistency detection between REQ-17 and REQ-41	API server running	Send POST to /predict/conflict	R1: "The system shall support at least 500 concurrent users without performance degradation." R2: "The system shall support a maximum of 200 concurrent users at peak load."	Detection: Inconsistency	JSON response received	As Expected	Pass
TC011	Verify duplicate detection between REQ-34 and REQ-35	API server running	Send POST to /predict/conflict	R1: "The system shall allow users register using email and password." R2: "The system should support registration by using email and password."	Detection: Duplicate	JSON response received	As Expected	Pass
TC012	Verify conflict detection between REQ-36 and REQ-37	API server running	Send POST to /predict/conflict	R1: "The registered user shall be able to view past orders from their profile page." R2: "The system shall allow guest users to checkout without creating an account."	Detection: Conflict	JSON response received	As Expected	Pass
TC013	Verify inconsistency detection between REQ-39 and REQ-40	API server running	Send POST to /predict/conflict	R1: "The system shall process all refund requests within 24 hours of submission." R2: "The system shall process refund requests within 7 to 10 business days after approval."	Detection: Inconsistency	JSON response received	As Expected	Pass

Continued on next page

Table 4.6 Continued from previous page

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/ Fail
TC014	Verify inconsistency detection between REQ-42 and REQ-43	API server running	Send POST to /predict/conflict	R1: "The system shall display product prices in USD only across all pages." R2: "The system shall allow users to view product prices in their local currency based on location."	Detection: Inconsistency	JSON response received	As Expected	Pass
TC015	Verify inconsistency detection between REQ-19 and REQ-45	API server running	Send POST to /predict/conflict	R1: “The system shall support at least 1000 concurrent users without performance degradation.” R2: “The system shall support a maximum of 300 concurrent users at peak load.”	Detection: Inconsistency	JSON response received	As Expected	Pass
TC016	Verify inconsistency detection between REQ-46 and REQ-47	API server running	Send POST to /predict/conflict	R1: “The system shall display all menu prices in USD only across all restaurant pages.” R2: “The system shall display menu prices in the local currency of the user based on their detected location.”	Detection: Inconsistency	JSON response received	As Expected	Pass
TC017	Verify inconsistency detection between REQ-48 and REQ-49	API server running	Send POST to /predict/conflict	R1: “The system shall cancel unaccepted orders automatically after 5 minutes of no restaurant response.” R2: “The system shall wait up to 15 minutes before cancelling orders that have not been accepted by the restaurant.”	Detection: Inconsistency	JSON response received	As Expected	Pass
TC018	Verify inconsistency detection between REQ-9 and REQ-39	API server running	Send POST to /predict/conflict	R1: “The system shall allow registered users to view order history from their account dashboard.” R2: “The system shall allow registered users to view their past food orders from their profile.”	Detection: Inconsistency	JSON response received	As Expected	Pass
TC019	Verify inconsistency detection between REQ-9 and REQ-40	API server running	Send POST to /predict/conflict	R1: “The system shall allow registered users to view order history from their account dashboard.” R2: “The system shall allow users to access previous order history from the account section.”	Detection: Inconsistency	JSON response received	As Expected	Pass

Continued on next page

Table 4.6 Continued from previous page

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/ Fail
TC020	Verify duplicate detection between REQ-2 and REQ-38	API server running	Send POST to /predict/conflict	R1: "The system shall allow registered users to add food items to cart with quantity and customization options." R2: "The system shall allow users to put food items into cart prior to checkout."	Detection: Duplicate	JSON response received	As Expected	Pass
TC021	Verify duplicate detection between REQ-37 and REQ-41	API server running	Send POST to /predict/conflict	R1: "The system shall allow customers to add meals to their basket before placing an order." R2: "The system shall allow guest users to place food orders without creating an account."	Detection: Duplicate	JSON response received	As Expected	Pass
TC022	Verify inconsistency detection between REQ-20 and REQ-46	API server running	Send POST to /predict/conflict	R1: "The system shall support at least 2000 concurrent users without performance degradation." R2: "The system shall support a maximum of 500 concurrent users at peak load."	Detection: Inconsistency	JSON response received	As Expected	Pass
TC023	Verify inconsistency detection between REQ-47 and REQ-48	API server running	Send POST to /predict/conflict	R1: "The system shall display all course prices in USD only across all pages." R2: "The system shall display course prices in the local currency of the student based on their detected location."	Detection: Inconsistency	JSON response received	As Expected	Pass
TC024	Verify inconsistency detection between REQ-49 and REQ-50	API server running	Send POST to /predict/conflict	R1: "The system shall automatically issue course completion certificates immediately after the final quiz submission." R2: "The system shall issue course completion certificates only after instructor manual approval of all submitted assignments."	Detection: Inconsistency	JSON response received	As Expected	Pass
TC025	Verify inconsistency detection between REQ-10 and REQ-40	API server running	Send POST to /predict/conflict	R1: "The system shall allow registered students to view their grades and feedback from the course dashboard." R2: "The system shall allow registered students to view their assessment scores from the learning portal."	Detection: Inconsistency	JSON response received	As Expected	Pass

Continued on next page

Table 4.6 Continued from previous page

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/ Fail
TC026	Verify inconsistency detection between REQ-10 and REQ-41	API server running	Send POST to /predict/conflict	R1: “The system shall allow registered students to view their grades and feedback from the course dashboard.” R2: “The system shall allow students to access their quiz and assignment results from the course page.”	Detection: Inconsistency	JSON response received	As Expected	Pass
TC027	Verify duplicate detection between REQ-4 and REQ-38	API server running	Send POST to /predict/conflict	R1: “The system shall allow registered students to enroll in courses after completing payment or free registration.” R2: “The system shall allow learners to sign up for available courses after making payment.”	Detection: Duplicate	JSON response received	As Expected	Pass
TC028	Verify duplicate detection between REQ-4 and REQ-39	API server running	Send POST to /predict/conflict	R1: “The system shall allow registered students to enroll in courses after completing payment or free registration.” R2: “The system shall allow students to register for a course upon successful fee submission.”	Detection: Duplicate	JSON response received	As Expected	Pass
TC029	Verify duplicate detection between REQ-38 and REQ-39	API server running	Send POST to /predict/conflict	R1: “The system shall allow learners to sign up for available courses after making payment.” R2: “The system shall allow students to register for a course upon successful fee submission.”	Detection: Duplicate	JSON response received	As Expected	Pass
TC030	Verify inconsistency detection between REQ-5 and REQ-12	API server running	Send POST to /predict/conflict	R1: "The system shall allow registered users to track order status in real time after purchase." R2: "The system shall allow registered users to view order history from their account dashboard."	Detection: Inconsistency	JSON response received	As Expected	Pass
TC031	Verify inconsistency detection between REQ-5 and REQ-36	API server running	Send POST to /predict/conflict	R1: "The system shall allow registered users to track order status in real time after purchase." R2: "The registered user shall be able to view past orders from their profile page."	Detection: Inconsistency	JSON response received	As Expected	Pass

Continued on next page

Table 4.6 Continued from previous page

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/ Fail
TC032	Verify inconsistency detection between REQ-10 and REQ-39	API server running	Send POST to /predict/conflict	R1: "The system shall allow admin users to process refund requests within 3 business days." R2: "The system shall process all refund requests within 24 hours of submission."	Detection: Inconsistency	JSON response received	As Expected	Pass
TC033	Verify conflict detection between REQ-1 and REQ-2	API server running	Send POST to /predict/conflict	R1: "The system shall allow admin users to create and manage courses with title, description, and category." R2: "The system shall allow instructors to upload course content including videos, PDFs, and presentations."	Detection: Conflict	JSON response received	As Expected	Pass
TC034	Verify conflict detection between REQ-1 and REQ-3	API server running	Send POST to /predict/conflict	R1: "The system shall allow admin users to create and manage courses with title, description, and category." R2: "The system shall allow registered students to browse and search available courses by category and keyword."	Detection: Conflict	JSON response received	As Expected	Pass
TC035	Verify conflict detection between REQ-1 and REQ-6	API server running	Send POST to /predict/conflict	R1: "The system shall allow admin users to create and manage courses with title, description, and category." R2: "The system shall allow instructors to schedule and host live classes with registered enrolled students."	Detection: Conflict	JSON response received	As Expected	Pass
TC036	Verify inconsistency detection between REQ-1 and REQ-8	API server running	Send POST to /predict/conflict	R1: "The system shall allow admin users to create and manage courses with title, description, and category." R2: "The system shall allow registered students to view their grades and feedback from the course dashboard."	Detection: Inconsistency	JSON response received	As Expected	Pass
TC037	Verify conflict detection between REQ-1 and REQ-15	API server running	Send POST to /predict/conflict	R1: "The system shall allow admin users to create and manage courses with title, description, and category." R2: "The system shall allow guest users to access and view full course content without registration."	Detection: Conflict	JSON response received	As Expected	Pass

Continued on next page

Table 4.6 Continued from previous page

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/ Fail
TC038	Verify conflict detection between REQ-4 and REQ-6	API server running	Send POST to /predict/conflict	R1: "The system shall allow instructors to upload course content including videos, PDFs, and presentations." R2: "The system shall allow instructors to schedule and host live classes with registered enrolled students."	Detection: Conflict	JSON response received	As Expected	Pass
TC039	Verify conflict detection between REQ-4 and REQ-9	API server running	Send POST to /predict/conflict	R1: "The system shall allow instructors to upload course content including videos, PDFs, and presentations." R2: "The system shall allow instructors to send announcements to all enrolled students of a course."	Detection: Conflict	JSON response received	As Expected	Pass
TC040	Verify conflict detection between REQ-7 and REQ-15	API server running	Send POST to /predict/conflict	R1: "The system shall allow registered students to enroll in courses after completing payment or free registration." R2: "The system shall allow guest users to access and view full course content without registration."	Detection: Conflict	JSON response received	As Expected	Pass
TC041	Verify inconsistency detection between REQ-7 and REQ-11	API server running	Send POST to /predict/conflict	R1: "The system shall allow registered students to enroll in courses after completing payment or free registration." R2: "The system shall allow registered students to track their course progress with completion percentage."	Detection: Inconsistency	JSON response received	As Expected	Pass
TC042	Verify duplicate detection between REQ-7 and REQ-12	API server running	Send POST to /predict/conflict	R1: "The system shall allow registered students to enroll in courses after completing payment or free registration." R2: "The system shall allow students to register for a course upon successful fee submission."	Detection: Duplicate	JSON response received	As Expected	Pass

Continued on next page

Table 4.6 Continued from previous page

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/ Fail
TC043	Verify conflict detection between REQ-7 and REQ-15	API server running	Send POST to /predict/conflict	R1: "The system shall allow registered students to enroll in courses after completing payment or free registration." R2: "The system shall allow guest users to access and view full course content without registration."	Detection: Conflict	JSON response received	As Expected	Pass
TC044	Verify conflict detection between REQ-10 and REQ-16	API server running	Send POST to /predict/conflict	R1: "The system shall allow registered students to access course content after successful enrollment." R2: "The system shall allow guest users to access and view full course content without registration."	Detection: Conflict	JSON response received	As Expected	Pass
TC045	Verify conflict detection between REQ-10 and REQ-17	API server running	Send POST to /predict/conflict	R1: "The system shall allow registered students to access course content after successful enrollment." R2: "The system shall require all users to register and log in before accessing any course content."	Detection: Conflict	JSON response received	As Expected	Pass
TC046	Verify inconsistency detection between REQ-11 and REQ-20	API server running	Send POST to /predict/conflict	R1: "The system shall allow registered students to track their course progress with completion percentage." R2: "The system shall allow registered students to view their assessment scores from the learning portal."	Detection: Inconsistency	JSON response received	As Expected	Pass
TC047	Verify conflict detection between REQ-13 and REQ-18	API server running	Send POST to /predict/conflict	R1: "The system shall allow registered students to attempt quizzes within the allowed time limit and submission window." R2: "The system shall allow students to access their quiz and assignment results from the course page."	Detection: Conflict	JSON response received	As Expected	Pass
TC048	Verify conflict detection between REQ-14 and REQ-21	API server running	Send POST to /predict/conflict	R1: "The system shall allow instructors to grade student assignments and provide written feedback." R2: "The system shall allow registered students to view their grades and feedback from the course dashboard."	Detection: Conflict	JSON response received	As Expected	Pass

Continued on next page

Table 4.6 Continued from previous page

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/ Fail
TC049	Verify inconsistency detection between REQ-18 and REQ-19	API server running	Send POST to /predict/conflict	R1: "The system shall allow registered students to view their grades and feedback from the course dashboard." R2: "The system shall allow registered students to participate in course discussion forums by posting and replying."	Detection: Inconsistency	JSON response received	As Expected	Pass
TC050	Verify conflict detection between REQ-15 and REQ-17	API server running	Send POST to /predict/conflict	R1: "The system shall allow guest users to access and view full course content without registration." R2: "The system shall require all users to register and log in before accessing any course content."	Detection: Conflict	JSON response received	As Expected	Pass
TC051	Verify conflict detection between REQ-22 and REQ-23	API server running	Send POST to /predict/conflict	R1: "The system shall display all course prices in USD only across all pages." R2: "The system shall display course prices in the local currency of the student based on their detected location."	Detection: Conflict	JSON response received	As Expected	Pass

Table 4.7: Unit Testing Results for Completeness Analysis

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/ Fail
TC001	Verify Product Catalog module coverage	API server running, Domain: ecommerce	Send POST to /api/ completeness	43 e-commerce requirements (REQ-1 to REQ-43) including product browsing and search requirements	Module: Product Catalog detected, Status: Covered	JSON response received	As Expected	Pass
TC002	Verify Shopping Cart module coverage	API server running, Domain: ecommerce	Send POST to /api/ completeness	43 e-commerce requirements (REQ-1 to REQ-43) including cart add/remove requirements	Module: Shopping Cart detected, Status: Covered	JSON response received	As Expected	Pass
TC003	Verify Order Placement module coverage	API server running, Domain: ecommerce	Send POST to /api/ completeness	43 e-commerce requirements (REQ-1 to REQ-43) including order and checkout requirements	Module: Order Placement detected, Status: Covered	JSON response received	As Expected	Pass

Continued on next page

Table 4.7 Continued from previous page

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/ Fail
TC004	Verify Payment Processing module coverage	API server running, Domain: ecommerce	Send POST to /api/ completeness	43 e-commerce requirements (REQ-1 to REQ-43) including payment method and gateway requirements	Module: Payment Processing detected, Status: Covered	JSON response received	As Expected	Pass
TC005	Verify Wishlist module coverage	API server running, Domain: ecommerce	Send POST to /api/ completeness	43 e-commerce requirements (REQ-1 to REQ-43) including wishlist requirements	Module: Wishlist detected, Status: Covered	JSON response received	As Expected	Pass
TC006	Verify Inventory Management module coverage	API server running, Domain: ecommerce	Send POST to /api/ completeness	43 e-commerce requirements (REQ-1 to REQ-43) including inventory and stock level requirements	Module: Inventory Management detected, Status: Covered	JSON response received	As Expected	Pass
TC007	Verify Order Tracking module coverage	API server running, Domain: ecommerce	Send POST to /api/ completeness	43 e-commerce requirements (REQ-1 to REQ-43) including order tracking and status requirements	Module: Order Tracking detected, Status: Covered	JSON response received	As Expected	Pass
TC008	Verify User Reviews module coverage	API server running, Domain: ecommerce	Send POST to /api/ completeness	43 e-commerce requirements (REQ-1 to REQ-43) including product review requirements	Module: User Reviews detected, Status: Covered	JSON response received	As Expected	Pass
TC009	Verify overall completeness status as Complete	API server running, Domain: ecommerce	Send POST to /api/ completeness	43 e-commerce requirements (REQ-1 to REQ-43) covering all 8 domain modules	Coverage: 100%, Status: Complete, Missing: 0 modules	JSON response received	As Expected	Pass
TC010	Verify no missing modules reported	API server running, Domain: ecommerce	Send POST to /api/ completeness	43 e-commerce requirements (REQ-1 to REQ-43) covering all 8 domain modules	Missing modules list: Empty	JSON response received	As Expected	Pass
TC011	Verify Course Management module coverage	API server running, Domain: LMS	Send POST to /api/ completeness	LMS requirements including course creation, update, category management	Module: Course Management detected, Status: Covered	JSON response received	As Expected	Pass

Continued on next page

Table 4.7 Continued from previous page

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/ Fail
TC012	Verify Enrollment module coverage	API server running, Domain: LMS	Send POST to /api/ completeness	LMS requirements including student enrollment, registration, and access control	Module: Enrollment detected, Status: Covered	JSON response received	As Expected	Pass
TC013	Verify Assessment & Quizzes module coverage	API server running, Domain: LMS	Send POST to /api/ completeness	LMS requirements including quiz creation, attempt, deadlines, and time limits	Module: Assessment & Quizzes detected, Status: Covered	JSON response received	As Expected	Pass
TC014	Verify Grading module coverage	API server running, Domain: LMS	Send POST to /api/ completeness	LMS requirements including assignment grading and feedback generation	Module: Grading detected, Status: Covered	JSON response received	As Expected	Pass
TC015	Verify Assignment Submission module coverage	API server running, Domain: LMS	Send POST to /api/ completeness	LMS requirements including file upload and submission before deadline	Module: Assignment Submission detected, Status: Covered	JSON response received	As Expected	Pass
TC016	Verify Discussion Forums module coverage	API server running, Domain: LMS	Send POST to /api/ completeness	LMS requirements including posting, replying, and engagement features	Module: Discussion Forums detected, Status: Covered	JSON response received	As Expected	Pass
TC017	Verify Content Delivery module coverage	API server running, Domain: LMS	Send POST to /api/ completeness	LMS requirements including video, PDF, presentation access and performance constraints	Module: Content Delivery detected, Status: Covered	JSON response received	As Expected	Pass
TC018	Verify Live Classes module coverage	API server running, Domain: LMS	Send POST to /api/ completeness	LMS requirements including scheduling and hosting live sessions	Module: Live Classes detected, Status: Covered	JSON response received	As Expected	Pass
TC019	Verify Certificates module coverage	API server running, Domain: LMS	Send POST to /api/ completeness	LMS requirements including certificate generation and download rules	Module: Certificates detected, Status: Covered	JSON response received	As Expected	Pass

Continued on next page

Table 4.7 Continued from previous page

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/ Fail
TC020	Verify Order Placement module coverage	API server running, Domain: Food Delivery	Send POST to /api/ completeness	Requirements including placing orders, scheduling orders, and confirmation flow	Module: Order Placement detected, Status: Covered	JSON response received	As Expected	Pass
TC021	Verify Payment Processing module coverage	API server running, Domain: Food Delivery	Send POST to /api/ completeness	Requirements including credit/debit card, wallet, COD, and secure payment processing	Module: Payment Processing detected, Status: Covered	JSON response received	As Expected	Pass
TC022	Verify Delivery Tracking module coverage	API server running, Domain: Food Delivery	Send POST to /api/ completeness	Requirements including real-time delivery status updates and agent tracking	Module: Delivery Tracking detected, Status: Covered	JSON response received	As Expected	Pass
TC023	Verify Cart Management module coverage	API server running, Domain: Food Delivery	Send POST to /api/ completeness	Requirements including add/remove items, quantity updates, and customization options	Module: Cart Management detected, Status: Covered	JSON response received	As Expected	Pass
TC024	Verify Rating & Reviews module coverage	API server running, Domain: Food Delivery	Send POST to /api/ completeness	Requirements including restaurant rating and post-delivery reviews	Module: Rating & Reviews detected, Status: Covered	JSON response received	As Expected	Pass
TC025	Verify Order History module coverage	API server running, Domain: Food Delivery	Send POST to /api/ completeness	Requirements including viewing past orders and reorder functionality	Module: Order History detected, Status: Covered	JSON response received	As Expected	Pass
TC026	Verify Restaurant & Menu Management module coverage	API server running, Domain: Food Delivery	Send POST to /api/ completeness	Requirements including restaurant menu CRUD and restaurant management features	Module: Restaurant & Menu Management detected, Status: Covered	JSON response received	As Expected	Pass
TC027	Verify Promotions & Discounts module coverage	API server running, Domain: Food Delivery	Send POST to /api/ completeness	Requirements including promo codes, discounts, and validity checks	Module: Promotions & Discounts detected, Status: Covered	JSON response received	As Expected	Pass

Table 4.8: Unit Testing Results for Requirement Prioritization

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/ Fail
TC001	Verify prioritization of REQ-001 (clear, no issues)	API server running	Send POST to /api/ prioritize	"The data shall be encrypted using AES-256."	Priority: Must Have, Score: 100, Rating: Excellent	JSON response received	As Expected	Pass
TC002	Verify prioritization of REQ-004 (clear, no issues)	API server running	Send POST to /api/ prioritize	"The system shall send order emails within 10 minutes of purchase."	Priority: Must Have, Score: 100, Rating: Excellent	JSON response received	As Expected	Pass
TC003	Verify prioritization of REQ-012 (clear, no issues)	API server running	Send POST to /api/ prioritize	"The system shall load the homepage within 2 seconds under normal network conditions."	Priority: Must Have, Score: 100, Rating: Excellent	JSON response received	As Expected	Pass
TC004	Verify prioritization of REQ-013 (clear, no issues)	API server running	Send POST to /api/ prioritize	"The user shall update profile information including name, phone, and address."	Priority: Must Have, Score: 100, Rating: Excellent	JSON response received	As Expected	Pass
TC005	Verify prioritization of REQ-014 (clear, no issues)	API server running	Send POST to /api/ prioritize	"The system shall allow password reset via an email link."	Priority: Must Have, Score: 100, Rating: Excellent	JSON response received	As Expected	Pass
TC006	Verify prioritization of REQ-015 (clear, no issues)	API server running	Send POST to /api/ prioritize	"The user shall delete the account and all associated data permanently."	Priority: Must Have, Score: 100, Rating: Excellent	JSON response received	As Expected	Pass
TC007	Verify prioritization of REQ-018 (clear, no issues)	API server running	Send POST to /api/ prioritize	"The system shall display product listings with images, descriptions, and prices."	Priority: Must Have, Score: 100, Rating: Excellent	JSON response received	As Expected	Pass

Continued on next page

Table 4.8 Continued from previous page

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/ Fail
TC008	Verify prioritization of REQ-019 (clear, no issues)	API server running	Send POST to /api/ prioritize	"The user shall search products by name, category, or keyword."	Priority: Must Have, Score: 100, Rating: Excellent	JSON response received	As Expected	Pass
TC009	Verify prioritization of REQ-021 (clear, no issues)	API server running	Send POST to /api/ prioritize	"The user shall filter products by price range from $0–$10,000."	Priority: Must Have, Score: 100, Rating: Excellent	JSON response received	As Expected	Pass
TC010	Verify prioritization of REQ-022 (clear, no issues)	API server running	Send POST to /api/ prioritize	"The system shall allow users to filter products by cheap, medium, and expensive."	Priority: Must Have, Score: 100, Rating: Excellent	JSON response received	As Expected	Pass
TC011	Verify prioritization of REQ-023 (clear, no issues)	API server running	Send POST to /api/ prioritize	"The user shall add products to the shopping cart with quantity selection."	Priority: Must Have, Score: 100, Rating: Excellent	JSON response received	As Expected	Pass
TC012	Verify prioritization of REQ-024 (clear, no issues)	API server running	Send POST to /api/ prioritize	"The system shall update the total cart price automatically when items change."	Priority: Must Have, Score: 100, Rating: Excellent	JSON response received	As Expected	Pass
TC013	Verify prioritization of REQ-026 (clear, no issues)	API server running	Send POST to /api/ prioritize	"The system shall save cart contents for 30 days."	Priority: Must Have, Score: 100, Rating: Excellent	JSON response received	As Expected	Pass
TC014	Verify prioritization of REQ-027 (clear, no issues)	API server running	Send POST to /api/ prioritize	"The system shall maintain atleast 99.9% uptime."	Priority: Must Have, Score: 100, Rating: Excellent	JSON response received	As Expected	Pass

Continued on next page

Table 4.8 Continued from previous page

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/ Fail
TC015	Verify prioritization of REQ-028 (clear, no issues)	API server running	Send POST to /api/ prioritize	"The user shall save items to a wishlist for future purchase."	Priority: Must Have, Score: 100, Rating: Excellent	JSON response received	As Expected	Pass
TC016	Verify prioritization of REQ-029 (clear, no issues)	API server running	Send POST to /api/ prioritize	"The user shall place orders using a selected shipping address."	Priority: Must Have, Score: 100, Rating: Excellent	JSON response received	As Expected	Pass
TC017	Verify prioritization of REQ-030 (clear, no issues)	API server running	Send POST to /api/ prioritize	"The system shall generate a unique order ID for each transaction."	Priority: Must Have, Score: 100, Rating: Excellent	JSON response received	As Expected	Pass
TC018	Verify prioritization of REQ-031 (clear, no issues)	API server running	Send POST to /api/ prioritize	"The user shall view order history including dates and statuses."	Priority: Must Have, Score: 100, Rating: Excellent	JSON response received	As Expected	Pass
TC019	Verify prioritization of REQ-033 (clear, no issues)	API server running	Send POST to /api/ prioritize	"The system shall process payments using cards and digital wallets."	Priority: Must Have, Score: 100, Rating: Excellent	JSON response received	As Expected	Pass
TC020	Verify prioritization of REQ-034 (clear, no issues)	API server running	Send POST to /api/ prioritize	"The system shall encrypt payment data using AES-256 and transmit using TLS 1.3."	Priority: Must Have, Score: 100, Rating: Excellent	JSON response received	As Expected	Pass
TC021	Verify prioritization of REQ-035 (clear, no issues)	API server running	Send POST to /api/ prioritize	"The system shall store passwords using bcrypt (minimum 10 rounds)."	Priority: Must Have, Score: 100, Rating: Excellent	JSON response received	As Expected	Pass

Continued on next page

Table 4.8 Continued from previous page

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/ Fail
TC022	Verify prioritization of REQ-037 (clear, no issues)	API server running	Send POST to /api/ prioritize	"The system shall provide real-time order shipment tracking to customers."	Priority: Must Have, Score: 100, Rating: Excellent	JSON response received	As Expected	Pass
TC023	Verify prioritization of REQ-038 (clear, no issues)	API server running	Send POST to /api/ prioritize	"The system shall support guest checkout without registration."	Priority: Must Have, Score: 100, Rating: Excellent	JSON response received	As Expected	Pass
TC024	Verify score reduction for REQ-003 (conflict flag)	API server running	Send POST to /api/ prioritize	"The system shall send order confirmation emails within 5 minutes."	Priority: Must Have, Score: 88, Rating: Good, Flag: Conflict	JSON response received	As Expected	Pass
TC025	Verify score reduction for REQ-025 (ambiguity flag)	API server running	Send POST to /api/ prioritize	"The user shall remove items from the cart."	Priority: Must Have, Score: 84, Rating: Good, Flag: Ambiguous	JSON response received	As Expected	Pass
TC026	Verify score reduction for REQ-020 (ambiguity flag)	API server running	Send POST to /api/ prioritize	"The system shall show product availability status in real time."	Priority: Must Have, Score: 81, Rating: Fair, Flag: Ambiguous	JSON response received	As Expected	Pass
TC027	Verify score reduction for REQ-010 (ambiguity flag)	API server running	Send POST to /api/ prioritize	"The system must not encrypt user data."	Priority: Must Have, Score: 81, Rating: Fair, Flag: Ambiguous	JSON response received	As Expected	Pass

Continued on next page

Table 4.8 Continued from previous page

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/ Fail
TC028	Verify score reduction for REQ-032 (ambiguity flag)	API server running	Send POST to /api/ prioritize	"The system shall send confirmation emails immediately."	Priority: Must Have, Score: 81, Rating: Fair, Flag: Ambiguous	JSON response received	As Expected	Pass
TC029	Verify score reduction for REQ-006 (ambiguity flag)	API server running	Send POST to /api/ prioritize	"The HR manager must dismiss personnel."	Priority: Must Have, Score: 80, Rating: Fair, Flag: Ambiguous	JSON response received	As Expected	Pass
TC030	Verify score reduction for REQ-009 (ambiguity flag)	API server running	Send POST to /api/ prioritize	"The system must encrypt all user data."	Priority: Must Have, Score: 80, Rating: Fair, Flag: Ambiguous	JSON response received	As Expected	Pass
TC031	Verify score reduction for REQ-017 (ambiguity flag)	API server running	Send POST to /api/ prioritize	"The system shall verify emails as quickly as possible."	Priority: Must Have, Score: 79, Rating: Fair, Flag: Ambiguous	JSON response received	As Expected	Pass
TC032	Verify score reduction for REQ-036 (ambiguity flag)	API server running	Send POST to /api/ prioritize	"The system shall process payments instantly."	Priority: Must Have, Score: 79, Rating: Fair, Flag: Ambiguous	JSON response received	As Expected	Pass
TC033	Verify score reduction for REQ-007 (ambiguity flag)	API server running	Send POST to /api/ prioritize	"The Physician must Book a Consultation for the Client."	Priority: Must Have, Score: 78, Rating: Fair, Flag: Ambiguous	JSON response received	As Expected	Pass

Continued on next page

Table 4.8 Continued from previous page

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/ Fail
TC034	Verify score reduction for REQ-008 (ambiguity flag)	API server running	Send POST to /api/ prioritize	"The Doctor must Schedule a Visit for the Patient."	Priority: Must Have, Score: 78, Rating: Fair, Flag: Ambiguous	JSON response received	As Expected	Pass
TC035	Verify score reduction for REQ-016 (ambiguity flag)	API server running	Send POST to /api/ prioritize	"The system shall provide a very easy registration experience."	Priority: Must Have, Score: 76, Rating: Fair, Flag: Ambiguous	JSON response received	As Expected	Pass
TC036	Verify score reduction for REQ-011 (ambiguity flag)	API server running	Send POST to /api/ prioritize	"Developers must be able to perform updates without difficulty."	Priority: Must Have, Score: 74, Rating: Fair, Flag: Ambiguous	JSON response received	As Expected	Pass
TC037	Verify score reduction for REQ-002 (ambiguity + conflict)	API server running	Send POST to /api/ prioritize	"The system shall be fast"	Priority: Must Have, Score: 65, Rating: Poor, Flags: Ambiguous + Conflict	JSON response received	As Expected	Pass
TC038	Verify Should Have category for REQ-005 (ambiguity flag)	API server running	Send POST to /api/ prioritize	"The HR manager can onboard new staff"	Priority: Should Have, Flag: Ambiguous	JSON response received	As Expected	Pass
TC039	Verify prioritization of REQ-004 (clear, no issues)	API server running	Send POST to /api/prioritize	"The system shall allow registered users to remove items from cart before checkout."	Must Have, Score: 100, Rating: Excellent	JSON response received	As Expected	Pass
TC040	Verify prioritization of REQ-005 (clear, no issues)	API server running	Send POST to /api/prioritize	"The system shall allow registered users to place food orders using saved payment methods with order confirmation."	Must Have, Score: 100, Rating: Excellent	JSON response received	As Expected	Pass

Continued on next page

Table 4.8 Continued from previous page

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/ Fail
TC041	Verify prioritization of REQ-006 (clear, no issues)	API server running	Send POST to /api/prioritize	"The system shall allow registered users to track delivery status in real time after order placement."	Must Have, Score: 100, Rating: Excellent	JSON response received	As Expected	Pass
TC042	Verify prioritization of REQ-007 (clear, no issues)	API server running	Send POST to /api/prioritize	"The system shall allow registered users to rate and review restaurants after order delivery."	Must Have, Score: 100, Rating: Excellent	JSON response received	As Expected	Pass
TC043	Verify prioritization of REQ-008 (clear, no issues)	API server running	Send POST to /api/prioritize	"The system shall allow registered users to apply promo codes during checkout with validity check."	Must Have, Score: 100, Rating: Excellent	JSON response received	As Expected	Pass
TC044	Verify prioritization of REQ-009 (clear, no issues)	API server running	Send POST to /api/prioritize	"The system shall allow registered users to schedule orders for a future delivery time with confirmation."	Must Have, Score: 100, Rating: Excellent	JSON response received	As Expected	Pass
TC045	Verify prioritization of REQ-010 (clear, no issues)	API server running	Send POST to /api/prioritize	"The system shall allow registered users to view order history from their account dashboard."	Must Have, Score: 100, Rating: Excellent	JSON response received	As Expected	Pass
TC046	Verify prioritization of REQ-011 (clear, no issues)	API server running	Send POST to /api/prioritize	"The system shall allow registered users to manage saved delivery addresses in account settings."	Must Have, Score: 100, Rating: Excellent	JSON response received	As Expected	Pass
TC047	Verify prioritization of REQ-012 (clear, no issues)	API server running	Send POST to /api/prioritize	"The system shall allow registered users to reorder from previous orders with a single action."	Must Have, Score: 100, Rating: Excellent	JSON response received	As Expected	Pass
TC048	Verify prioritization of REQ-013 (clear, no issues)	API server running	Send POST to /api/prioritize	"The system shall allow restaurant owners to manage their menu by adding, updating, or removing items."	Must Have, Score: 100, Rating: Excellent	JSON response received	As Expected	Pass

Continued on next page

Table 4.8 Continued from previous page

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/ Fail
TC049	Verify prioritization of REQ-014 (clear, no issues)	API server running	Send POST to /api/prioritize	"The system shall allow restaurant owners to accept or reject incoming orders within 2 minutes."	Must Have, Score: 100, Rating: Excellent	JSON response received	As Expected	Pass
TC050	Verify prioritization of REQ-015 (clear, no issues)	API server running	Send POST to /api/prioritize	"The system shall allow delivery agents to update delivery status at each checkpoint in real time."	Must Have, Score: 100, Rating: Excellent	JSON response received	As Expected	Pass
TC051	Verify prioritization of REQ-016 (clear, no issues)	API server running	Send POST to /api/prioritize	"The system shall allow admin users to manage restaurant listings and verify partner restaurants."	Must Have, Score: 100, Rating: Excellent	JSON response received	As Expected	Pass
TC052	Verify prioritization of REQ-017 (clear, no issues)	API server running	Send POST to /api/prioritize	"The system shall allow registered users to complete payment using credit card, debit card, cash on delivery, or digital wallet."	Must Have, Score: 100, Rating: Excellent	JSON response received	As Expected	Pass
TC053	Verify prioritization of REQ-018 (clear, no issues)	API server running	Send POST to /api/prioritize	"The system shall allow registered users to contact customer support through in-app chat after order placement."	Must Have, Score: 100, Rating: Excellent	JSON response received	As Expected	Pass
TC054	Verify prioritization of REQ-019 (clear, no issues)	API server running	Send POST to /api/prioritize	"The system shall load restaurant listings within 3 seconds under normal network conditions."	Must Have, Score: 100, Rating: Excellent	JSON response received	As Expected	Pass
TC055	Verify prioritization of REQ-020 (clear, no issues)	API server running	Send POST to /api/prioritize	"The system shall support at least 1000 concurrent users without performance degradation."	Must Have, Score: 100, Rating: Excellent	JSON response received	As Expected	Pass
TC056	Verify prioritization of REQ-021 (clear, no issues)	API server running	Send POST to /api/prioritize	"The system shall encrypt all payment and personal data using AES-256 encryption."	Must Have, Score: 100, Rating: Excellent	JSON response received	As Expected	Pass

Continued on next page

Table 4.8 Continued from previous page

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/ Fail
TC057	Verify prioritization of REQ-022 (clear, no issues)	API server running	Send POST to /api/prioritize	"The system shall authenticate all users using JWT tokens with 24-hour expiration."	Must Have, Score: 100, Rating: Excellent	JSON response received	As Expected	Pass
TC058	Verify prioritization of REQ-023 (clear, no issues)	API server running	Send POST to /api/prioritize	"The system shall log all user actions including login, order placement, and payment for audit purposes."	Must Have, Score: 100, Rating: Excellent	JSON response received	As Expected	Pass
TC059	Verify prioritization of REQ-024 (clear, no issues)	API server running	Send POST to /api/prioritize	"The system shall be available 99.9 percent of the time on a monthly basis."	Must Have, Score: 100, Rating: Excellent	JSON response received	As Expected	Pass
TC060	Verify prioritization of REQ-025 (clear, no issues)	API server running	Send POST to /api/prioritize	"The system shall recover from any server failure within 5 minutes without loss of student progress data."	Must Have, Score: 100, Rating: Excellent	JSON response received	As Expected	Pass
TC061	Verify prioritization of REQ-026 (clear, no issues)	UI available	Send POST to /api/prioritize	"The system shall provide a responsive interface accessible on desktop, tablet, and mobile browsers."	Must Have, Score: 100, Rating: Excellent	JSON response received	As Expected	Pass
TC062	Verify prioritization of REQ-027 (clear, no issues)	API server running	Send POST to /api/prioritize	"The system shall allow new students to enroll in their first course within 5 minutes without external help."	Must Have, Score: 100, Rating: Excellent	JSON response received	As Expected	Pass
TC063	Verify prioritization of REQ-028 (clear, no issues)	System deployed	Send POST to /api/prioritize	"The system shall run on Chrome, Firefox, Safari, and Edge browsers without functional differences."	Must Have, Score: 100, Rating: Excellent	JSON response received	As Expected	Pass
TC064	Verify prioritization of REQ-029 (clear, no issues)	API server running	Send POST to /api/prioritize	"The system shall integrate with third-party video streaming services including YouTube and Vimeo."	Must Have, Score: 100, Rating: Excellent	JSON response received	As Expected	Pass

Continued on next page

Table 4.8 Continued from previous page

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/ Fail
TC065	Verify prioritization of REQ-030 (clear, no issues)	Deployment setup ready	Send POST to /api/prioritize	"The system shall be deployable on AWS, Azure, and Google Cloud without code changes."	Must Have, Score: 100, Rating: Excellent	JSON response received	As Expected	Pass
TC066	Verify prioritization of REQ-032 (clear, no issues)	API available	Send POST to /api/prioritize	"The system shall provide API documentation for all endpoints to support future third-party integrations."	Must Have, Score: 100, Rating: Excellent	JSON response received	As Expected	Pass
TC067	Verify prioritization of REQ-038 (clear, no issues)	System active	Send POST to /api/prioritize	"The system shall allow customers to add meals to their basket before placing an order."	Must Have, Score: 100, Rating: Excellent	JSON response received	As Expected	Pass
TC068	Verify prioritization of REQ-039 (clear, no issues)	System active	Send POST to /api/prioritize	"The system shall allow users to put food items into cart prior to checkout."	Must Have, Score: 100, Rating: Excellent	JSON response received	As Expected	Pass
TC069	Verify prioritization of REQ-040 (clear, no issues)	System active	Send POST to /api/prioritize	"The system shall allow registered users to view their past food orders from their profile."	Must Have, Score: 100, Rating: Excellent	JSON response received	As Expected	Pass
TC070	Verify prioritization of REQ-041 (clear, no issues)	System active	Send POST to /api/prioritize	"The system shall allow users to access previous order history from the account section."	Must Have, Score: 100, Rating: Excellent	JSON response received	As Expected	Pass
TC071	Verify prioritization of REQ-042 (clear, no issues)	System active	Send POST to /api/prioritize	"The system shall allow guest users to place food orders without creating an account."	Must Have, Score: 100, Rating: Excellent	JSON response received	As Expected	Pass
TC072	Verify prioritization of REQ-044 (clear, no issues)	System active	Send POST to /api/prioritize	"The system shall automatically assign the nearest available delivery agent to every new order."	Must Have, Score: 100, Rating: Excellent	JSON response received	As Expected	Pass

Continued on next page

Table 4.8 Continued from previous page

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/ Fail
TC073	Verify prioritization of REQ-045 (clear, no issues)	System active	Send POST to /api/prioritize	"The system shall allow restaurant owners to choose and assign their own preferred delivery agents."	Must Have, Score: 100, Rating: Excellent	JSON response received	As Expected	Pass
TC074	Verify prioritization of REQ-046 (clear, no issues)	Load test environment	Send POST to /api/prioritize	"The system shall support a maximum of 300 concurrent users at peak load."	Must Have, Score: 100, Rating: Excellent	JSON response received	As Expected	Pass
TC075	Verify prioritization of REQ-047 (clear, no issues)	UI check	Send POST to /api/prioritize	"The system shall display all menu prices in USD only across all restaurant pages."	Must Have, Score: 100, Rating: Excellent	JSON response received	As Expected	Pass
TC076	Verify prioritization of REQ-048 (clear, no issues)	Location service active	Send POST to /api/prioritize	"The system shall display menu prices in the local currency of the user based on their detected location."	Must Have, Score: 100, Rating: Excellent	JSON response received	As Expected	Pass
TC077	Verify prioritization of REQ-049 (clear, no issues)	Order system active	Send POST to /api/prioritize	"The system shall cancel unaccepted orders automatically after 5 minutes of no restaurant response."	Must Have, Score: 100, Rating: Excellent	JSON response received	As Expected	Pass
TC078	Verify prioritization of REQ-050 (clear, no issues)	Order system active	Send POST to /api/prioritize	"The system shall wait up to 15 minutes before cancelling orders that have not been accepted by the restaurant."	Must Have, Score: 100, Rating: Excellent	JSON response received	As Expected	Pass

Table 4.9: Unit Testing Results for Change Impact Analyzer

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/ Fail
TC001	Verify score for R4	Metrics calculated	Run calculation	ID=8, BC=0.03, CC=0.8	1.00	Score stored	1.00	Pass
TC002	Verify score for R1	Metrics calculated	Run calculation	ID=4, BC=0, CC=0	0.453	Score stored	0.453	Pass
TC003	Verify score for R19	Metrics calculated	Run calculation	ID=3, BC=0.006, CC=0.5	0.388	Score stored	0.388	Pass
TC004	Verify score for R15	Metrics calculated	Run calculation	ID=3, BC=0.003, CC=0.4	0.376	Score stored	0.376	Pass

Continued on next page

Table 4.9 Continued from previous page

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/ Fail
TC005	Verify score for R2	Metrics calculated	Run calculation	ID=2, BC=0.01, CC=1	0.32	Score stored	0.32	Pass
TC006	Verify score for R10	Metrics calculated	Run calculation	ID=2, BC=0.006, CC=0.5	0.274	Score stored	0.274	Pass
TC007	Verify score for R20	Metrics calculated	Run calculation	ID=2, BC=0, CC=0	0.227	Score stored	0.227	Pass
TC008	Verify score for R17	Metrics calculated	Run calculation	ID=1, BC=0.001, CC=1	0.199	Score stored	0.199	Pass
TC009	Verify score for R5	Metrics calculated	Run calculation	ID=1, BC=0.003, CC=0.5	0.158	Score stored	0.158	Pass
TC010	Verify score for R12	Metrics calculated	Run calculation	ID=1, BC=0.003, CC=0.5	0.158	Score stored	0.158	Pass
TC011	Verify score for R11	Metrics calculated	Run calculation	ID=1, BC=0, CC=0	0.113	Score stored	0.113	Pass
TC012	Verify score for R23	Metrics calculated	Run calculation	ID=1, BC=0, CC=0	0.113	Score stored	0.113	Pass
TC013	Verify score for R42	Metrics calculated	Run calculation	ID=1, BC=0, CC=0	0.113	Score stored	0.113	Pass
TC014	Verify score for R7	Metrics calculated	Run calculation	ID=0, BC=0, CC=1	0.085	Score stored	0.085	Pass
TC015	Verify score for R16	Metrics calculated	Run calculation	ID=0, BC=0, CC=1	0.085	Score stored	0.085	Pass
TC016	Verify score for R21	Metrics calculated	Run calculation	ID=0, BC=0, CC=1	0.085	Score stored	0.085	Pass
TC017	Verify score for R22	Metrics calculated	Run calculation	ID=0, BC=0, CC=1	0.085	Score stored	0.085	Pass
TC018	Verify score for R34	Metrics calculated	Run calculation	ID=0, BC=0, CC=1	0.085	Score stored	0.085	Pass
TC019	Verify score for R35	Metrics calculated	Run calculation	ID=0, BC=0, CC=1	0.085	Score stored	0.085	Pass
TC020	Verify score for R43	Metrics calculated	Run calculation	ID=0, BC=0, CC=1	0.085	Score stored	0.085	Pass
TC021	Verify score for R3	Metrics calculated	Run calculation	ID=0, BC=0, CC=0.667	0.057	Score stored	0.057	Pass
TC022	Verify score for R41	Metrics calculated	Run calculation	ID=0, BC=0, CC=0.667	0.057	Score stored	0.057	Pass
TC023	Verify score for R18	Metrics calculated	Run calculation	ID=0, BC=0, CC=0.556	0.047	Score stored	0.047	Pass
TC024	Verify score for R8	Metrics calculated	Run calculation	ID=0, BC=0, CC=0.5	0.042	Score stored	0.042	Pass
TC025	Verify score for R37	Metrics calculated	Run calculation	ID=0, BC=0, CC=0.5	0.042	Score stored	0.042	Pass
TC026	Verify score for R38	Metrics calculated	Run calculation	ID=0, BC=0, CC=0.5	0.042	Score stored	0.042	Pass
TC027	Verify score for R28	Metrics calculated	Run calculation	ID=0, BC=0, CC=0.385	0.033	Score stored	0.033	Pass

Continued on next page

Table 4.9 Continued from previous page

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/ Fail
TC028	Verify score for R6	Metrics calculated	Run calculation	ID=0, BC=0, CC=0.375	0.032	Score stored	0.032	Pass
TC029	Verify score for R36	Metrics calculated	Run calculation	ID=0, BC=0, CC=0.375	0.032	Score stored	0.032	Pass
TC030	Verify score for R39	Metrics calculated	Run calculation	ID=0, BC=0, CC=0.375	0.032	Score stored	0.032	Pass
TC031	Verify score for R40	Metrics calculated	Run calculation	ID=0, BC=0, CC=0.375	0.032	Score stored	0.032	Pass
TC032	Verify score for R25	Metrics calculated	Run calculation	ID=0, BC=0, CC=0.333	0.028	Score stored	0.028	Pass
TC033	Verify score for R9	Metrics calculated	Run calculation	ID=0, BC=0, CC=0	0	Score stored	0	Pass
TC034	Verify score for R13	Metrics calculated	Run calculation	ID=0, BC=0, CC=0	0	Score stored	0	Pass
TC035	Verify score for R14	Metrics calculated	Run calculation	ID=0, BC=0, CC=0	0	Score stored	0	Pass
TC036	Verify score for R24	Metrics calculated	Run calculation	ID=0, BC=0, CC=0	0	Score stored	0	Pass
TC037	Verify score for R26	Metrics calculated	Run calculation	ID=0, BC=0, CC=0	0	Score stored	0	Pass
TC038	Verify score for R27	Metrics calculated	Run calculation	ID=0, BC=0, CC=0	0	Score stored	0	Pass
TC039	Verify score for R29	Metrics calculated	Run calculation	ID=0, BC=0, CC=0	0	Score stored	0	Pass
TC040	Verify score for R30	Metrics calculated	Run calculation	ID=0, BC=0, CC=0	0	Score stored	0	Pass
TC041	Verify score for R31	Metrics calculated	Run calculation	ID=0, BC=0, CC=0	0	Score stored	0	Pass
TC042	Verify score for R32	Metrics calculated	Run calculation	ID=0, BC=0, CC=0	0	Score stored	0	Pass
TC043	Verify score for R33	Metrics calculated	Run calculation	ID=0, BC=0, CC=0	0	Score stored	0	Pass
TC044	Validate impact score computation for R1	Graph generated (50 nodes)	Run Reqify analysis pipeline	R1: system registration requirement	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC045	Validate impact score computation for R2	Graph generated (50 nodes)	Run Reqify analysis pipeline	R2: user tracking requirement	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC046	Validate impact score computation for R3	Graph generated (50 nodes)	Run Reqify analysis pipeline	R3: user registration requirement	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC047	Validate impact score computation for R4	Graph generated (50 nodes)	Run Reqify analysis pipeline	R4: system availability requirement	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC048	Validate impact score computation for R5	Graph generated (50 nodes)	Run Reqify analysis pipeline	R5: core registration logic requirement	Centrality metrics computed	Scores stored in report	As Expected	Pass

Continued on next page

Table 4.9 Continued from previous page

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/ Fail
TC049	Validate impact score computation for R6	Graph generated (50 nodes)	Run Reqify analysis pipeline	R6: authentication/registration module	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC050	Validate impact score computation for R7	Graph generated (50 nodes)	Run Reqify analysis pipeline	R7: registration validation logic	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC051	Validate impact score computation for R8	Graph generated (50 nodes)	Run Reqify analysis pipeline	R8: registration constraint handling	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC052	Validate impact score computation for R9	Graph generated (50 nodes)	Run Reqify analysis pipeline	R9: registration consistency rules	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC053	Validate impact score computation for R10	Graph generated (50 nodes)	Run Reqify analysis pipeline	R10: registration workflow step	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC054	Validate impact score computation for R11	Graph generated (50 nodes)	Run Reqify analysis pipeline	R11: registration dependency rule	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC055	Validate impact score computation for R12	Graph generated (50 nodes)	Run Reqify analysis pipeline	R12: multi-step registration flow	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC056	Validate impact score computation for R13	Graph generated (50 nodes)	Run Reqify analysis pipeline	R13: restaurant module base rule	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC057	Validate impact score computation for R14	Graph generated (50 nodes)	Run Reqify analysis pipeline	R14: restaurant interaction logic	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC058	Validate impact score computation for R15	Graph generated (50 nodes)	Run Reqify analysis pipeline	R15: delivery tracking logic	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC059	Validate impact score computation for R16	Graph generated (50 nodes)	Run Reqify analysis pipeline	R16: admin control module	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC060	Validate impact score computation for R17	Graph generated (50 nodes)	Run Reqify analysis pipeline	R17: registration extended flow	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC061	Validate impact score computation for R18	Graph generated (50 nodes)	Run Reqify analysis pipeline	R18: registration validation rule set	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC062	Validate impact score computation for R19	Graph generated (50 nodes)	Run Reqify analysis pipeline	R19: restaurant loading module	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC063	Validate impact score computation for R20	Graph generated (50 nodes)	Run Reqify analysis pipeline	R20: advanced system support module	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC064	Validate impact score computation for R21	Graph generated (50 nodes)	Run Reqify analysis pipeline	R21: encryption module dependency	Centrality metrics computed	Scores stored in report	As Expected	Pass

Continued on next page

Table 4.9 Continued from previous page

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/ Fail
TC065	Validate impact score computation for R22	Graph generated (50 nodes)	Run Reqify analysis pipeline	R22: authentication system core	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC066	Validate impact score computation for R23	Graph generated (50 nodes)	Run Reqify analysis pipeline	R23: logging subsystem	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC067	Validate impact score computation for R24	Graph generated (50 nodes)	Run Reqify analysis pipeline	R24: system availability constraint	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC068	Validate impact score computation for R25	Graph generated (50 nodes)	Run Reqify analysis pipeline	R25: recovery module	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC069	Validate impact score computation for R26	Graph generated (50 nodes)	Run Reqify analysis pipeline	R26: reporting subsystem	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC070	Validate impact score computation for R27	Graph generated (50 nodes)	Run Reqify analysis pipeline	R27: new user module	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC071	Validate impact score computation for R28	Graph generated (50 nodes)	Run Reqify analysis pipeline	R28: browser compatibility layer	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC072	Validate impact score computation for R29	Graph generated (50 nodes)	Run Reqify analysis pipeline	R29: integration module	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC073	Validate impact score computation for R30	Graph generated (50 nodes)	Run Reqify analysis pipeline	R30: deployment pipeline	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC074	Validate impact score computation for R31	Graph generated (50 nodes)	Run Reqify analysis pipeline	R31: modular architecture rule	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC075	Validate impact score computation for R32	Graph generated (50 nodes)	Run Reqify analysis pipeline	R32: API service module	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC076	Validate impact score computation for R33	Graph generated (50 nodes)	Run Reqify analysis pipeline	R33: delivery service logic	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC077	Validate impact score computation for R34	Graph generated (50 nodes)	Run Reqify analysis pipeline	R34: scalability handling logic	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC078	Validate impact score computation for R35	Graph generated (50 nodes)	Run Reqify analysis pipeline	R35: security service module	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC079	Validate impact score computation for R36	Graph generated (50 nodes)	Run Reqify analysis pipeline	R36: delivery assurance logic	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC080	Validate impact score computation for R37	Graph generated (50 nodes)	Run Reqify analysis pipeline	R37: payment processing module	Centrality metrics computed	Scores stored in report	As Expected	Pass

Continued on next page

Table 4.9 Continued from previous page

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/ Fail
TC081	Validate impact score computation for R38	Graph generated (50 nodes)	Run Reqify analysis pipeline	R38: customer service module	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC082	Validate impact score computation for R39	Graph generated (50 nodes)	Run Reqify analysis pipeline	R39: user support module	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC083	Validate impact score computation for R40	Graph generated (50 nodes)	Run Reqify analysis pipeline	R40: registration optimization logic	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC084	Validate impact score computation for R41	Graph generated (50 nodes)	Run Reqify analysis pipeline	R41: user interaction optimization	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC085	Validate impact score computation for R42	Graph generated (50 nodes)	Run Reqify analysis pipeline	R42: guest access module	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC086	Validate impact score computation for R43	Graph generated (50 nodes)	Run Reqify analysis pipeline	R43: input validation rules	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC087	Validate impact score computation for R44	Graph generated (50 nodes)	Run Reqify analysis pipeline	R44: automation module	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC088	Validate impact score computation for R45	Graph generated (50 nodes)	Run Reqify analysis pipeline	R45: restaurant service flow	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC089	Validate impact score computation for R46	Graph generated (50 nodes)	Run Reqify analysis pipeline	R46: recommendation engine module	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC090	Validate impact score computation for R47	Graph generated (50 nodes)	Run Reqify analysis pipeline	R47: display subsystem	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC091	Validate impact score computation for R48	Graph generated (50 nodes)	Run Reqify analysis pipeline	R48: menu display module	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC092	Validate impact score computation for R49	Graph generated (50 nodes)	Run Reqify analysis pipeline	R49: cancellation handling module	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC093	Validate impact score computation for R50	Graph generated (50 nodes)	Run Reqify analysis pipeline	R50: timeout handling module	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC094	Validate impact score computation for R1	Graph generated (50 nodes)	Run Reqify analysis pipeline	R1: system registration requirement	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC095	Validate impact score computation for R2	Graph generated (50 nodes)	Run Reqify analysis pipeline	R2: user tracking requirement	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC096	Validate impact score computation for R3	Graph generated (50 nodes)	Run Reqify analysis pipeline	R3: user registration requirement	Centrality metrics computed	Scores stored in report	As Expected	Pass

Continued on next page

Table 4.9 Continued from previous page

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/ Fail
TC097	Validate impact score computation for R4	Graph generated (50 nodes)	Run Reqify analysis pipeline	R4: system availability requirement	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC098	Validate impact score computation for R5	Graph generated (50 nodes)	Run Reqify analysis pipeline	R5: core registration logic requirement	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC099	Validate impact score computation for R6	Graph generated (50 nodes)	Run Reqify analysis pipeline	R6: authentication/registration module	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC100	Validate impact score computation for R7	Graph generated (50 nodes)	Run Reqify analysis pipeline	R7: registration validation logic	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC101	Validate impact score computation for R8	Graph generated (50 nodes)	Run Reqify analysis pipeline	R8: registration constraint handling	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC102	Validate impact score computation for R9	Graph generated (50 nodes)	Run Reqify analysis pipeline	R9: registration consistency rules	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC103	Validate impact score computation for R10	Graph generated (50 nodes)	Run Reqify analysis pipeline	R10: registration workflow step	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC104	Validate impact score computation for R11	Graph generated (50 nodes)	Run Reqify analysis pipeline	R11: registration dependency rule	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC105	Validate impact score computation for R12	Graph generated (50 nodes)	Run Reqify analysis pipeline	R12: multi-step registration flow	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC106	Validate impact score computation for R13	Graph generated (50 nodes)	Run Reqify analysis pipeline	R13: restaurant module base rule	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC107	Validate impact score computation for R14	Graph generated (50 nodes)	Run Reqify analysis pipeline	R14: restaurant interaction logic	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC108	Validate impact score computation for R15	Graph generated (50 nodes)	Run Reqify analysis pipeline	R15: delivery tracking logic	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC109	Validate impact score computation for R16	Graph generated (50 nodes)	Run Reqify analysis pipeline	R16: admin control module	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC110	Validate impact score computation for R17	Graph generated (50 nodes)	Run Reqify analysis pipeline	R17: registration extended flow	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC111	Validate impact score computation for R18	Graph generated (50 nodes)	Run Reqify analysis pipeline	R18: registration validation rule set	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC112	Validate impact score computation for R19	Graph generated (50 nodes)	Run Reqify analysis pipeline	R19: restaurant loading module	Centrality metrics computed	Scores stored in report	As Expected	Pass

Continued on next page

Table 4.9 Continued from previous page

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/ Fail
TC113	Validate impact score computation for R20	Graph generated (50 nodes)	Run Reqify analysis pipeline	R20: advanced system support module	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC114	Validate impact score computation for R21	Graph generated (50 nodes)	Run Reqify analysis pipeline	R21: encryption module dependency	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC115	Validate impact score computation for R22	Graph generated (50 nodes)	Run Reqify analysis pipeline	R22: authentication system core	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC116	Validate impact score computation for R23	Graph generated (50 nodes)	Run Reqify analysis pipeline	R23: logging subsystem	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC117	Validate impact score computation for R24	Graph generated (50 nodes)	Run Reqify analysis pipeline	R24: system availability constraint	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC118	Validate impact score computation for R25	Graph generated (50 nodes)	Run Reqify analysis pipeline	R25: recovery module	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC119	Validate impact score computation for R26	Graph generated (50 nodes)	Run Reqify analysis pipeline	R26: reporting subsystem	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC120	Validate impact score computation for R27	Graph generated (50 nodes)	Run Reqify analysis pipeline	R27: new user module	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC121	Validate impact score computation for R28	Graph generated (50 nodes)	Run Reqify analysis pipeline	R28: browser compatibility layer	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC122	Validate impact score computation for R29	Graph generated (50 nodes)	Run Reqify analysis pipeline	R29: integration module	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC123	Validate impact score computation for R30	Graph generated (50 nodes)	Run Reqify analysis pipeline	R30: deployment pipeline	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC124	Validate impact score computation for R31	Graph generated (50 nodes)	Run Reqify analysis pipeline	R31: modular architecture rule	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC125	Validate impact score computation for R32	Graph generated (50 nodes)	Run Reqify analysis pipeline	R32: API service module	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC126	Validate impact score computation for R33	Graph generated (50 nodes)	Run Reqify analysis pipeline	R33: delivery service logic	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC127	Validate impact score computation for R34	Graph generated (50 nodes)	Run Reqify analysis pipeline	R34: scalability handling logic	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC128	Validate impact score computation for R35	Graph generated (50 nodes)	Run Reqify analysis pipeline	R35: security service module	Centrality metrics computed	Scores stored in report	As Expected	Pass

Continued on next page

Table 4.9 Continued from previous page

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/ Fail
TC129	Validate impact score computation for R36	Graph generated (50 nodes)	Run Reqify analysis pipeline	R36: delivery assurance logic	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC130	Validate impact score computation for R37	Graph generated (50 nodes)	Run Reqify analysis pipeline	R37: payment processing module	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC131	Validate impact score computation for R38	Graph generated (50 nodes)	Run Reqify analysis pipeline	R38: customer service module	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC132	Validate impact score computation for R39	Graph generated (50 nodes)	Run Reqify analysis pipeline	R39: user support module	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC133	Validate impact score computation for R40	Graph generated (50 nodes)	Run Reqify analysis pipeline	R40: registration optimization logic	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC134	Validate impact score computation for R41	Graph generated (50 nodes)	Run Reqify analysis pipeline	R41: user interaction optimization	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC135	Validate impact score computation for R42	Graph generated (50 nodes)	Run Reqify analysis pipeline	R42: guest access module	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC136	Validate impact score computation for R43	Graph generated (50 nodes)	Run Reqify analysis pipeline	R43: input validation rules	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC137	Validate impact score computation for R44	Graph generated (50 nodes)	Run Reqify analysis pipeline	R44: automation module	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC138	Validate impact score computation for R45	Graph generated (50 nodes)	Run Reqify analysis pipeline	R45: restaurant service flow	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC139	Validate impact score computation for R46	Graph generated (50 nodes)	Run Reqify analysis pipeline	R46: recommendation engine module	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC140	Validate impact score computation for R47	Graph generated (50 nodes)	Run Reqify analysis pipeline	R47: display subsystem	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC141	Validate impact score computation for R48	Graph generated (50 nodes)	Run Reqify analysis pipeline	R48: menu display module	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC142	Validate impact score computation for R49	Graph generated (50 nodes)	Run Reqify analysis pipeline	R49: cancellation handling module	Centrality metrics computed	Scores stored in report	As Expected	Pass
TC143	Validate impact score computation for R50	Graph generated (50 nodes)	Run Reqify analysis pipeline	R50: timeout handling module	Centrality metrics computed	Scores stored in report	As Expected	Pass

Table 4.10: Unit Testing Results for Risk Estimator

Test Case ID	Test Objective	Pre-condition	Steps	Test Data (Dependency, Volatility, Ambiguity)	Expected Result (Risk Index, Risk Level)	Post-condition	Actual Result	Pass/ Fail
TC001	Verify risk score for REQ-30	Metrics calculated	Run calculation	0, 0.257, 0.77	0.398, MEDIUM	Risk stored	0.398, MEDIUM	Pass
TC002	Verify risk score for REQ-29	Metrics calculated	Run calculation	0, 0.247, 0.74	0.382, MEDIUM	Risk stored	0.382, MEDIUM	Pass
TC003	Verify risk score for REQ-32	Metrics calculated	Run calculation	0, 0.247, 0.74	0.382, MEDIUM	Risk stored	0.382, MEDIUM	Pass
TC004	Verify risk score for REQ-31	Metrics calculated	Run calculation	0, 0.227, 0.68	0.351, LOW	Risk stored	0.351, LOW	Pass
TC005	Verify risk score for REQ-4	Metrics calculated	Run calculation	1, 0.197, 0	0.319, LOW	Risk stored	0.319, LOW	Pass
TC006	Verify risk score for REQ-27	Metrics calculated	Run calculation	0, 0.177, 0.53	0.274, LOW	Risk stored	0.274, LOW	Pass
TC007	Verify risk score for REQ-1	Metrics calculated	Run calculation	0.453, 0.085, 0	0.143, LOW	Risk stored	0.143, LOW	Pass
TC008	Verify risk score for REQ-19	Metrics calculated	Run calculation	0.388, 0.073, 0	0.122, LOW	Risk stored	0.122, LOW	Pass
TC009	Verify risk score for REQ-15	Metrics calculated	Run calculation	0.376, 0.073, 0	0.120, LOW	Risk stored	0.120, LOW	Pass
TC010	Verify risk score for REQ-2	Metrics calculated	Run calculation	0.320, 0.052, 0	0.098, LOW	Risk stored	0.098, LOW	Pass
TC011	Verify risk score for REQ-10	Metrics calculated	Run calculation	0.274, 0.052, 0	0.087, LOW	Risk stored	0.087, LOW	Pass
TC012	Verify risk score for REQ-20	Metrics calculated	Run calculation	0.227, 0.042, 0	0.072, LOW	Risk stored	0.072, LOW	Pass
TC013	Verify risk score for REQ-17	Metrics calculated	Run calculation	0.199, 0.030, 0	0.060, LOW	Risk stored	0.060, LOW	Pass
TC014	Verify risk score for REQ-5	Metrics calculated	Run calculation	0.158, 0.030, 0	0.050, LOW	Risk stored	0.050, LOW	Pass
TC015	Verify risk score for REQ-12	Metrics calculated	Run calculation	0.158, 0.030, 0	0.050, LOW	Risk stored	0.050, LOW	Pass
TC016	Verify risk score for REQ-11	Metrics calculated	Run calculation	0.113, 0.021, 0	0.036, LOW	Risk stored	0.036, LOW	Pass
TC017	Verify risk score for REQ-23	Metrics calculated	Run calculation	0.113, 0.021, 0	0.036, LOW	Risk stored	0.036, LOW	Pass
TC018	Verify risk score for REQ-42	Metrics calculated	Run calculation	0.113, 0.021, 0	0.036, LOW	Risk stored	0.036, LOW	Pass
TC019	Verify risk score for REQ-7	Metrics calculated	Run calculation	0.085, 0.009, 0	0.024, LOW	Risk stored	0.024, LOW	Pass
TC020	Verify risk score for REQ-16	Metrics calculated	Run calculation	0.085, 0.009, 0	0.024, LOW	Risk stored	0.024, LOW	Pass
TC021	Verify risk score for REQ-21	Metrics calculated	Run calculation	0.085, 0.009, 0	0.024, LOW	Risk stored	0.024, LOW	Pass
TC022	Verify risk score for REQ-22	Metrics calculated	Run calculation	0.085, 0.009, 0	0.024, LOW	Risk stored	0.024, LOW	Pass
TC023	Verify risk score for REQ-34	Metrics calculated	Run calculation	0.085, 0.009, 0	0.024, LOW	Risk stored	0.024, LOW	Pass

Continued on next page

Table 4.10 Continued from previous page

Test Case ID	Test Objective	Pre-condition	Steps	Test Data (Dependency, Volatility, Ambiguity)	Expected Result (Risk Index, Risk Level)	Post-condition	Actual Result	Pass/ Fail
TC024	Verify risk score for REQ-35	Metrics calculated	Run calculation	0.085, 0.009, 0	0.024, LOW	Risk stored	0.024, LOW	Pass
TC025	Verify risk score for REQ-43	Metrics calculated	Run calculation	0.085, 0.009, 0	0.024, LOW	Risk stored	0.024, LOW	Pass
TC026	Verify risk score for REQ-18	Metrics calculated	Run calculation	0.047, 0.018, 0	0.018, LOW	Risk stored	0.018, LOW	Pass
TC027	Verify risk score for REQ-3	Metrics calculated	Run calculation	0.057, 0.009, 0	0.017, LOW	Risk stored	0.017, LOW	Pass
TC028	Verify risk score for REQ-41	Metrics calculated	Run calculation	0.057, 0.009, 0	0.017, LOW	Risk stored	0.017, LOW	Pass
TC029	Verify risk score for REQ-8	Metrics calculated	Run calculation	0.042, 0.009, 0	0.014, LOW	Risk stored	0.014, LOW	Pass
TC030	Verify risk score for REQ-37	Metrics calculated	Run calculation	0.042, 0.009, 0	0.014, LOW	Risk stored	0.014, LOW	Pass
TC031	Verify risk score for REQ-38	Metrics calculated	Run calculation	0.042, 0.009, 0	0.014, LOW	Risk stored	0.014, LOW	Pass
TC032	Verify risk score for REQ-28	Metrics calculated	Run calculation	0.033, 0.009, 0	0.011, LOW	Risk stored	0.011, LOW	Pass
TC033	Verify risk score for REQ-6	Metrics calculated	Run calculation	0.032, 0.009, 0	0.011, LOW	Risk stored	0.011, LOW	Pass
TC034	Verify risk score for REQ-36	Metrics calculated	Run calculation	0.032, 0.009, 0	0.011, LOW	Risk stored	0.011, LOW	Pass
TC035	Verify risk score for REQ-39	Metrics calculated	Run calculation	0.032, 0.009, 0	0.011, LOW	Risk stored	0.011, LOW	Pass
TC036	Verify risk score for REQ-40	Metrics calculated	Run calculation	0.032, 0.009, 0	0.011, LOW	Risk stored	0.011, LOW	Pass
TC037	Verify risk score for REQ-25	Metrics calculated	Run calculation	0.028, 0.009, 0	0.010, LOW	Risk stored	0.010, LOW	Pass
TC038	Verify risk score for REQ-9	Metrics calculated	Run calculation	0, 0, 0	0, LOW	Risk stored	0, LOW	Pass
TC039	Verify risk score for REQ-13	Metrics calculated	Run calculation	0, 0, 0	0, LOW	Risk stored	0, LOW	Pass
TC040	Verify risk score for REQ-36	Metrics calculated	Run calculation	0, 0.78, 0.26	0.403, MEDIUM	Risk stored	0.403, MEDIUM	Pass
TC041	Verify risk score for REQ-34	Metrics calculated	Run calculation	0, 0.77, 0.257	0.398, MEDIUM	Risk stored	0.398, MEDIUM	Pass
TC042	Verify risk score for REQ-9	Metrics calculated	Run calculation	0.152, 0.66, 0.263	0.394, MEDIUM	Risk stored	0.394, MEDIUM	Pass
TC043	Verify risk score for REQ-33	Metrics calculated	Run calculation	0, 0.76, 0.253	0.393, MEDIUM	Risk stored	0.393, MEDIUM	Pass
TC044	Verify risk score for REQ-35	Metrics calculated	Run calculation	0, 0.73, 0.243	0.377, MEDIUM	Risk stored	0.377, MEDIUM	Pass
TC045	Verify risk score for REQ-37	Metrics calculated	Run calculation	0, 0.69, 0.23	0.356, LOW	Risk stored	0.356, LOW	Pass
TC046	Verify risk score for REQ-5	Metrics calculated	Run calculation	1, 0, 0.207	0.322, LOW	Risk stored	0.322, LOW	Pass

Continued on next page

Table 4.10 Continued from previous page

Test Case ID	Test Objective	Pre-condition	Steps	Test Data (Dependency, Volatility, Ambiguity)	Expected Result (Risk Index, Risk Level)	Post-condition	Actual Result	Pass/ Fail
TC047	Verify risk score for REQ-4	Metrics calculated	Run calculation	0.773, 0, 0.16	0.249, LOW	Risk stored	0.249, LOW	Pass
TC048	Verify risk score for REQ-3	Metrics calculated	Run calculation	0.549, 0, 0.103	0.174, LOW	Risk stored	0.174, LOW	Pass
TC049	Verify risk score for REQ-13	Metrics calculated	Run calculation	0.51, 0, 0.123	0.171, LOW	Risk stored	0.171, LOW	Pass
TC050	Verify risk score for REQ-7	Metrics calculated	Run calculation	0.435, 0, 0.08	0.137, LOW	Risk stored	0.137, LOW	Pass
TC051	Verify risk score for REQ-22	Metrics calculated	Run calculation	0.389, 0, 0.08	0.125, LOW	Risk stored	0.125, LOW	Pass
TC052	Verify risk score for REQ-1	Metrics calculated	Run calculation	0.345, 0, 0.07	0.111, LOW	Risk stored	0.111, LOW	Pass
TC053	Verify risk score for REQ-2	Metrics calculated	Run calculation	0.319, 0, 0.057	0.099, LOW	Risk stored	0.099, LOW	Pass
TC054	Verify risk score for REQ-8	Metrics calculated	Run calculation	0.284, 0, 0.067	0.094, LOW	Risk stored	0.094, LOW	Pass
TC055	Verify risk score for REQ-15	Metrics calculated	Run calculation	0.28, 0, 0.067	0.093, LOW	Risk stored	0.093, LOW	Pass
TC056	Verify risk score for REQ-10	Metrics calculated	Run calculation	0.272, 0, 0.067	0.091, LOW	Risk stored	0.091, LOW	Pass
TC057	Verify risk score for REQ-6	Metrics calculated	Run calculation	0.27, 0, 0.057	0.087, LOW	Risk stored	0.087, LOW	Pass
TC058	Verify risk score for REQ-24	Metrics calculated	Run calculation	0.23, 0, 0.047	0.074, LOW	Risk stored	0.074, LOW	Pass
TC059	Verify risk score for REQ-20	Metrics calculated	Run calculation	0.201, 0, 0.033	0.062, LOW	Risk stored	0.062, LOW	Pass
TC060	Verify risk score for REQ-27	Metrics calculated	Run calculation	0.115, 0, 0.023	0.037, LOW	Risk stored	0.037, LOW	Pass
TC061	Verify risk score for REQ-32	Metrics calculated	Run calculation	0.115, 0, 0.023	0.037, LOW	Risk stored	0.037, LOW	Pass
TC062	Verify risk score for REQ-47	Metrics calculated	Run calculation	0.115, 0, 0.023	0.037, LOW	Risk stored	0.037, LOW	Pass
TC063	Verify risk score for REQ-25	Metrics calculated	Run calculation	0.086, 0, 0.01	0.025, LOW	Risk stored	0.025, LOW	Pass
TC064	Verify risk score for REQ-26	Metrics calculated	Run calculation	0.086, 0, 0.01	0.025, LOW	Risk stored	0.025, LOW	Pass
TC065	Verify risk score for REQ-31	Metrics calculated	Run calculation	0.086, 0, 0.01	0.025, LOW	Risk stored	0.025, LOW	Pass
TC066	Verify risk score for REQ-48	Metrics calculated	Run calculation	0.086, 0, 0.01	0.025, LOW	Risk stored	0.025, LOW	Pass
TC067	Verify risk score for REQ-12	Metrics calculated	Run calculation	0.049, 0, 0.02	0.019, LOW	Risk stored	0.019, LOW	Pass
TC068	Verify risk score for REQ-17	Metrics calculated	Run calculation	0.057, 0, 0.01	0.018, LOW	Risk stored	0.018, LOW	Pass
TC069	Verify risk score for REQ-38	Metrics calculated	Run calculation	0.057, 0, 0.01	0.018, LOW	Risk stored	0.018, LOW	Pass

Continued on next page

Table 4.10 Continued from previous page

Test Case ID	Test Objective	Pre-condition	Steps	Test Data (Dependency, Volatility, Ambiguity)	Expected Result (Risk Index, Risk Level)	Post-condition	Actual Result	Pass/ Fail
TC070	Verify risk score for REQ-39	Metrics calculated	Run calculation	0.057, 0, 0.01	0.018, LOW	Risk stored	0.018, LOW	Pass
TC071	Verify risk score for REQ-46	Metrics calculated	Run calculation	0.057, 0, 0.01	0.018, LOW	Risk stored	0.018, LOW	Pass
TC072	Verify risk score for REQ-18	Metrics calculated	Run calculation	0.043, 0, 0.02	0.018, LOW	Risk stored	0.018, LOW	Pass
TC073	Verify risk score for REQ-29	Metrics calculated	Run calculation	0.043, 0, 0.02	0.018, LOW	Risk stored	0.018, LOW	Pass
TC074	Verify risk score for REQ-16	Metrics calculated	Run calculation	0.043, 0, 0.01	0.014, LOW	Risk stored	0.014, LOW	Pass
TC075	Verify risk score for REQ-21	Metrics calculated	Run calculation	0.043, 0, 0.01	0.014, LOW	Risk stored	0.014, LOW	Pass
TC076	Verify risk score for REQ-11	Metrics calculated	Run calculation	0.037, 0, 0.01	0.013, LOW	Risk stored	0.013, LOW	Pass
TC077	Verify risk score for REQ-19	Metrics calculated	Run calculation	0.037, 0, 0.01	0.013, LOW	Risk stored	0.013, LOW	Pass
TC078	Verify risk score for REQ-42	Metrics calculated	Run calculation	0.037, 0, 0.01	0.013, LOW	Risk stored	0.013, LOW	Pass
TC079	Verify risk score for REQ-43	Metrics calculated	Run calculation	0.037, 0, 0.01	0.013, LOW	Risk stored	0.013, LOW	Pass
TC080	Verify risk score for REQ-23	Metrics calculated	Run calculation	0.034, 0, 0.01	0.012, LOW	Risk stored	0.012, LOW	Pass
TC081	Verify risk score for REQ-44	Metrics calculated	Run calculation	0.027, 0, 0.01	0.010, LOW	Risk stored	0.010, LOW	Pass
TC082	Verify risk score for REQ-45	Metrics calculated	Run calculation	0.027, 0, 0.01	0.010, LOW	Risk stored	0.010, LOW	Pass
TC083	Verify risk score for REQ-7	Metrics calculated	Run calculation	0.019, 0, 0.01	0.007, LOW	Risk stored	0.007, LOW	Pass
TC084	Verify risk score for REQ-40	Metrics calculated	Run calculation	0.017, 0, 0.01	0.007, LOW	Risk stored	0.007, LOW	Pass
TC085	Verify risk score for REQ-41	Metrics calculated	Run calculation	0.017, 0, 0.01	0.007, LOW	Risk stored	0.007, LOW	Pass
TC086	Verify risk score for REQ-49	Metrics calculated	Run calculation	0.017, 0, 0.01	0.007, LOW	Risk stored	0.007, LOW	Pass
TC087	Verify risk score for REQ-50	Metrics calculated	Run calculation	0.017, 0, 0.01	0.007, LOW	Risk stored	0.007, LOW	Pass
TC088	Verify risk score for REQ-23	Metrics calculated	Run calculation	0.017, 0, 0.01	0.006, LOW	Risk stored	0.006, LOW	Pass
TC089	Verify risk score for REQ-13	Metrics calculated	Run calculation	0, 0, 0	0, LOW	Risk stored	0, LOW	Pass

Chapter 5

Conclusions and Future Work

5.1 Conclusion

5.2 Future Work

Bibliography

[1] Ardhendu Mandal and S Pal. Identifying the reasons for software project failure and some of their proposed remedial through bridge process models. *International Journal of Computer Sciences and Engineering*, 3:118–126, 01 2015.

[2] Sabina-Cristiana Necula, Florin Dumitriu, and Valeric Greavu-erban. A systematic literature review on using natural language processing in software requirements engineering. *Electronics*, 13(11), 2024.

Appendix A

Appendices

A.1 Appendix A

A.1.1 Use Case Diagram example (Online Shopping System)

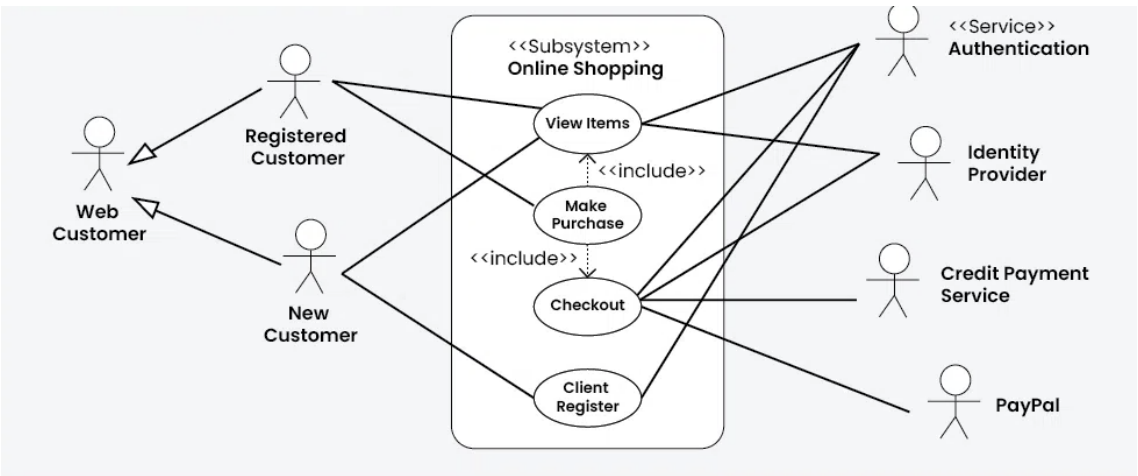

Figure A.1: Use Case Diagram for the Online Shopping System

A.1.2 Detail Use Case Example

ID	#1
Name	Overview elements
Short Description	All elements are shown in a list, where the user can set different filters.
Goal	Displaying elements in a changeable view.
Preconditions	The user got access to the system and is logged in. The user got the right to see schedules.
Success End Condition	The correct elements (filter and sorting) are displayed.
Fall End Condition	Elements not matching the criteria are displayed.
Stakeholder	Customer manager
Trigger	Login in as customer manager (because overview is on the starting screen for this role).
Normal Flow	1. The list of all elements matching default criteria are shown. 2. The user changes the filter criteria. 3. The user presses the Filter button. 4. The list shows all matching elements. Optional: 5. The user presses the Clear button. 6. The list of all elements matching default criteria are shown.
Alternative Flows	With click on the column headers, the list sorting can be changed. Different sorting for the different columns is described in the table below.
Includes	Login
Frequency of Use	About 50 times per day
Constraints and Special Requirements	None
Assumptions	None
Notes and Issues	None

Figure A.2: Detail Use Case Example

A.1.3 Event-Response Table for a Highway Intersection

Event	System State	Response
Road sensor detects vehicle entering left-turn lane.	Left-turn signal is red. Cross-traffic signal is green.	Start green-to-amber countdown timer for cross-traffic signal.
Green-to-amber countdown timer reaches zero.	Cross-traffic signal is green.	1. Turn cross-traffic signal amber. 2. Start amber-to-red countdown timer.
Amber-to-red countdown timer reaches zero.	Cross-traffic signal is amber.	1. Turn cross-traffic signal red. 2. Wait 1 second. 3. Turn left-turn signal green. 4. Start left-turn-signal countdown timer.
Pedestrian presses a specific walk-request button.	Pedestrian sign is solid Don't Walk. Walk-request countdown timer is not activated.	Start walk-request countdown timer.
Pedestrian presses walk-request button.	Pedestrian sign is solid Don't Walk. Walk-request countdown timer is activated.	Do nothing.
Walk-request countdown timer reaches zero plus the amber display time.	Pedestrian sign is solid Don't Walk.	Change all green traffic signals to amber.
Walk-request countdown timer reaches zero.	Pedestrian sign is solid Don't Walk.	1. Change all amber traffic signals to red. 2. Wait 1 second. 3. Set pedestrian sign to Walk. 4. Start don't-walk countdown timer.

Figure A.3: Example of Event Response Table

A.1.4 Story Board Example For Android App

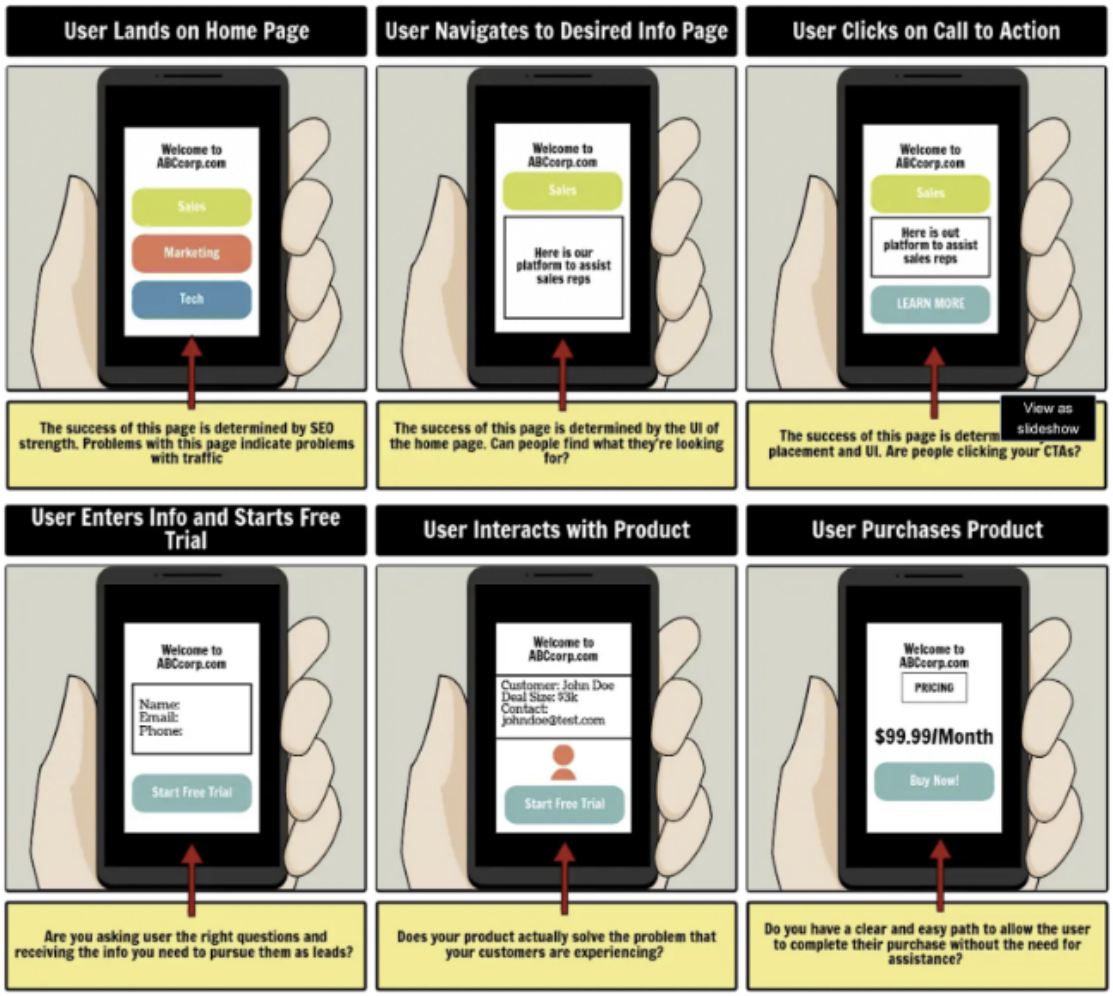

Figure A.4: Example of Story Board

A.2 Appendix B

A.2.1 Domain Model Example For Online Shopping

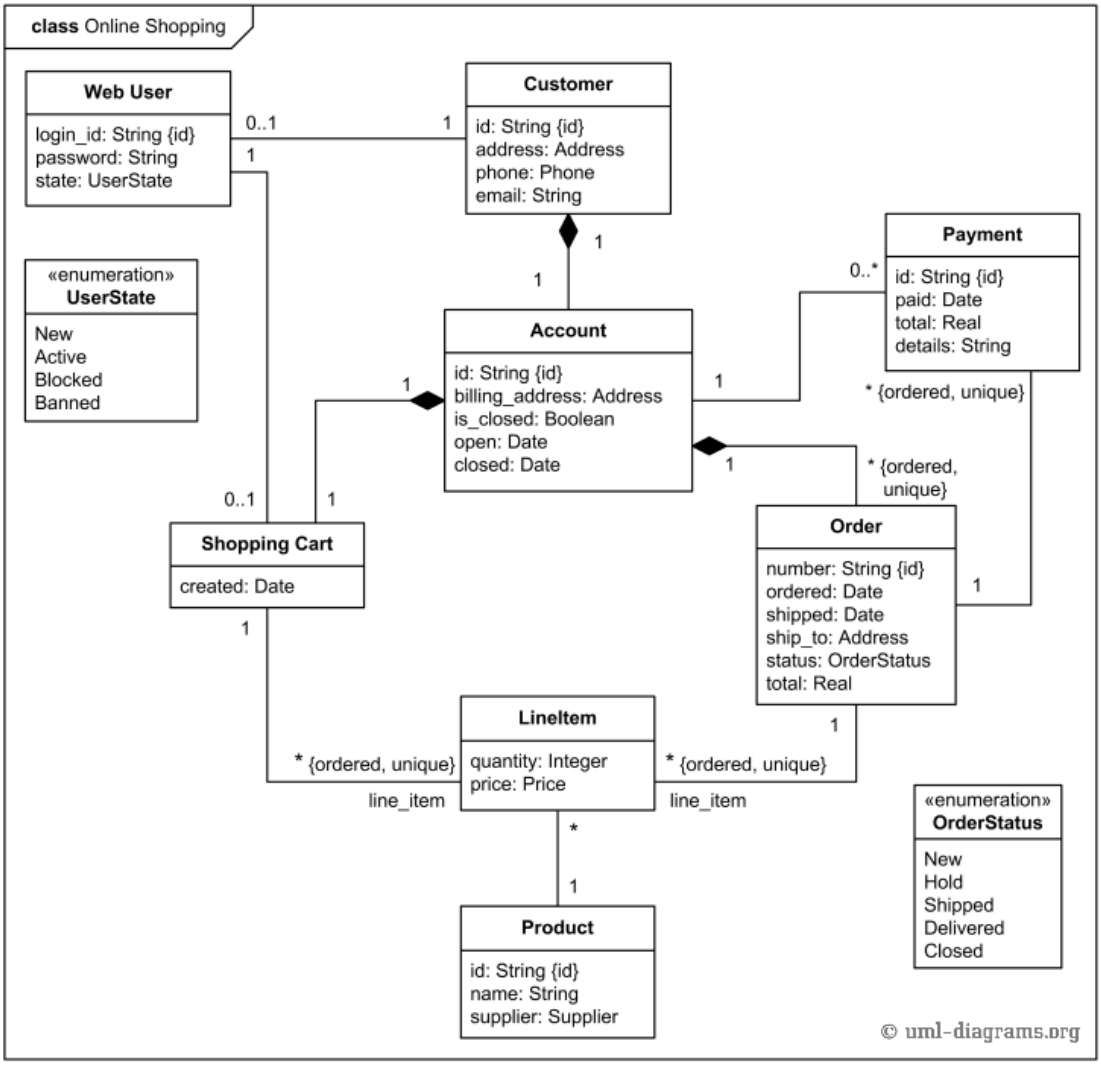

Figure A.5: Domain Model Example For Online Shopping Application

A.3 Appendix C

A.3.1 Box And Line Example For Online Shopping

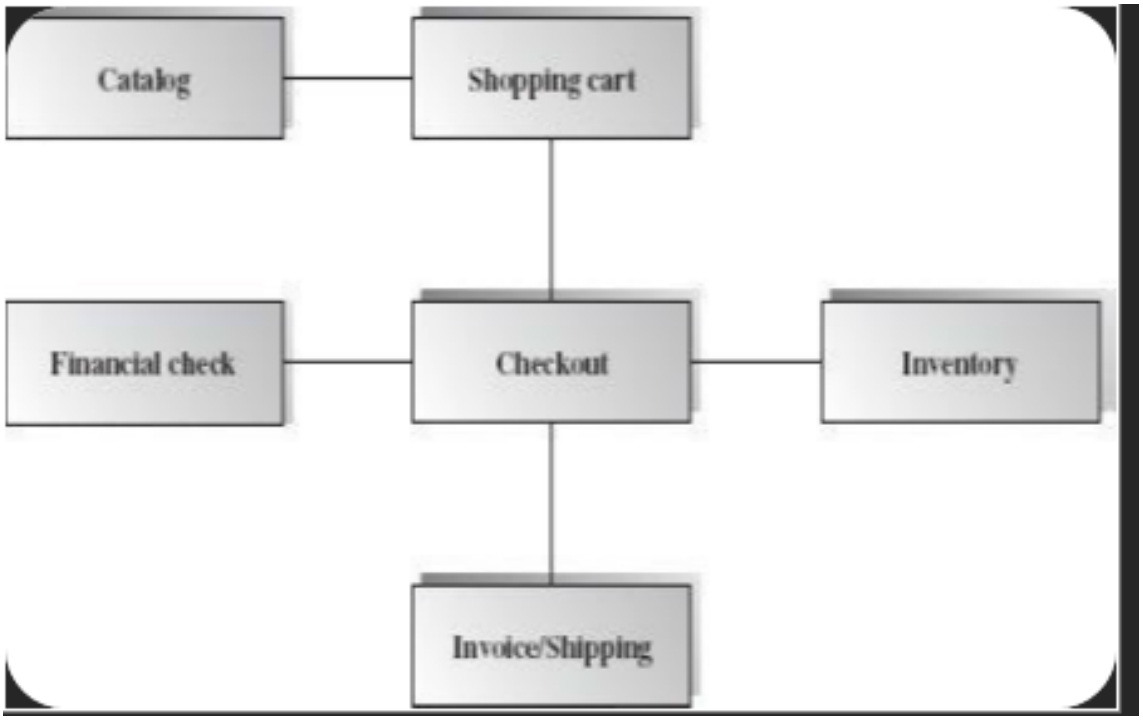

Figure A.6: Box and Line Diagram For Online Shopping Application

A.3.2 Architecture Pattern Example For Online Shopping

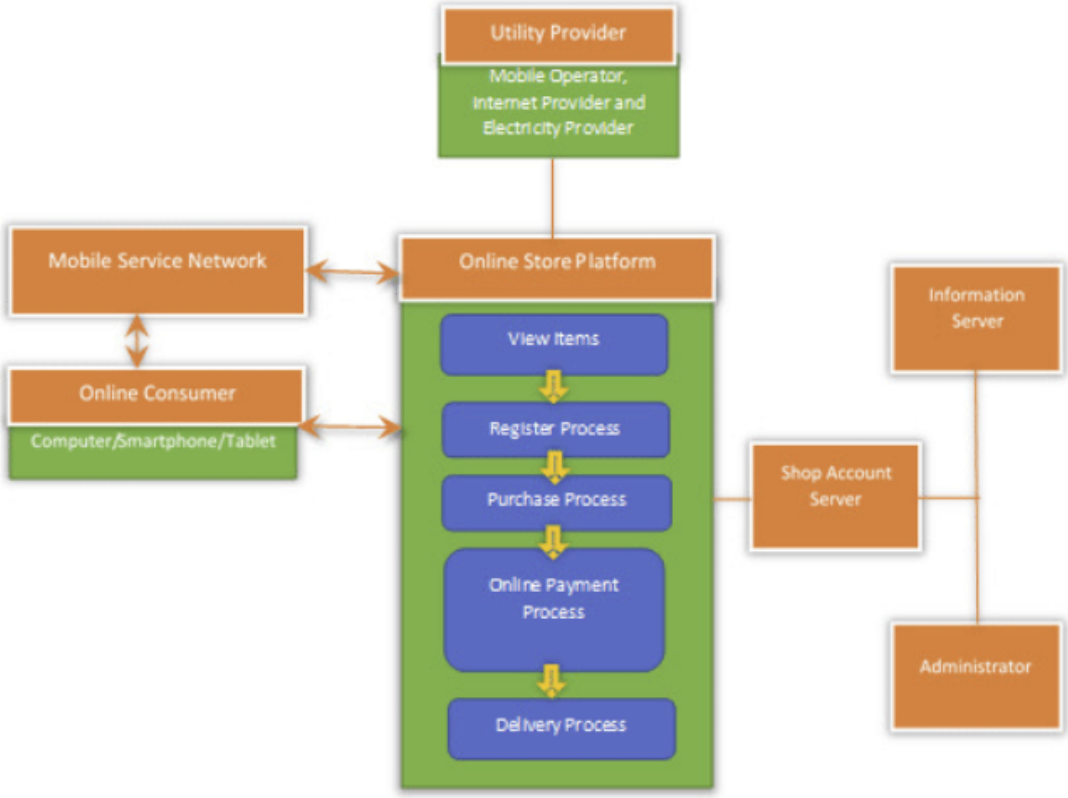

Figure A.7: Architecture Pattern For Online Shopping Application

A.4 Appendix D

A.4.1 Activity Diagram

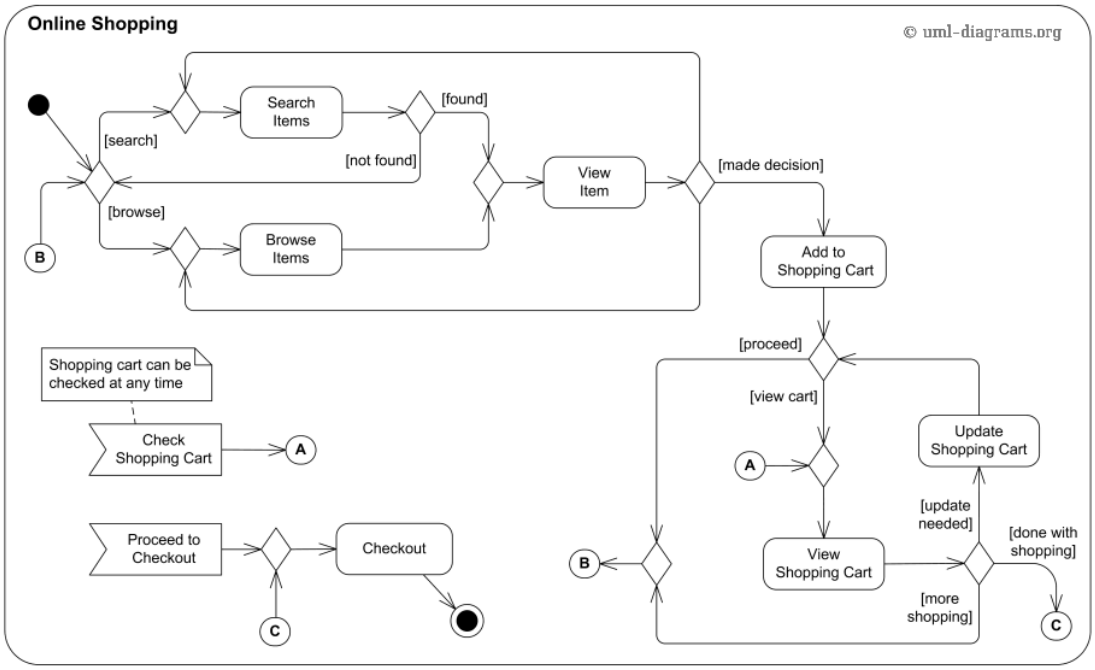

Figure A.8: Activity Diagram For Online Shopping Application

A.4.2 Class Diagram

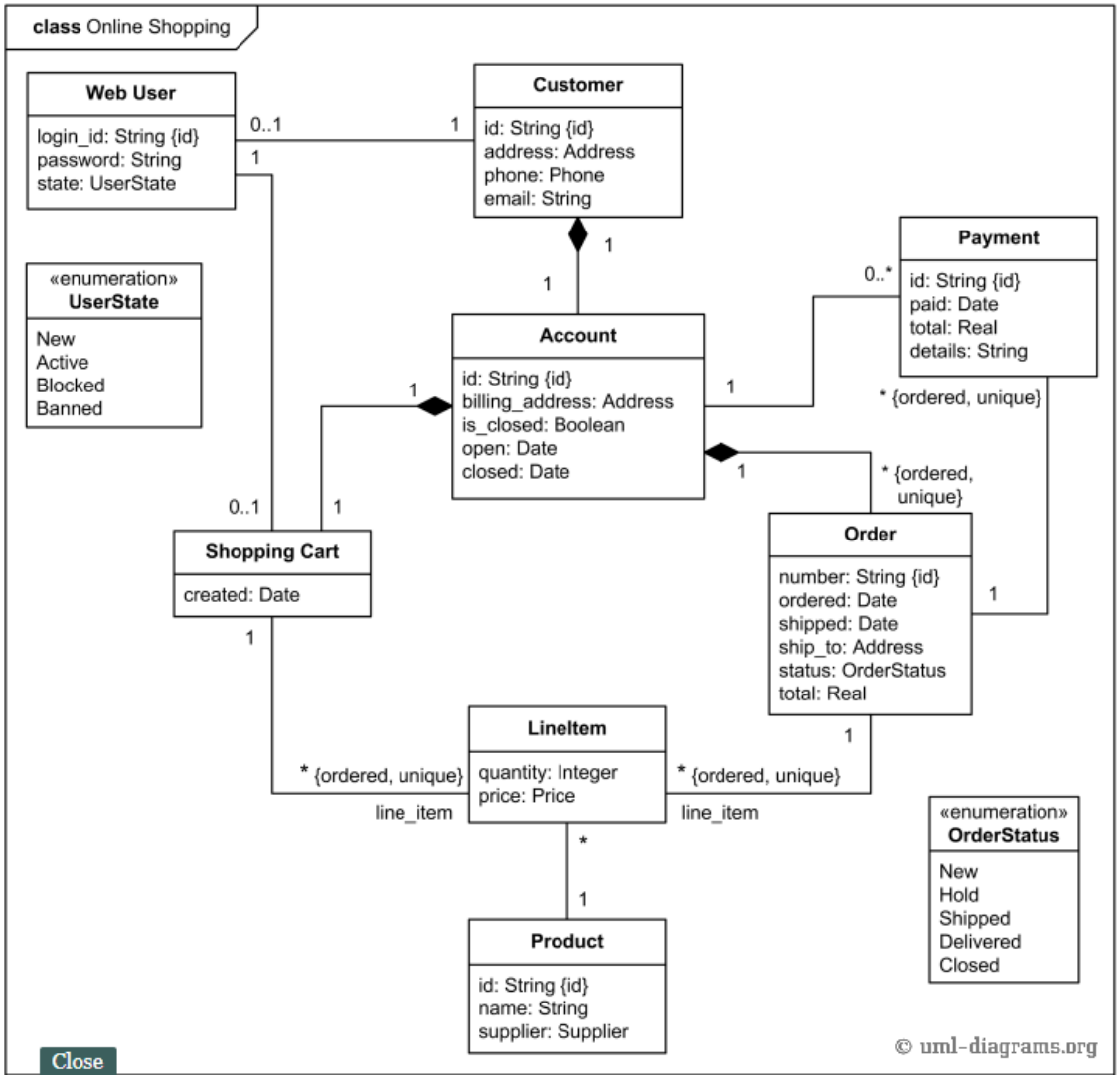

Figure A.9: Class Diagram For Online Shopping Application

A.4.3 Sequence Diagram

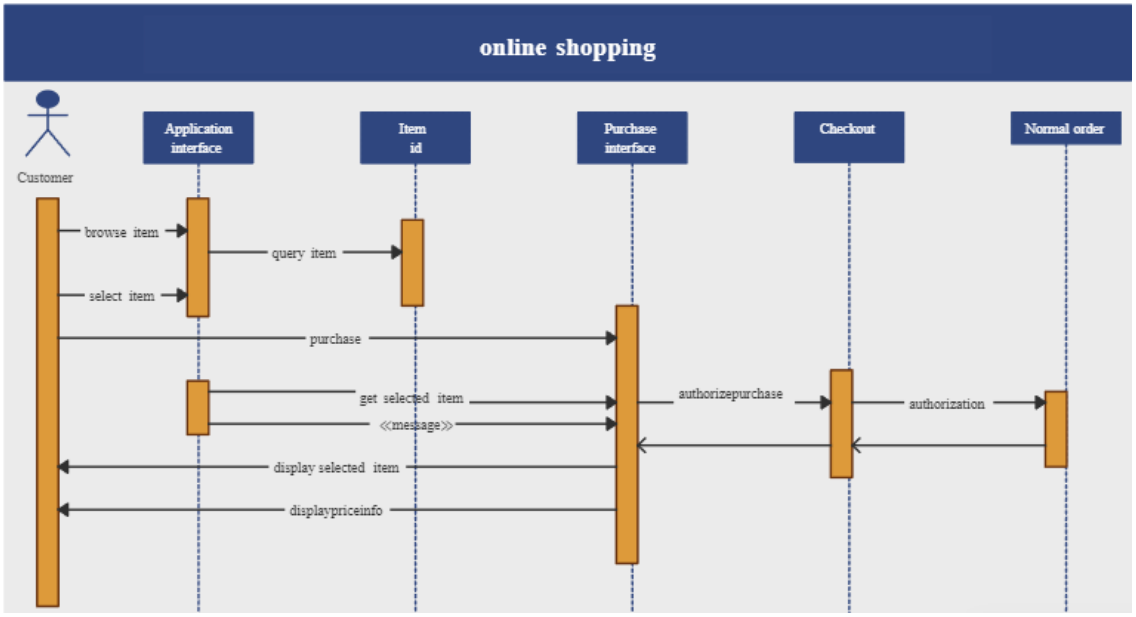

Figure A.10: Sequence Diagram For Online Shopping Application

A.4.4 State Transition Diagram

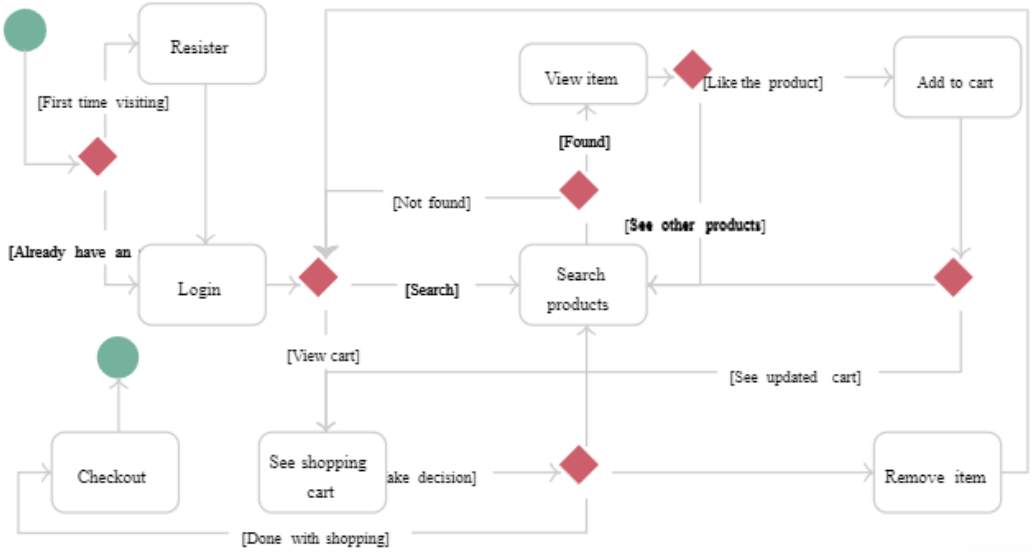

Figure A.11: State Transition Diagram For Online Shopping Application

A.4.5 Data Flow Diagram

Software Design Description for <Project> *Page 21*

Data Flow Diagram

Data flow diagram symbols, symbol names, and examples

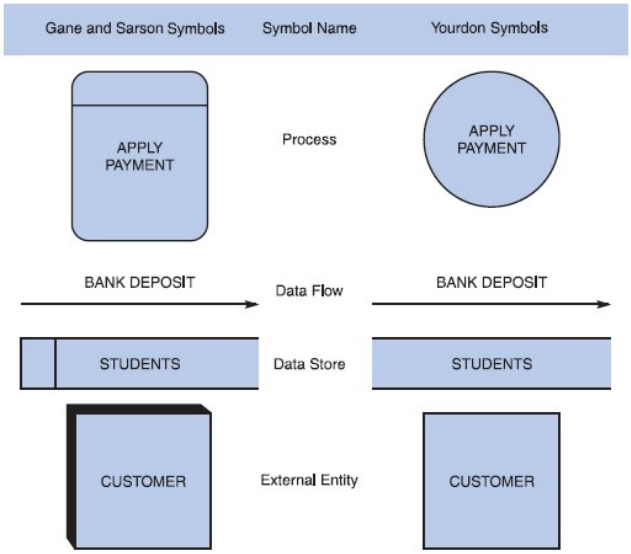

Figure B-5 Data flow diagram symbols, symbol names, and examples of the Gane and Sarson and Yourdon symbol sets.

Guidelines for Drawing DFDs

Step 1: Draw a Context Diagram: The first step in constructing a set of DFDs is to draw a context diagram. A **context diagram** is a top-level view of an information system that shows the system's boundaries and scope. Data stores are not shown in the context diagram because they are contained within the system and remain hidden until more detailed diagrams are created.

Example

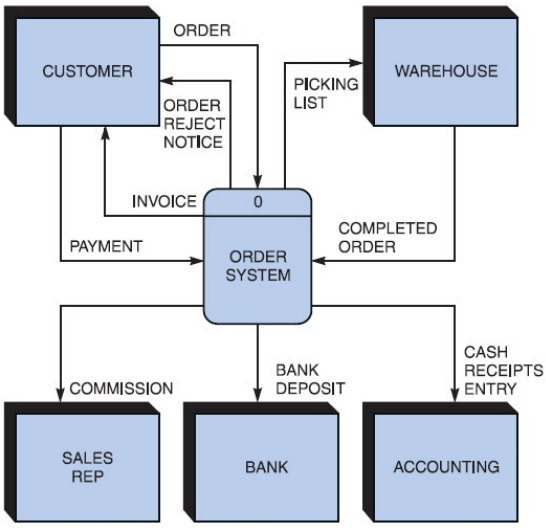

Figure B-6 Context diagram DFD for an order system.

Step 2: Draw a Diagram 0 DFD: To show the detail inside the black box, you create DFD diagram 0. **Diagram 0** zooms in on the system and shows major internal processes, data flows, and data stores. Diagram 0 also repeats the entities and data flows that appear in the context diagram. When you expand the context diagram into DFD diagram 0, you must retain all the connections that flow into and out of process 0.

Example

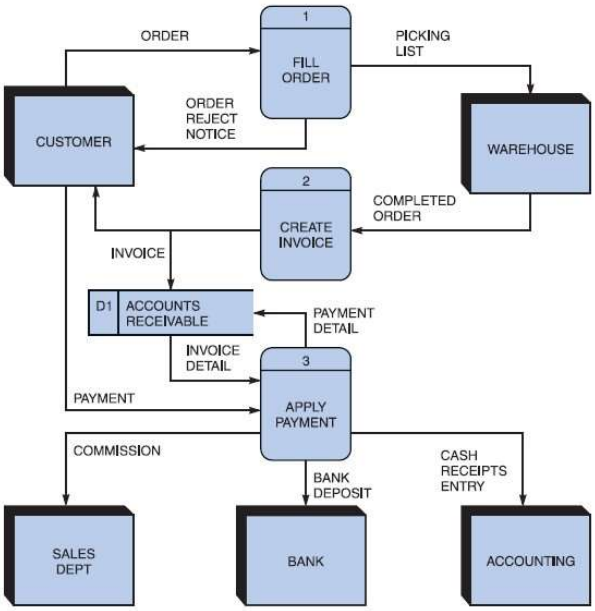

Figure B-7 Diagram 0 DFD for the order system.

Step 3: Draw the Lower-Level Diagrams:

To create lower-level diagrams, you must use leveling and balancing techniques. **Leveling** is the process of drawing a series of increasingly detailed diagrams, until all functional primitives are identified.

Leveling Example

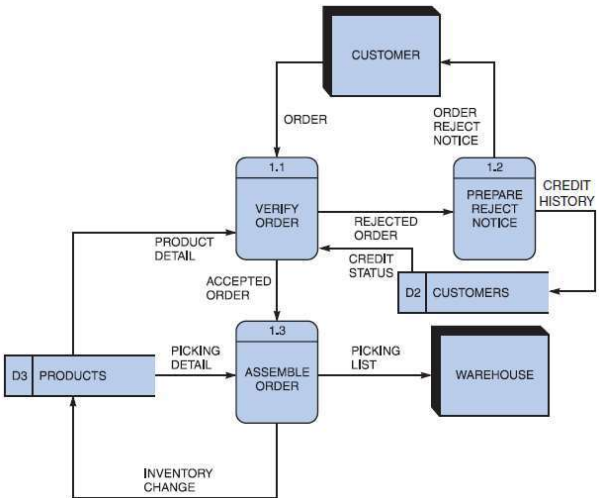

Figure B-8 Diagram 1 DFD shows details of the FILL ORDER process in the order system.